



模式识别与计算机视觉

Chapter 11: Recognition

王利民

媒体计算课题组

<http://mcg.nju.edu.cn/>



大作业要求



- 综述性研究报告：针对一个专门话题，进行深入调研分析，形成一篇高质量的综述。
- 培养：查阅文献，阅读文献，撰写文献的能力，可能启发对某个方向深入研究的兴趣。
- 提交格式：**CVPR**论文格式，**建议英文撰写**。
- 调研范围：顶级会议和顶级期刊论文，以及最新的arXiv论文，
- 不要仅仅去看一些公众号、知乎、中文博客
- 提交时间：**8月10号**



课程大纲 (基础部分)



- Chapter 1. Introduction
- Chapter 2. Images: Lighting and Color
- Chapter 3. Filtering
- Chapter 4. Frequency Domain and Sampling
- Chapter 5. Template, Pyramid, and Filter Banks
- Chapter 6. Edges
- Chapter 7. Segmentation and Grouping
- Chapter 8 & 9. Interest Points
- Chapter 10. Fitting and Alignment
- Chapter 11. Recognition
- Chapter 12. Detection

简介&图像基础

图像处理
底层视觉

中层视觉
边缘与分割

中层视觉
匹配与对齐

高层视觉
识别与检测



课程大纲（前沿部分）



- Topic1: Introduction to Deep Learning
- Topic2: CNN and Transformer Architectures
- Topic3: Tracking in the Wild
- Topic4: Video Understanding: Action and Grounding
- Topic5: Fine-grained Video Understanding
- Topic6: Advances in Video MLLM
- Topic7: Advances in Visual Generation
- Topic8: Advances in Skill Learning and Embodied Intelligence



Today' class



- Overview of image categorization
- Spatial pyramids bag-of-words categorizer
- Deep convolutional neural networks (CNNs)
- Object detection



What do you see in this image?



Forest



Categorization: describe, predict, or interact



Is it **dangerous**?

Is it **alive**?

How **fast** does it run?

Is it **soft**?

Does it have a **tail**?

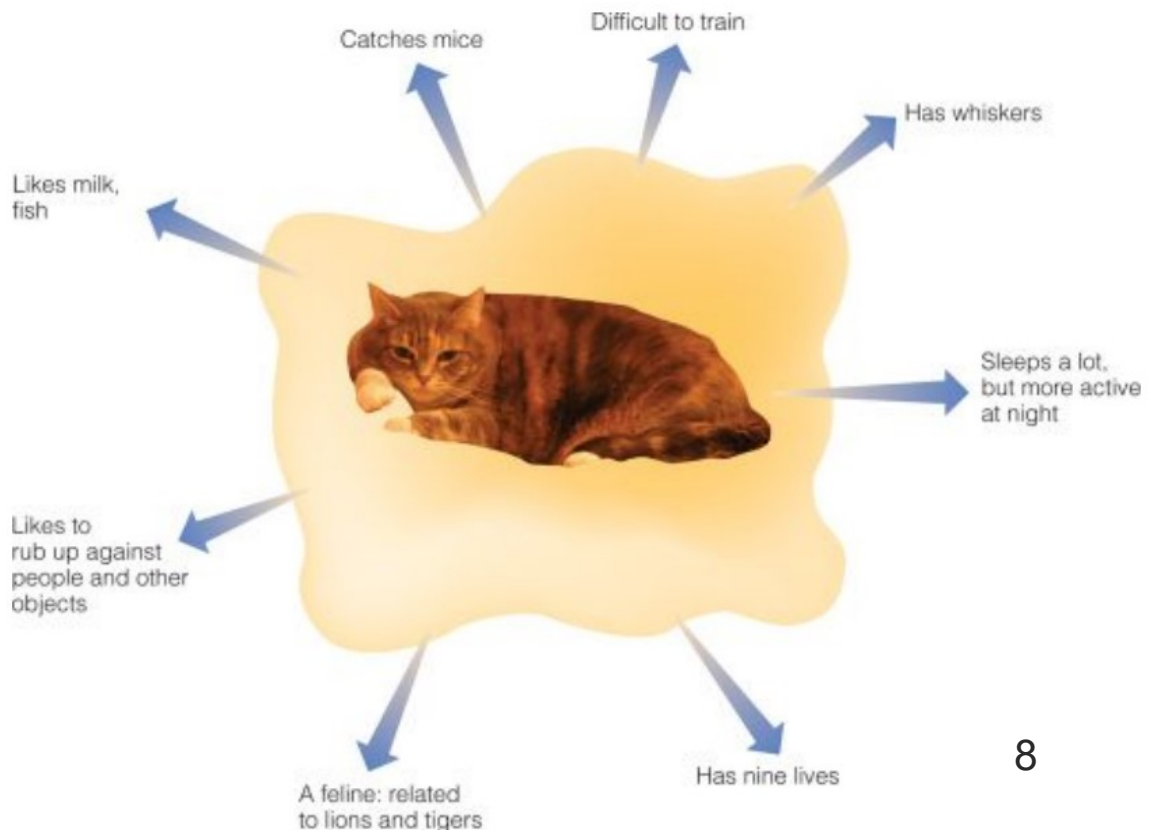
Can I **poke with it**?



Why do we care about categories?



- Predict behavior, beyond of what is immediately perceived
- Pointers to knowledge
 - Help to understand individual cases not previously encountered
- Communication



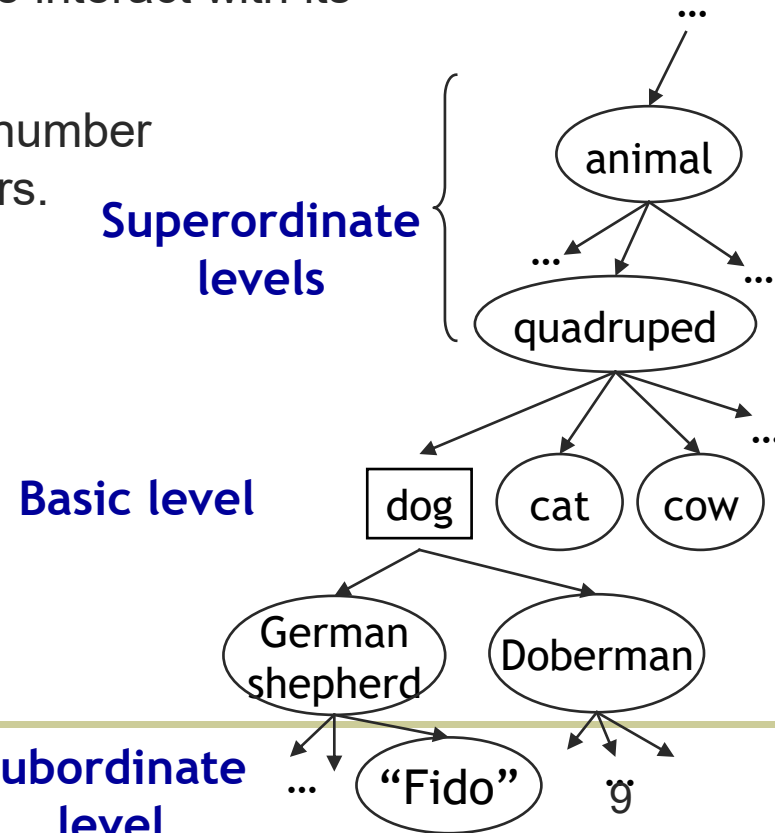
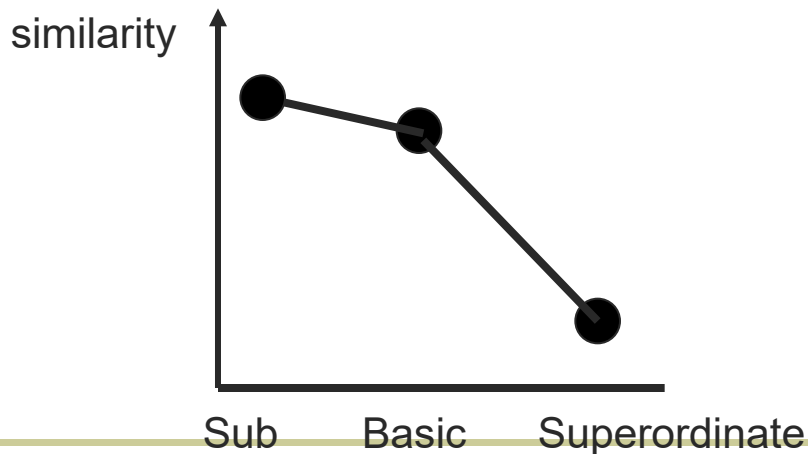


Levels of categorization [Rosch 70s]



Definition of Basic Level:

- **Similar shape:** Basic level categories are the highest-level category for which their members have similar shapes.
- **Similar motor interactions:** ... for which people interact with its members using similar motor sequences.
- **Common attributes:** ... there are a significant number of attributes in common between pairs of members.





Theory of categorization



How do we determine if something is a member of a particular category?

- Definitional approach
- Prototype approach
- Exemplar approach



Definitional approach: classical view of categories

■ Plato & Aristotle

- Categories are defined by a list of properties shared by all elements in a category
- Category membership is binary
- Every member in the category is equal



Aristotle by Francesco Hayez

The Categories (Aristotle)

Slide Credit: A. A. Efros



Prototype Model

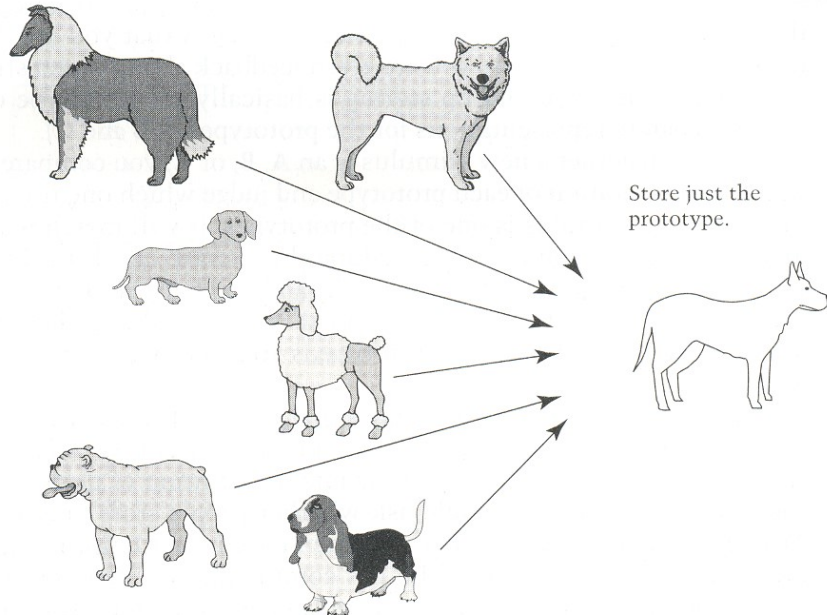


Figure 7.3. Schematic of the prototype model. Although many exemplars are seen, only the prototype is stored. The prototype is updated continually to incorporate more experience with new exemplars.

Category judgments are made by comparing a new exemplar to the prototype.

Exemplars Model

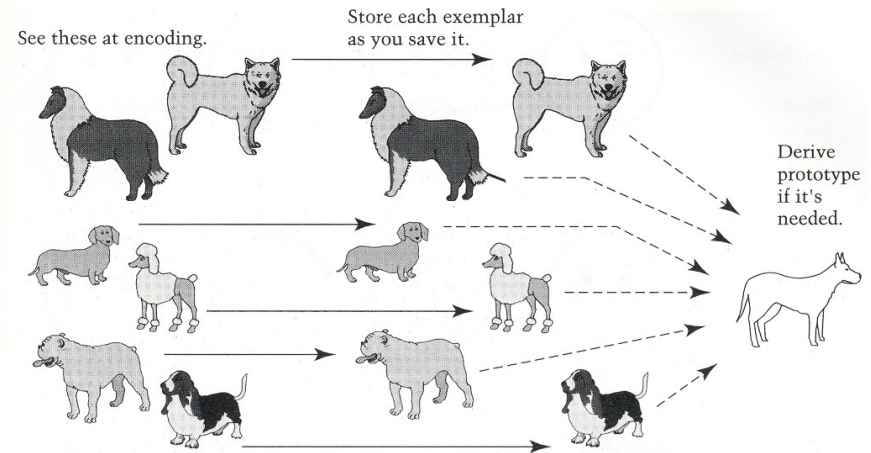


Figure 7.4. Schematic of the exemplar model. As each exemplar is seen, it is encoded into memory. A prototype is abstracted only when it is needed, for example, when a new exemplar must be categorized.

Category judgments are made by comparing a new exemplar to all the old exemplars of a category or to the exemplar that is the most appropriate



Image categorization



■ Cat vs Dog





Image categorization



■ Object recognition

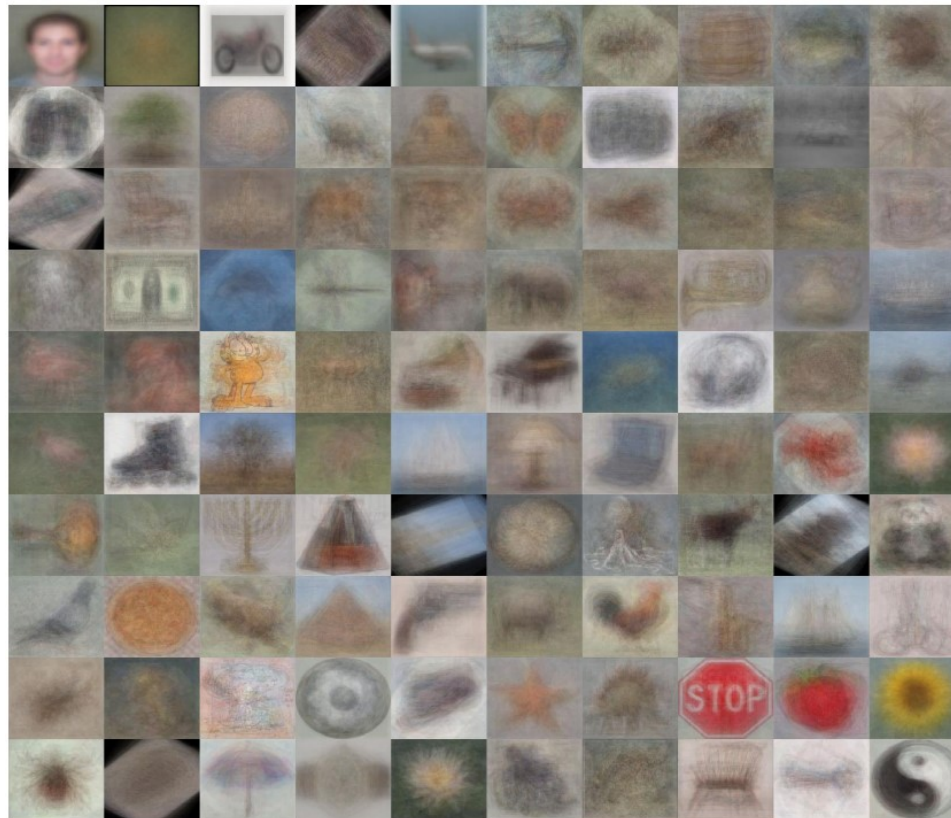




Image categorization



■ Fine-grained recognition



Generalist



Insect catching



Grain eating



Coniferous-seed eating



Nectar feeding



Chiseling



Dip netting



Surface skimming



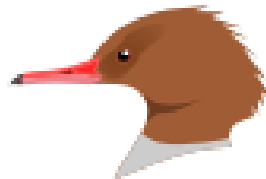
Scything



Probing



Aerial fishing



Pursuit fishing



Scavenging



Raptorial



Filter feeding



Image categorization



■ Place recognition





Image categorization



■ Visual font recognition

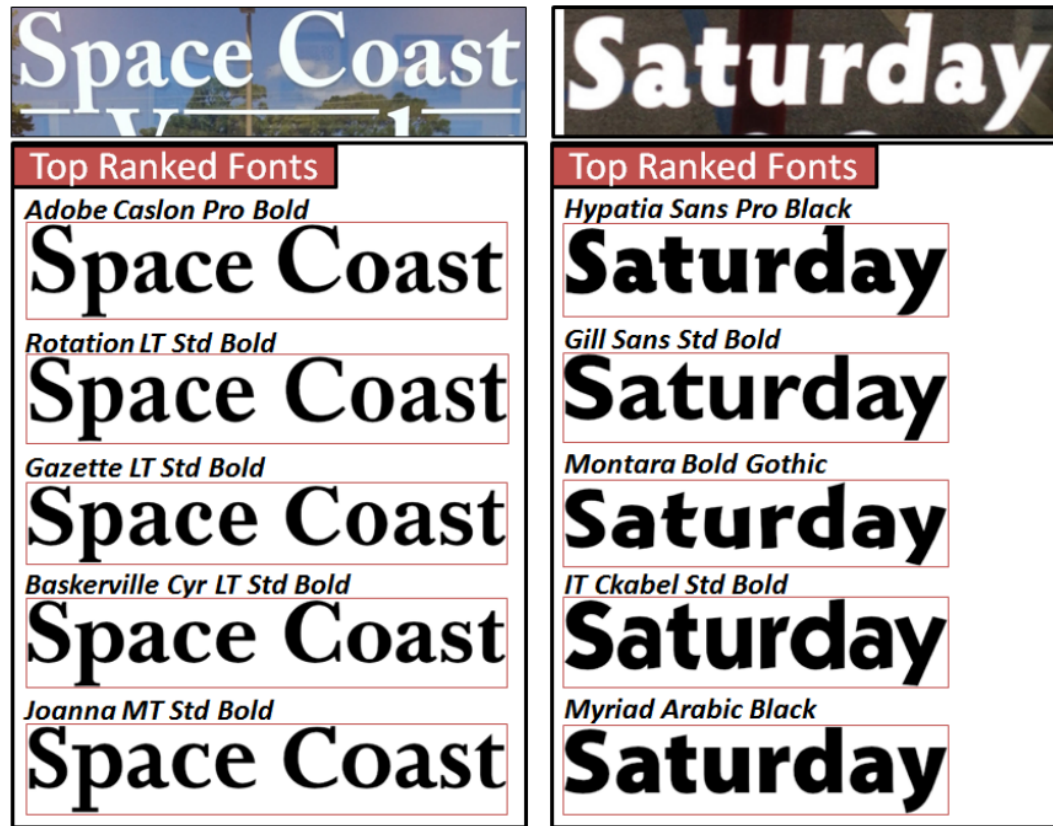




Image categorization



■ Image style recognition



HDR



Macro



Baroque



Rococo



Vintage



Noir



Northern Renaissance



Cubism



Minimal



Hazy



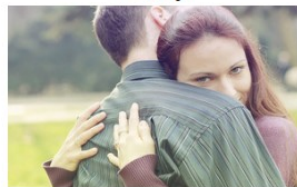
Impressionism



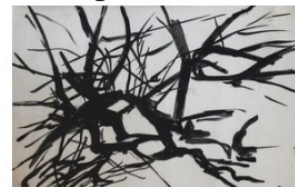
Post-Impressionism



Long Exposure



Romantic



Abs. Expressionism



Color Field Painting

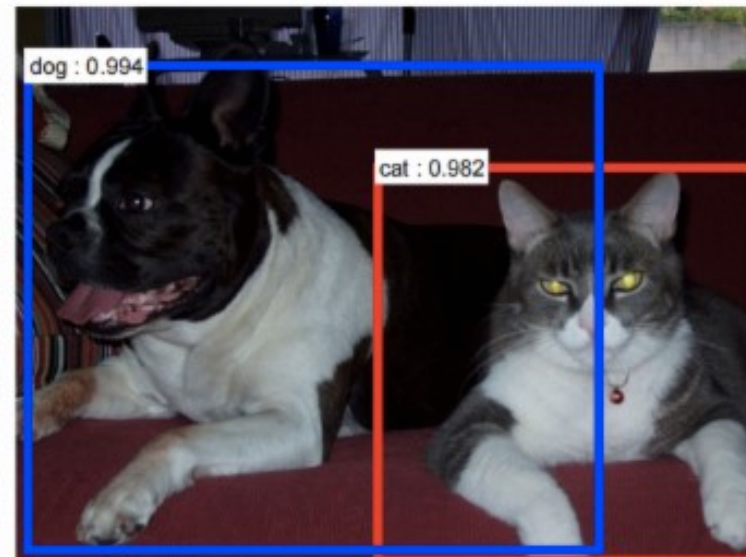
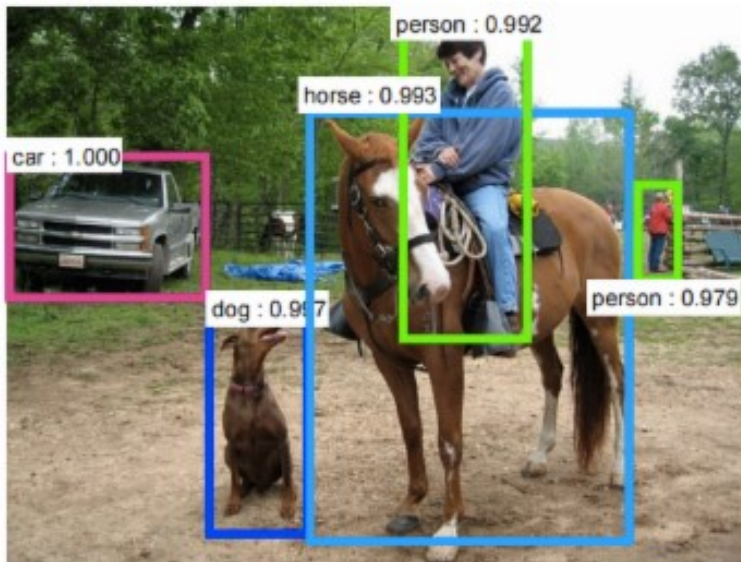
Flickr Style: 80K images covering 20 styles.

Wikipaintings: 85K images for 25 art genres.

[[Karayev et al. BMVC 2014](#)]



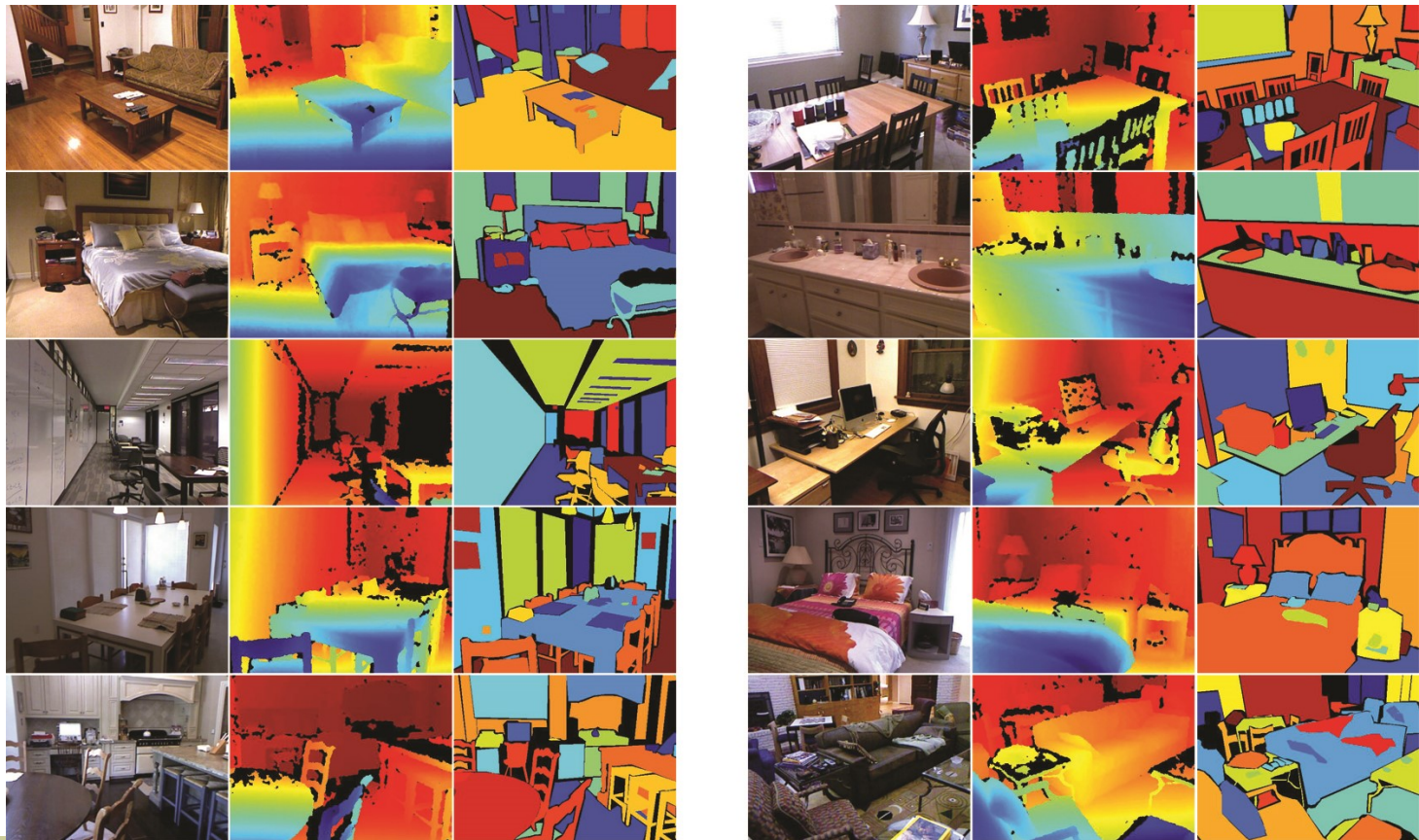
Object Category Detection



Region categorization



■ Semantic segmentation from RGBD images

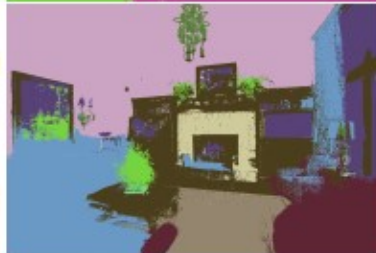
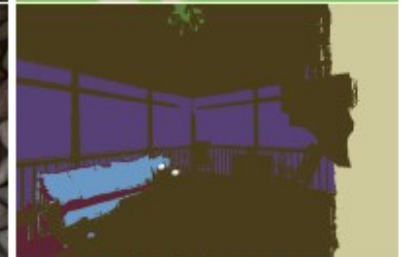




Region categorization



brick	food	painted	tile
carpet	glass	paper	stone
ceramic	hair	plastic	water
fabric	leather	polishedstone	wood
foliage	metal	skin	

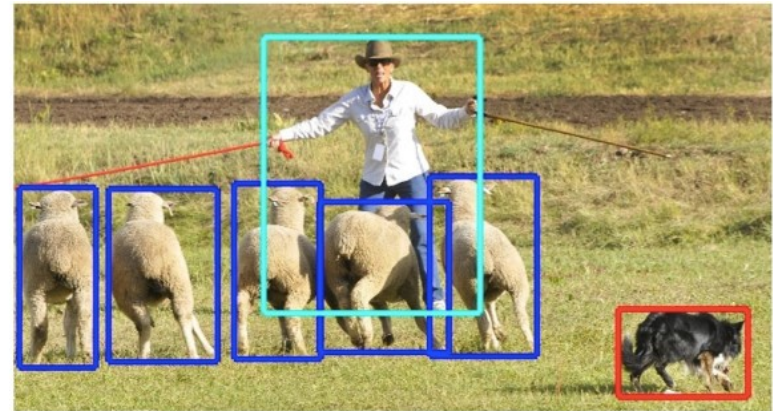




Detection, semantic segmentation, instance segmentation



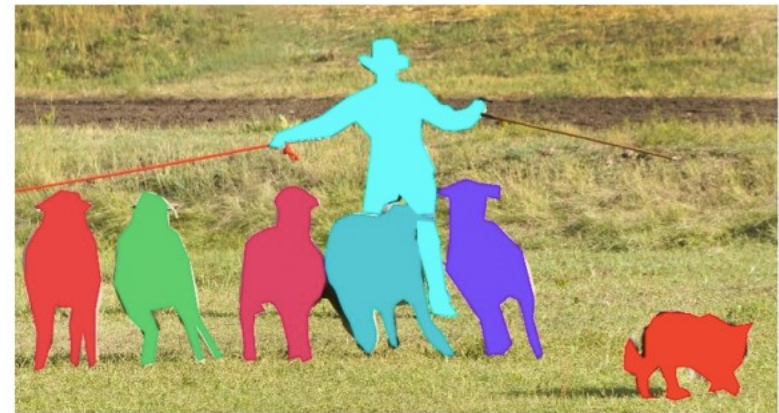
image classification



object detection



semantic segmentation



instance segmentation



Categorization from supervised learning



Examples

+

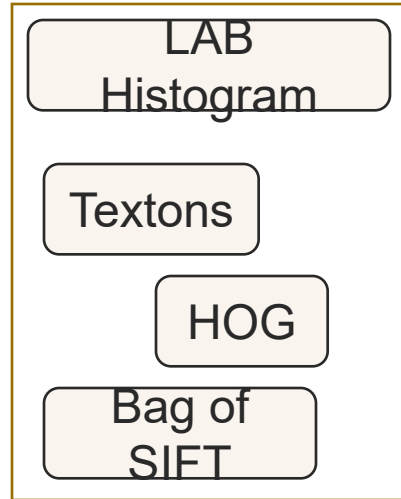
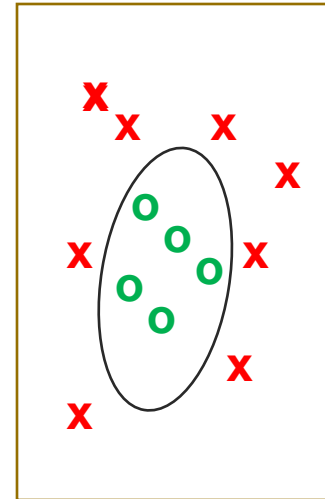


Image Features

+

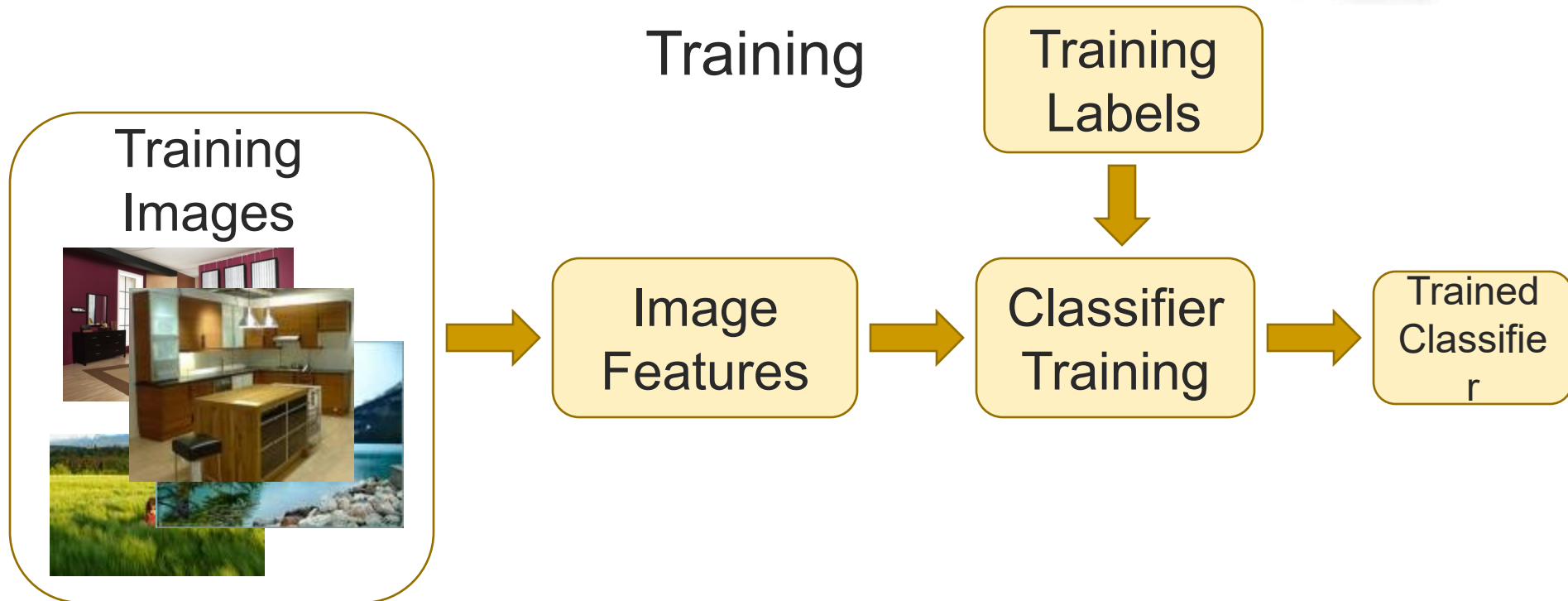


Classifier

= Category label



Training phase





Testing phase



Training

Training Labels

Training Images

Image Features

Classifier Training

Trained Classifier

Testing

Image Features

Trained Classifier

Prediction
Outdoor

Test Image



Today' class



- Overview of image categorization
- **Spatial pyramids bag-of-words categorizer**
- Deep convolutional neural networks (CNNs)
- Object detection



Example: Spatial Pyramid BoW Classifier

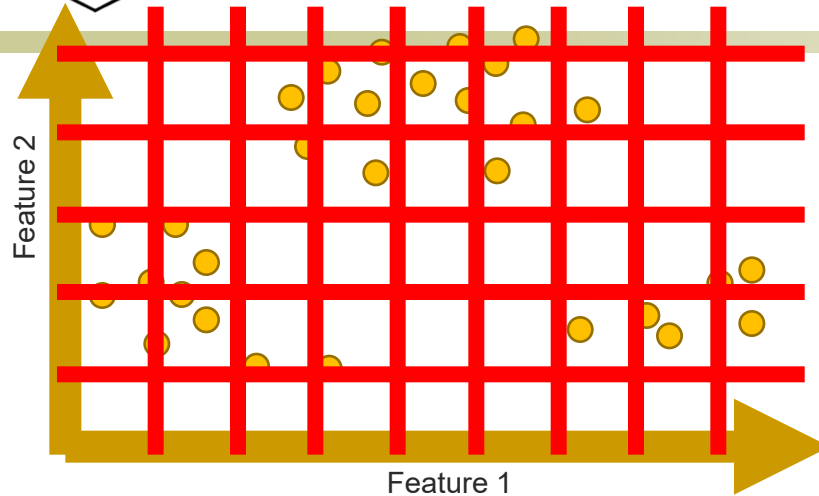


- Features: spatially binned histograms of clustered SIFT descriptors
- Classifier: SVM



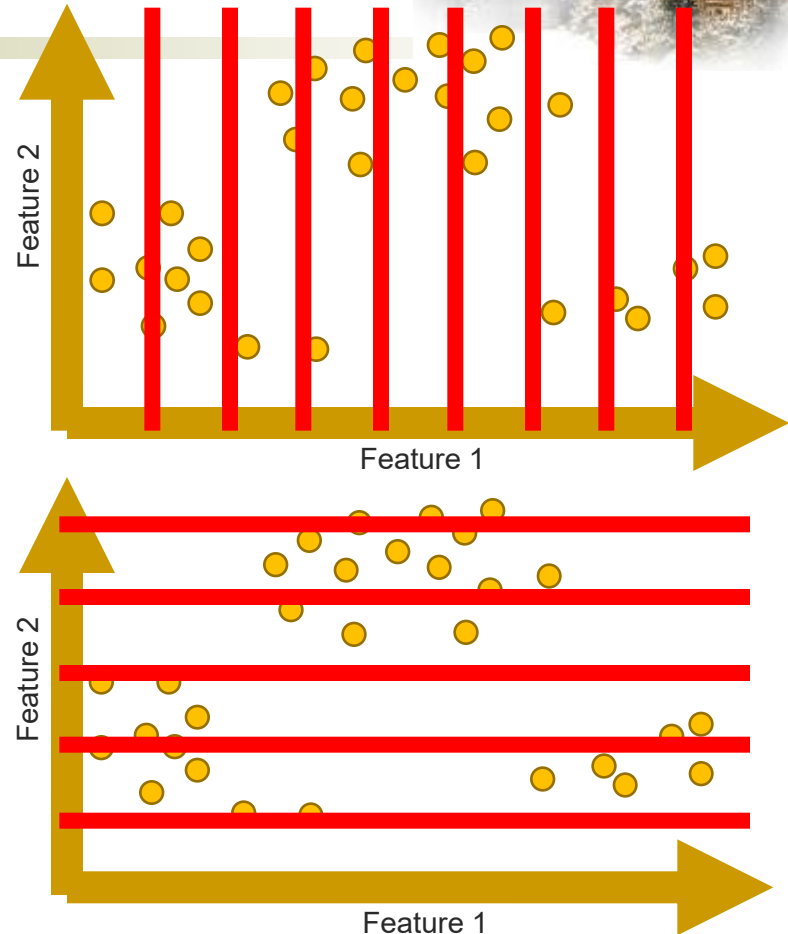


Joint vs marginal binning



Joint histogram

- Requires lots of data
- Loss of resolution to avoid empty bins



Marginal histogram

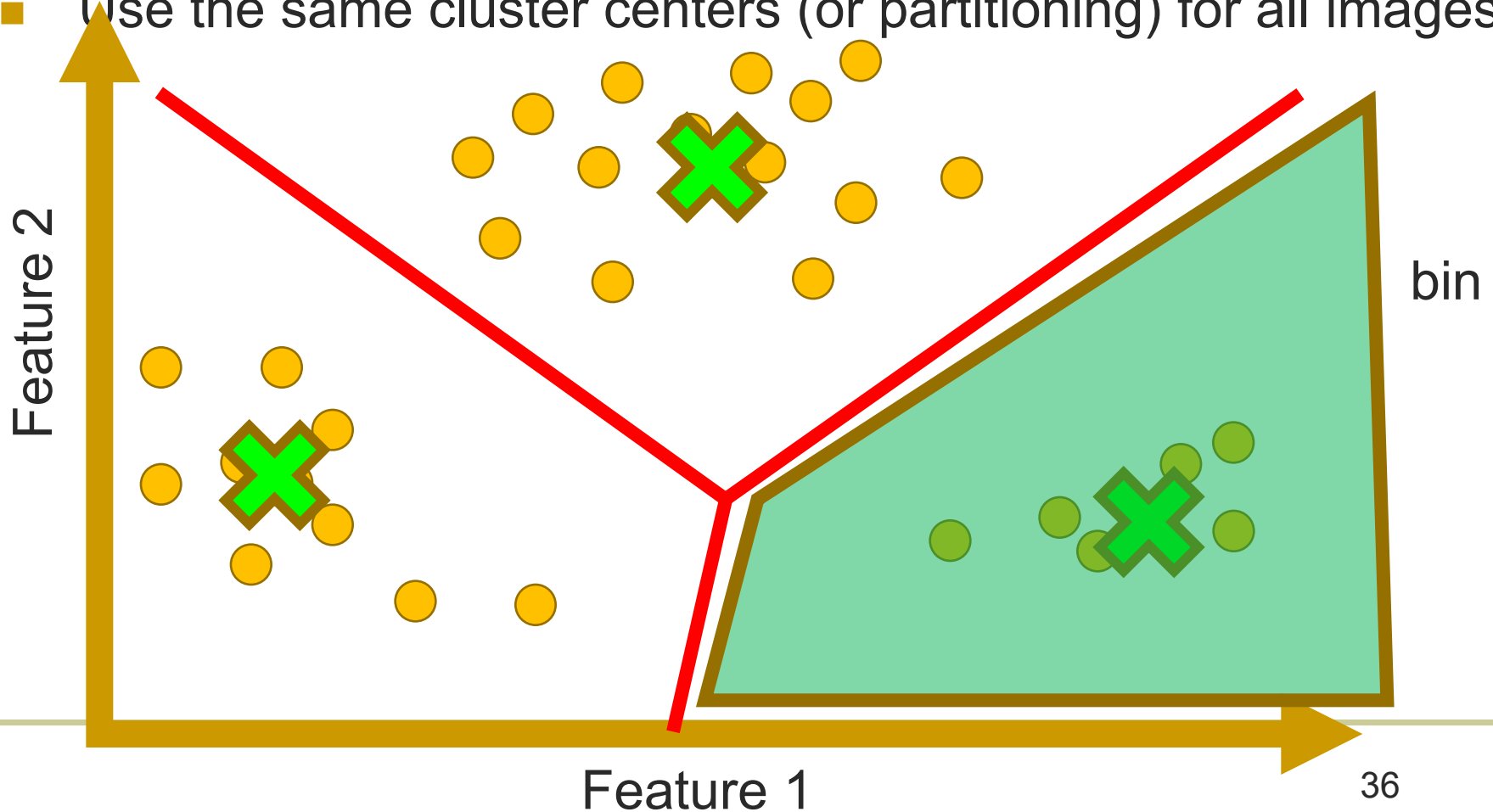
- Requires independent features
- More data/bin than joint histogram



Histogram with clustering



- Cluster data (or partition) into K clusters, count how many samples appear in each cluster to get K -dim histogram
- Use the same cluster centers (or partitioning) for all images





Computing histogram distance



- Cosine similarity (dot product of normalized counts)
- Histogram intersection

$$\text{histint}(h_i, h_j) = 1 - \sum_{m=1}^K \min(h_i(m), h_j(m))$$

- Chi-squared Histogram matching distance

$$\chi^2(h_i, h_j) = \frac{1}{2} \sum_{m=1}^K \frac{[h_i(m) - h_j(m)]^2}{h_i(m) + h_j(m)}$$

- Earth mover's distance (Cross-bin similarity measure)

- minimal cost paid to transform one distribution into the other



Histograms: implementation issues



■ Quantization

- Grids: fast but applicable only with few dimensions
- Clustering: slower but can quantize data in higher dimensions



Few Bins

Need less data

Coarser representation

Many Bins

Need more data

Finer representation

■ Matching

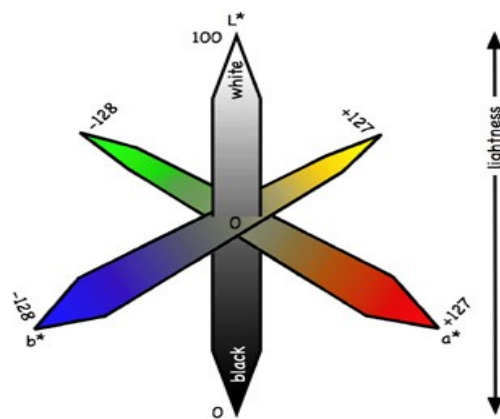
- Histogram intersection or Euclidean/Cosine may be faster
- Chi-squared often works better
- Earth mover's distance is good for when nearby bins represent similar values



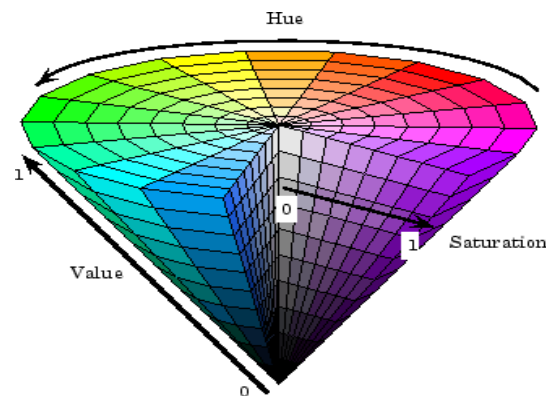
What kind of things do we compute histograms of?



■ Color

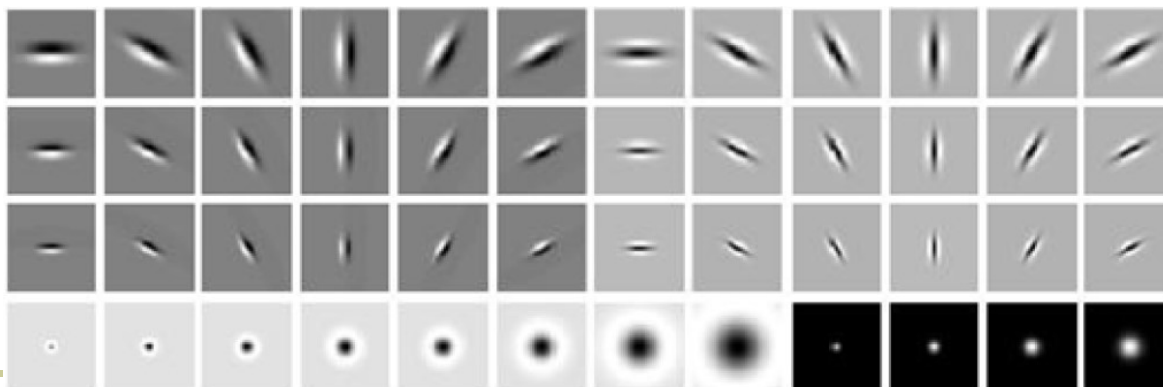


L*a*b* color space



HSV color space

■ Texture (filter banks or HOG over regions)





What kind of things do we compute histograms of?



■ Histograms of descriptors

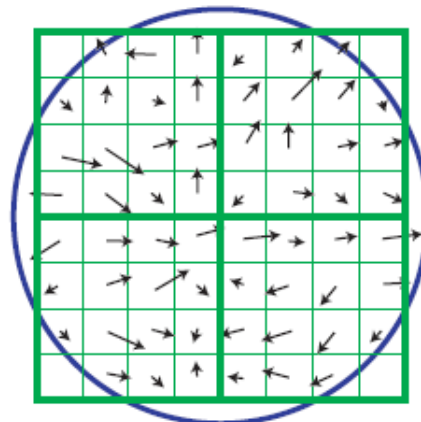
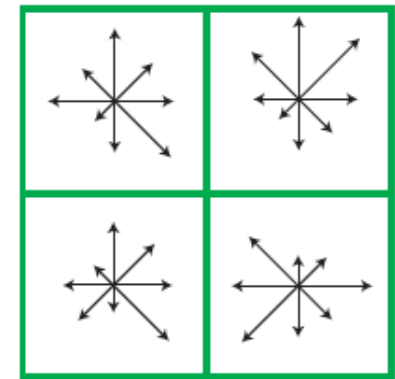


Image gradients



Keypoint descriptor

SIFT – [Lowe IJCV 2004]

■ “Bag of visual words”



What are the right features?



Depend on what you want to know!

- Object: shape

- Local shape info, shading, shadows, texture

- Scene : geometric layout

- linear perspective, gradients, line segments

- Material properties: albedo, feel, hardness

- Color, texture

- Action: motion

- Optical flow, tracked points



General principles of features



■ Coverage

- Ensure that all relevant info is captured

■ Concision

- Minimize number of features without sacrificing coverage

■ Directness

- Ideal features are independently useful for prediction



Image categorization with BoW



Training

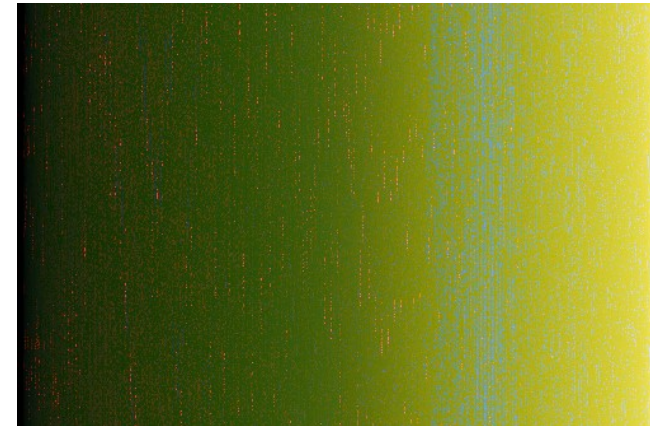
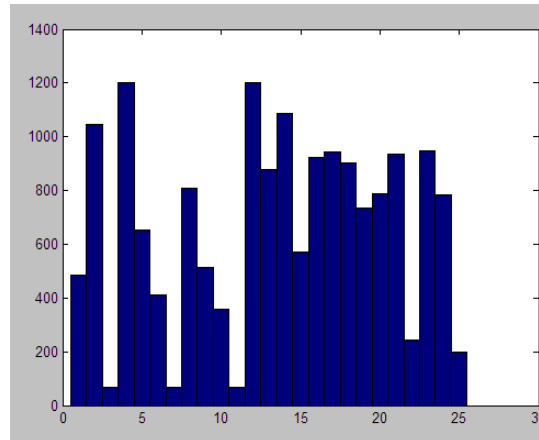
1. Extract keypoints and descriptors for all training images
2. Cluster descriptors
3. Quantize descriptors using cluster centers to get “visual words”
4. Represent each image by normalized counts of “visual words”
5. Train classifier on labeled examples using histogram values as features

Testing

1. Extract keypoints/descriptors and quantize into visual words
2. Compute visual word histogram
3. Compute label or confidence using classifier



But what about spatial layout?



All of these images have the same color histogram



Spatial pyramid



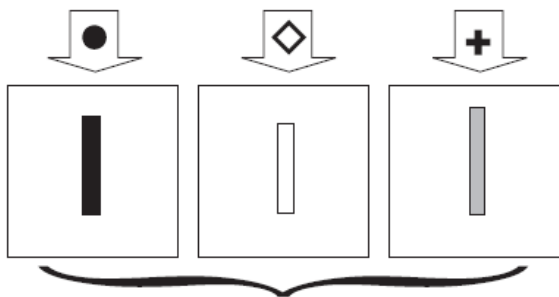
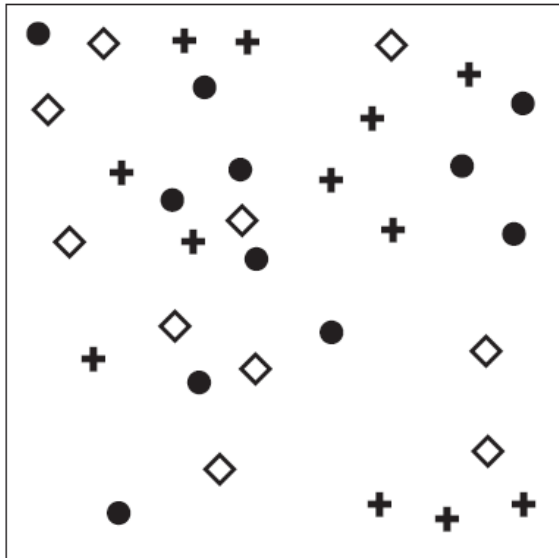
Compute histogram in each spatial bin



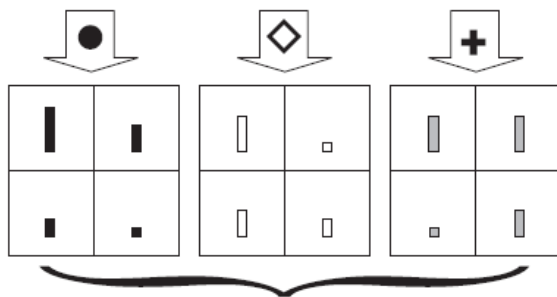
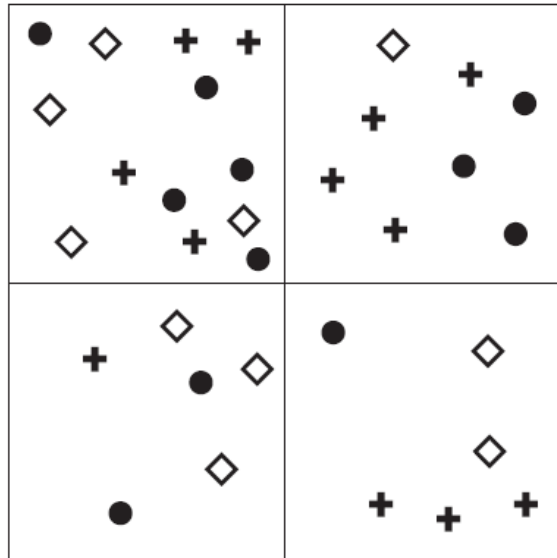
Spatial pyramid



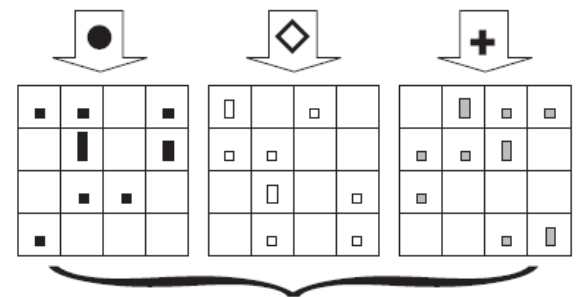
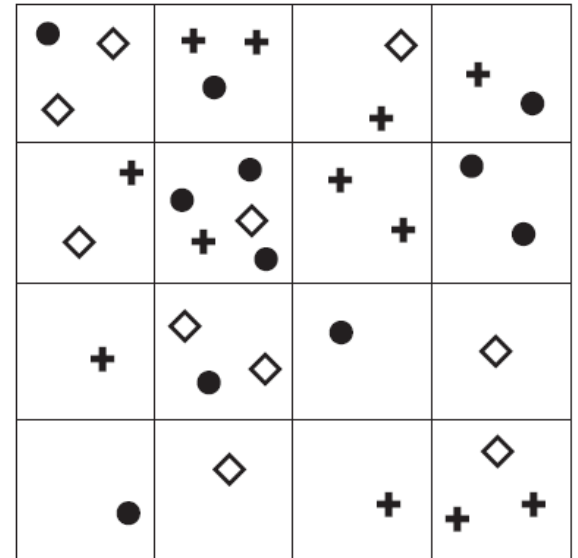
level 0



level 1



level 2

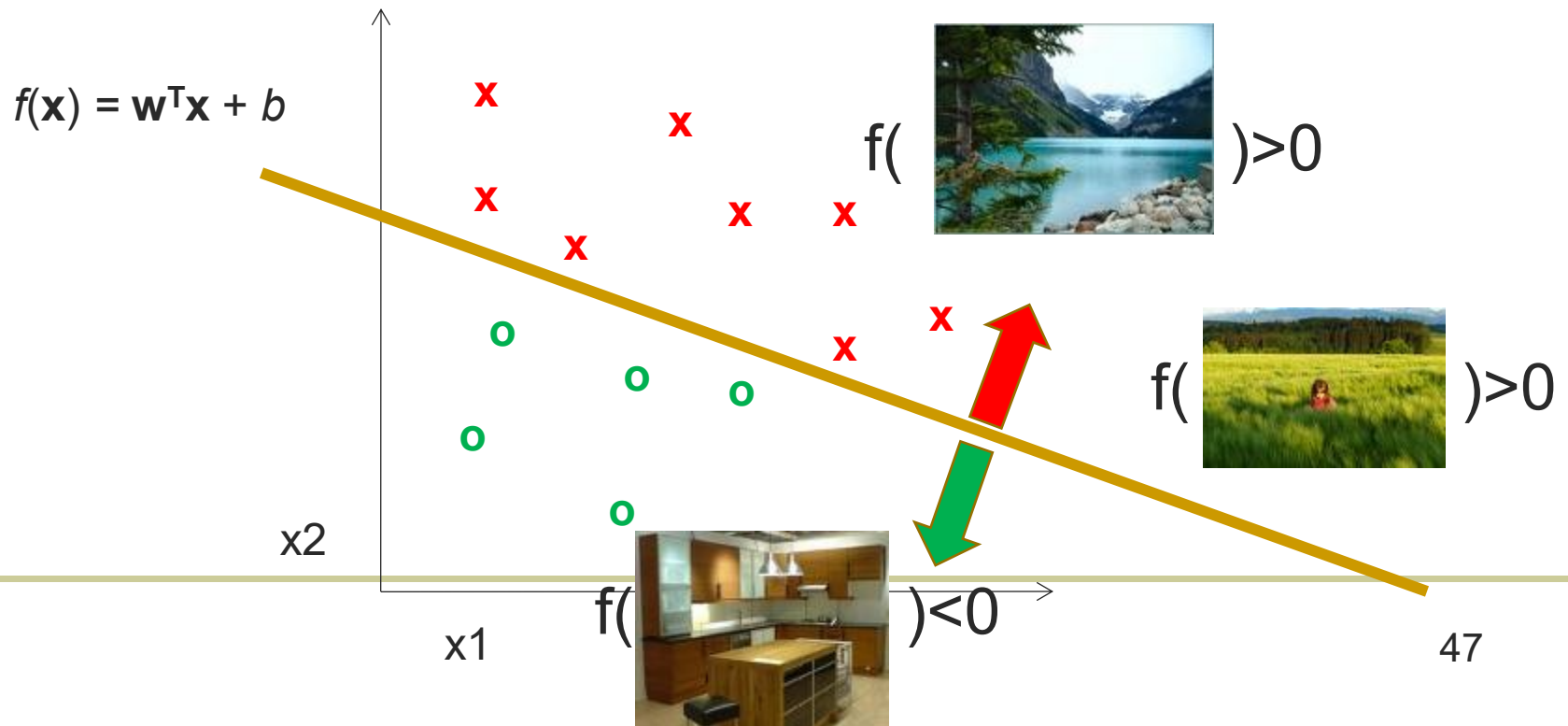




Linear SVM classifier



Find the hyperplane that separate examples of different categories

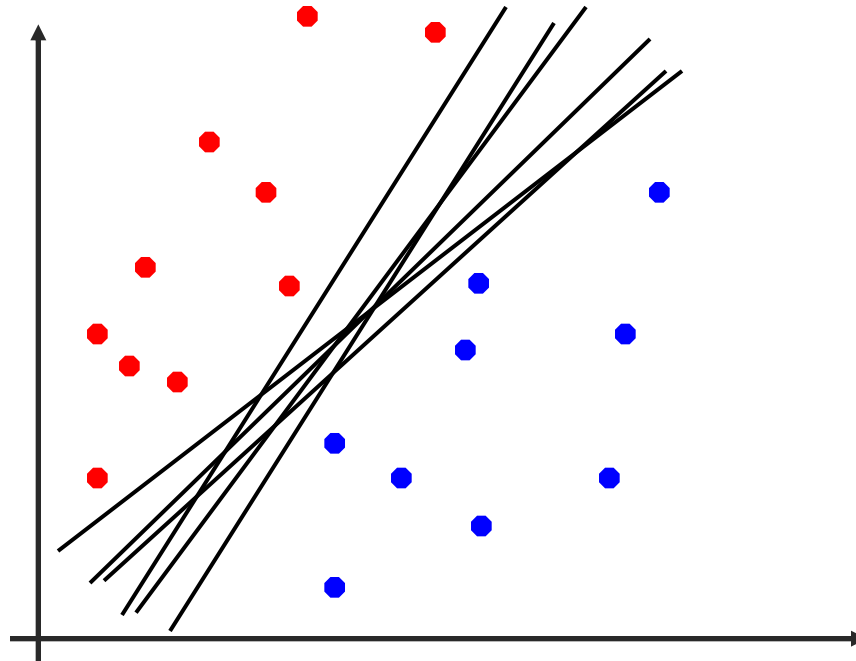




Linear Separators



- Which of the linear separators is best?

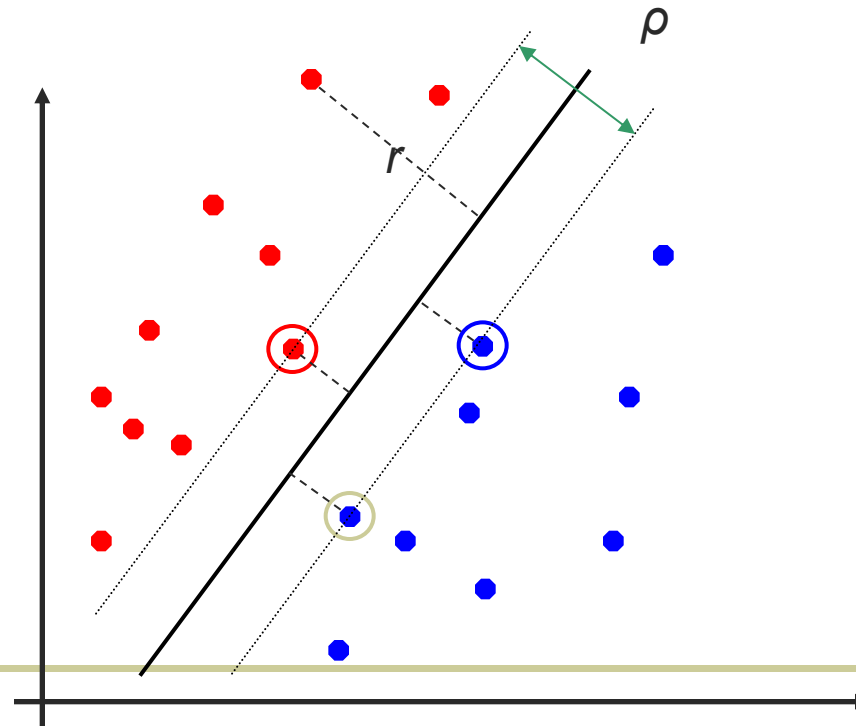




Classification Margin



- Distance from example $\mathbf{x}_i$ to the separator is $r = \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|}$
- Examples closest to the hyperplane are **support vectors**.
- **Margin** ρ of the separator is the distance between support vectors.

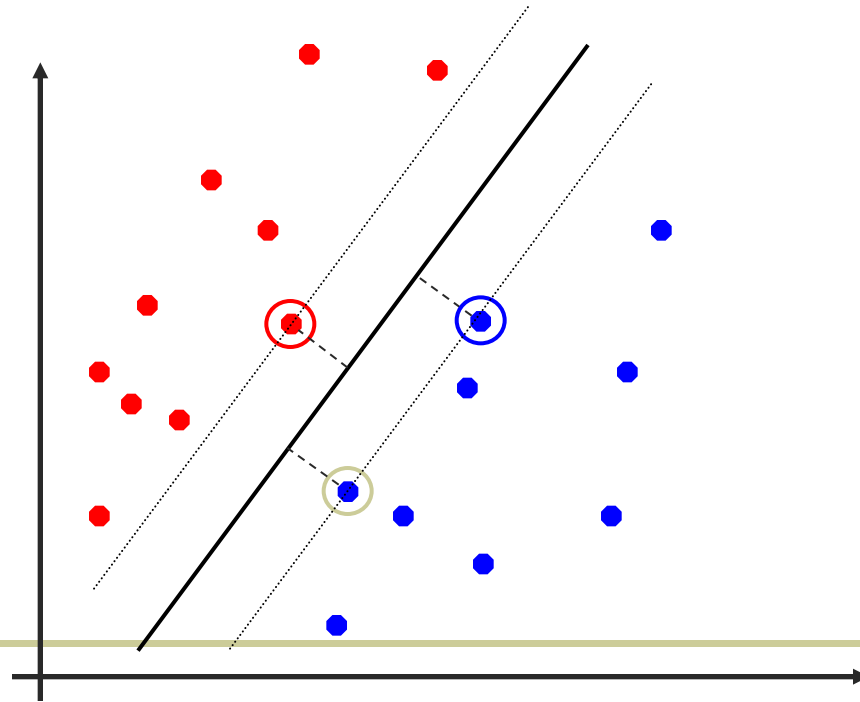




Maximum Margin Classification



- Implies that only support vectors matter; other training examples are ignorable.





Linear SVM Formulation



Find weights that try to get all examples right with a confidence margin while limiting the L2 norm of the weight vector

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \leftarrow \text{Loss and regularization}$$

such that

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

Parameter C trades off between cost of hard examples and penalty on having large feature weights



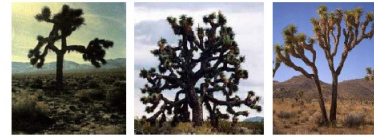
Spatial Pyramids Results



minaret (97.6%)



windsor chair (94.6%)



joshua tree (87.9%)



okapi (87.8%)



cougar body (27.6%)



beaver (27.5%)



crocodile (25.0%)



ant (25.0%)

	Weak features		Strong features (200)	
L	Single-level	Pyramid	Single-level	Pyramid
0	15.5 $\pm$ 0.9		41.2 $\pm$ 1.2	
1	31.4 $\pm$ 1.2	32.8 $\pm$ 1.3	55.9 $\pm$ 0.9	57.0 $\pm$ 0.8
2	47.2 $\pm$ 1.1	49.3 $\pm$ 1.4	63.6 $\pm$ 0.9	64.6 $\pm$ 0.8
3	52.2 $\pm$ 0.8	54.0 $\pm$ 1.1	60.3 $\pm$ 0.9	64.6 $\pm$ 0.7

Table 2. Classification results for the Caltech-101 database.



Today' class



- Overview of image categorization
- Spatial pyramids bag-of-words categorizer
- **Deep convolutional neural networks (CNNs)**
- Object detection



Categorization involves **features** and a classifier



Training
Images



Training

Training
Labels

Image
Features

Classifier
Training

Trained
Classifier

Testing

Image
Features

Trained
Classifier

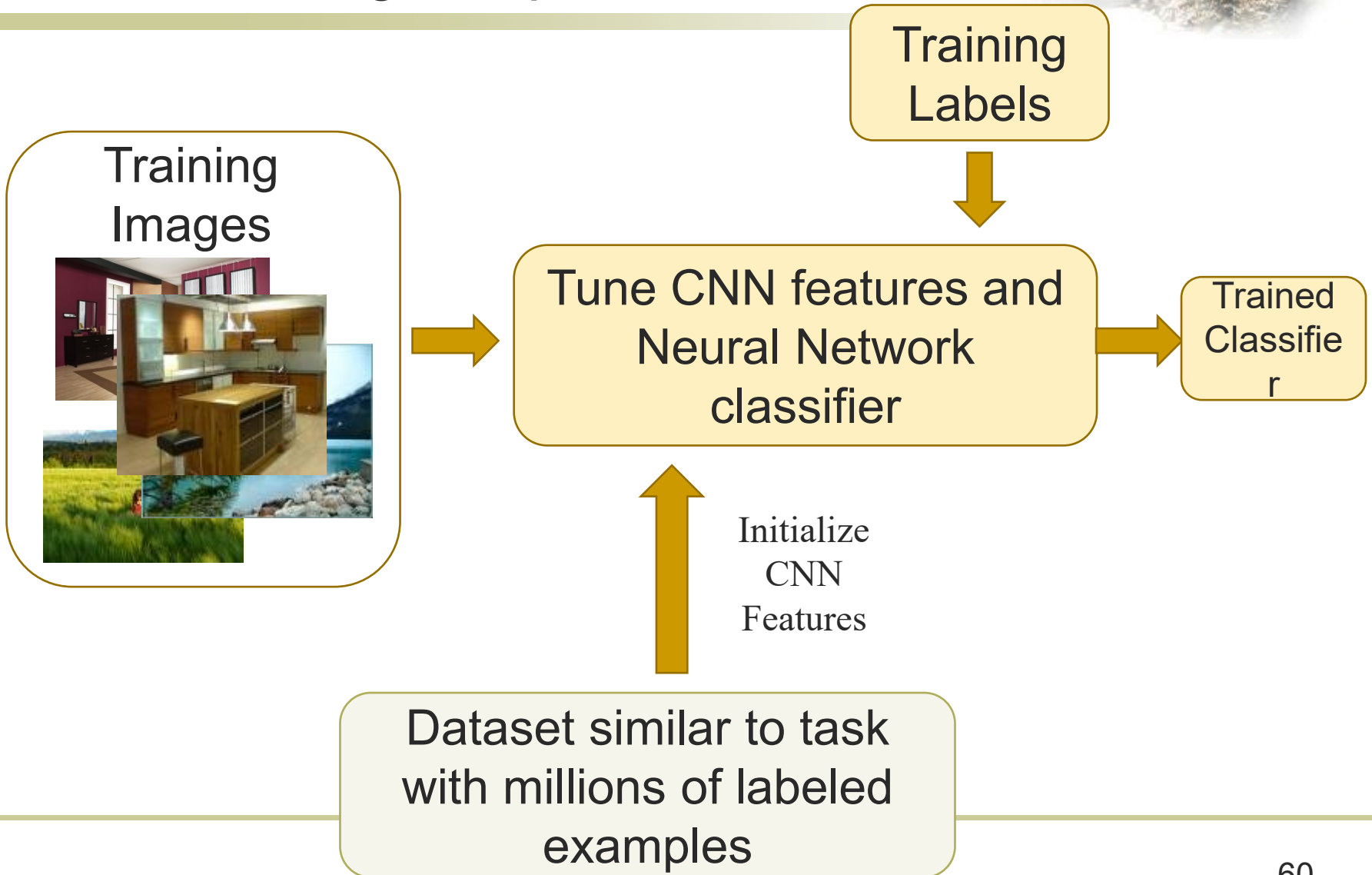
Prediction
Outdoor

Test Image





New training setup with moderate sized datasets





History of deep convolutional nets

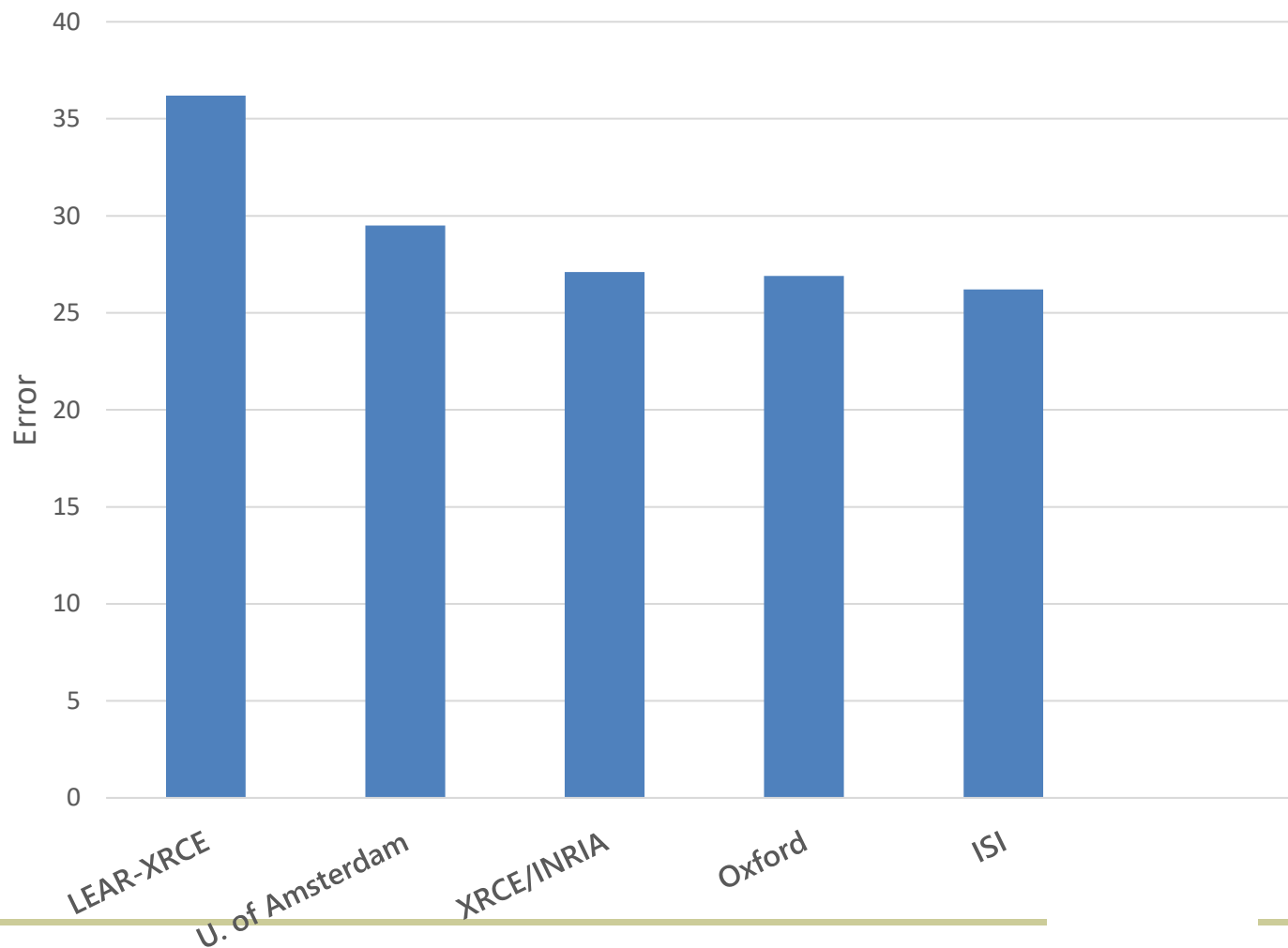


- 1950's: neural nets (perceptron) invented by Rosenblatt
- 1980's/1990's: Neural nets are popularized and then abandoned as being interesting idea but impossible to optimize or “unprincipled”
- 1990's: LeCun achieves state-of-art performance on character recognition with convolutional network (main ideas of today's networks)
- 2000's: Hinton, Bottou, Bengio, LeCun, Ng, and others keep trying stuff with deep networks but without much traction/acclaim in vision
- 2010-2011: Substantial progress in some areas, but vision community still unconvinced
 - Some neural net researchers get **ANGRY** at being ignored/rejected
- 2012: shock at ECCV 2012 with ImageNet challenge



2012 ImageNet 1K

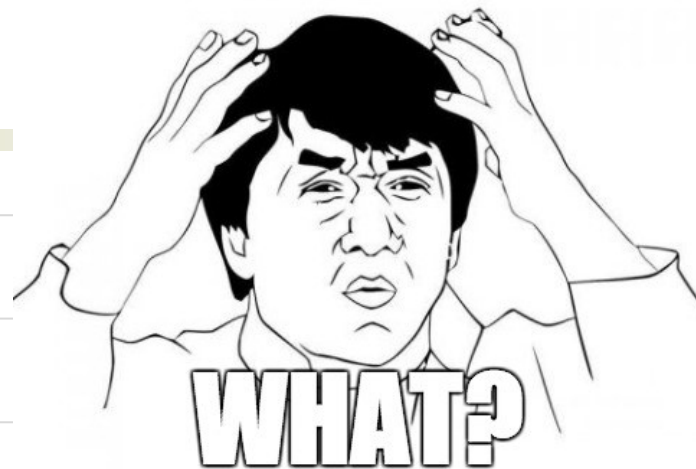
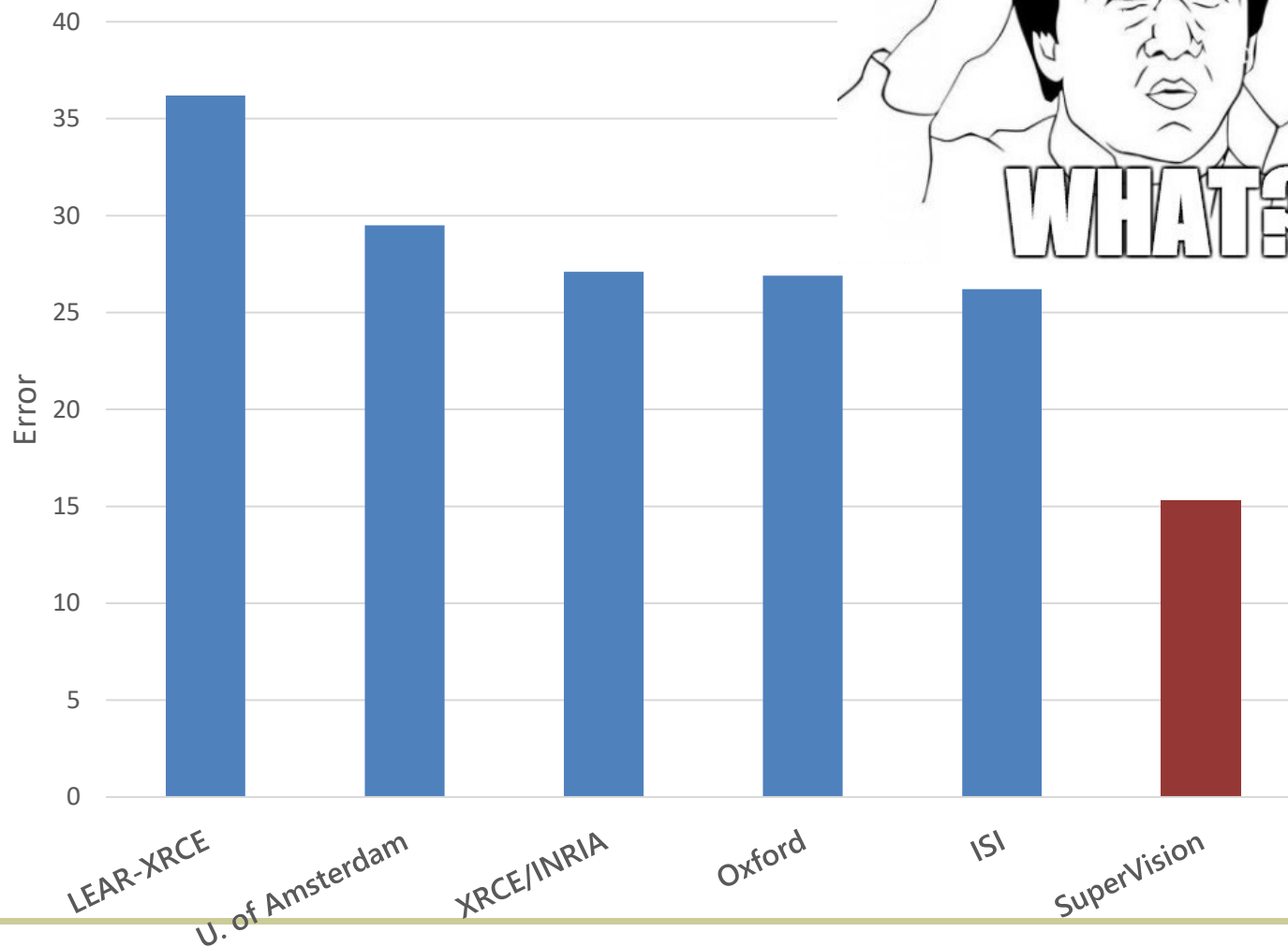
(Fall 2012)





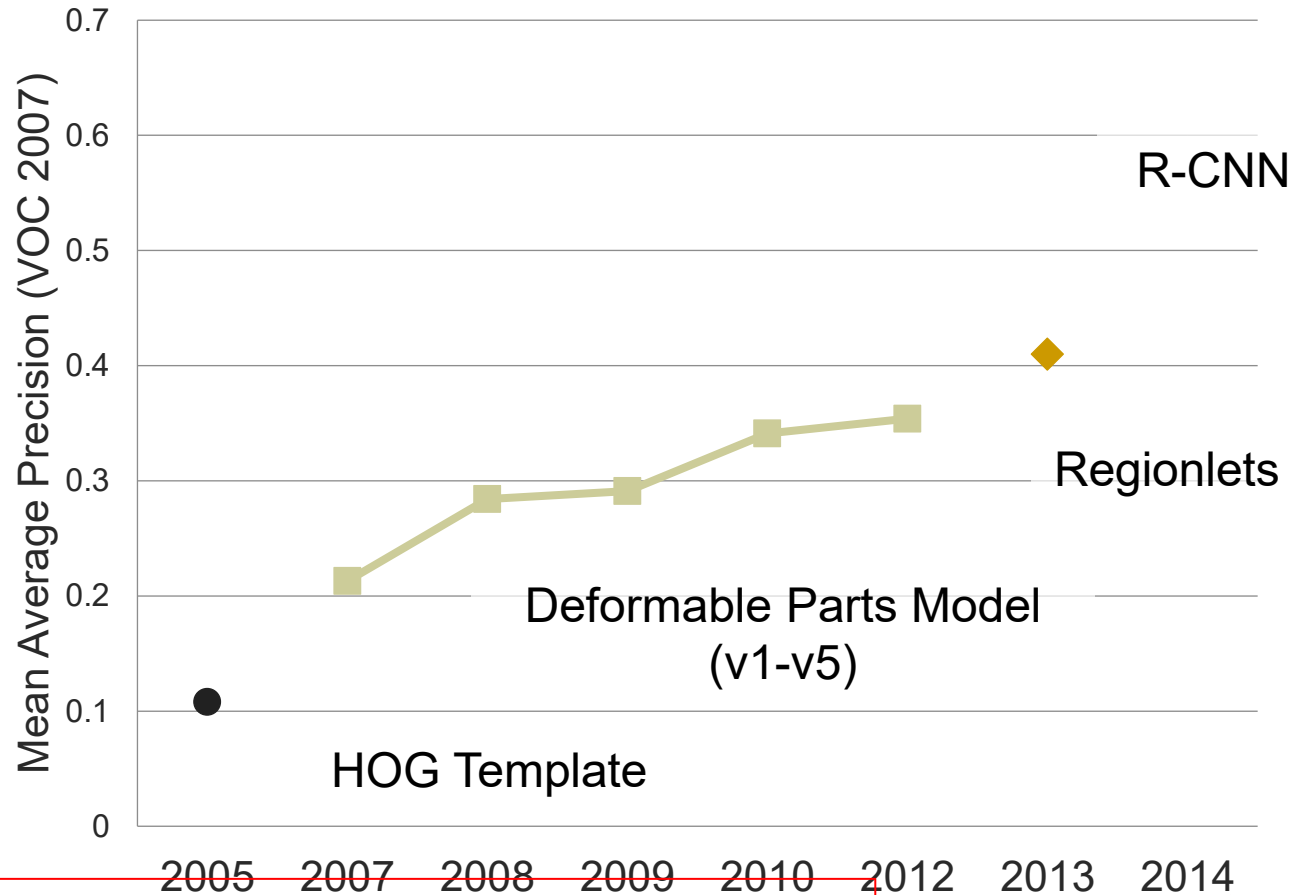
2012 ImageNet 1K

(Fall 2012)





Improvements in Object Detection

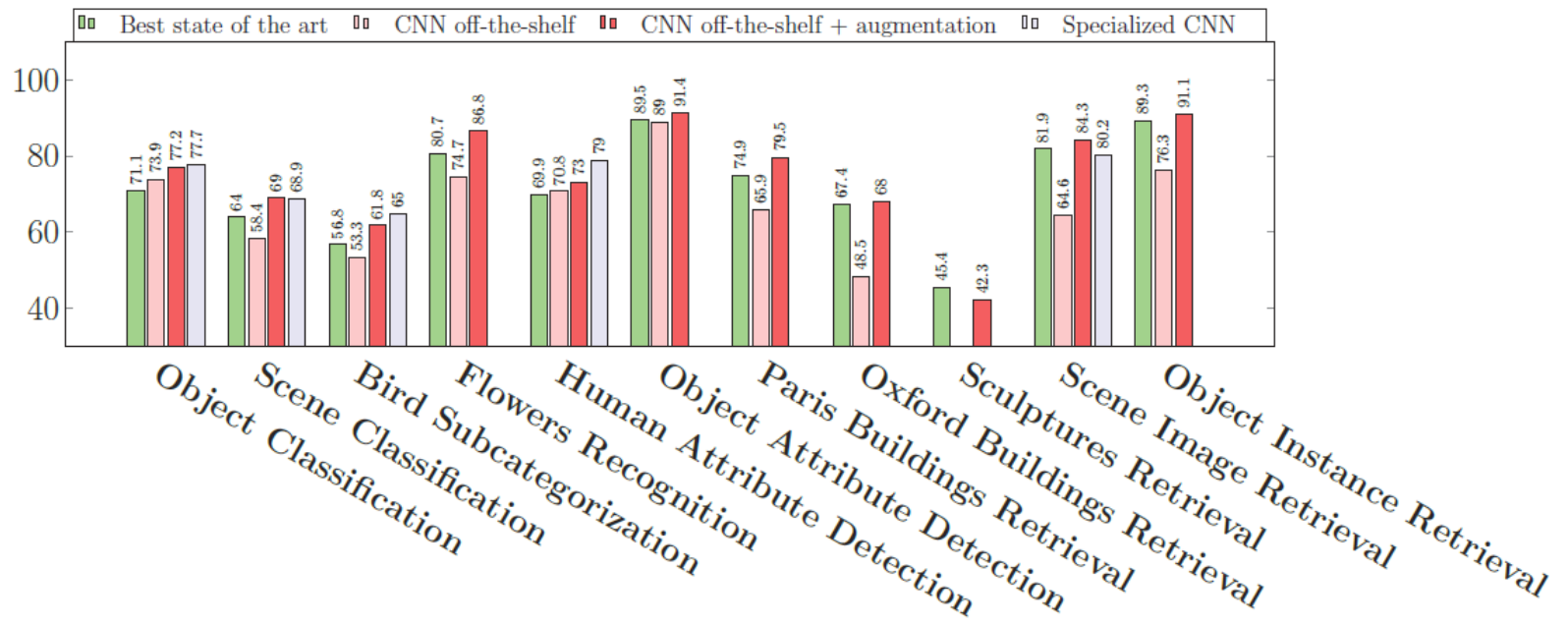
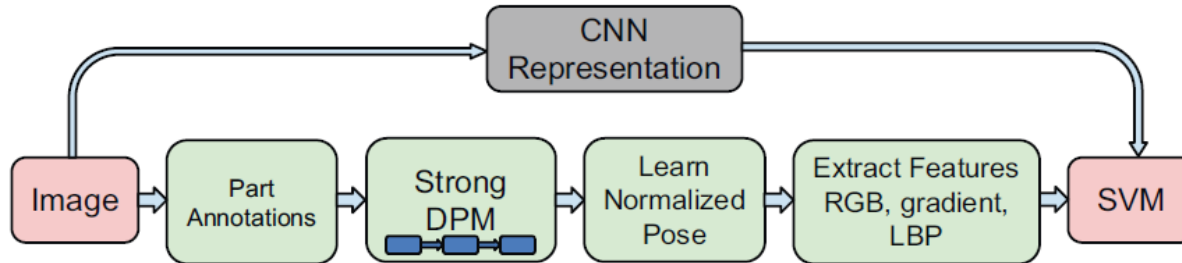


Key Advance: Learn effective features from massive amounts of labeled data *and* adapt to new tasks with less data

➔ Better Features



“CNN Features off-the-shelf: an Astounding Baseline for Recognition”





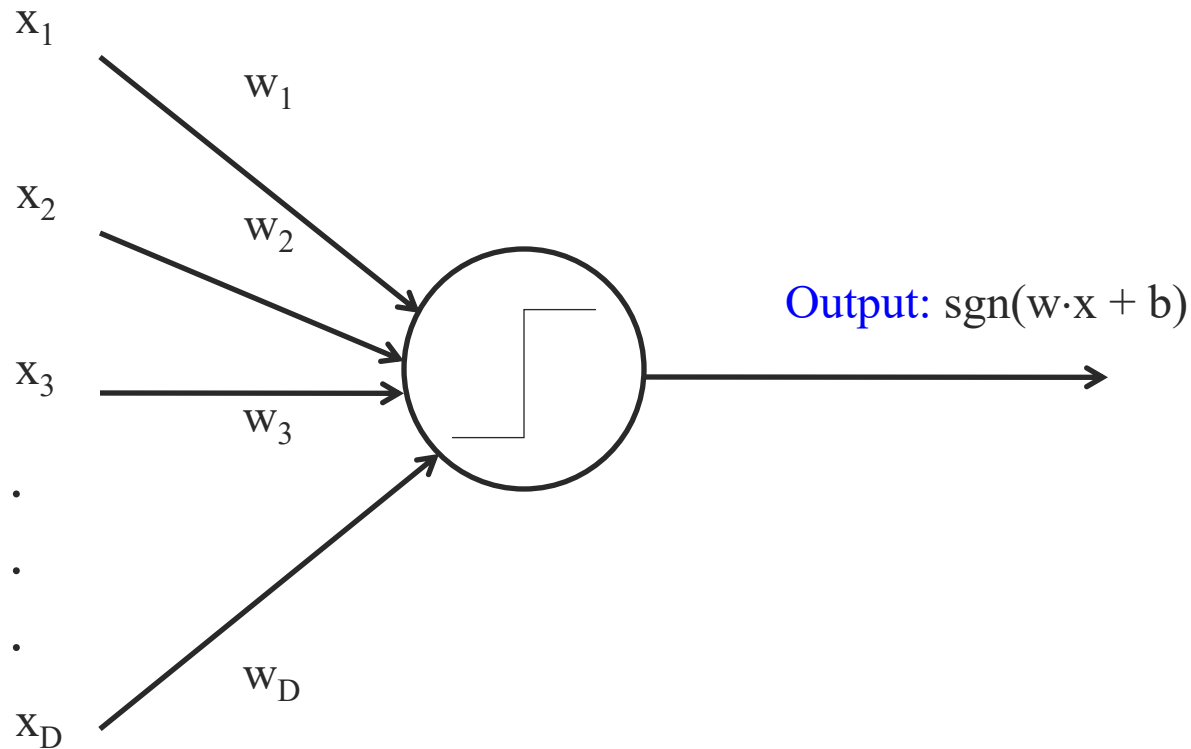
Rewind...



Input

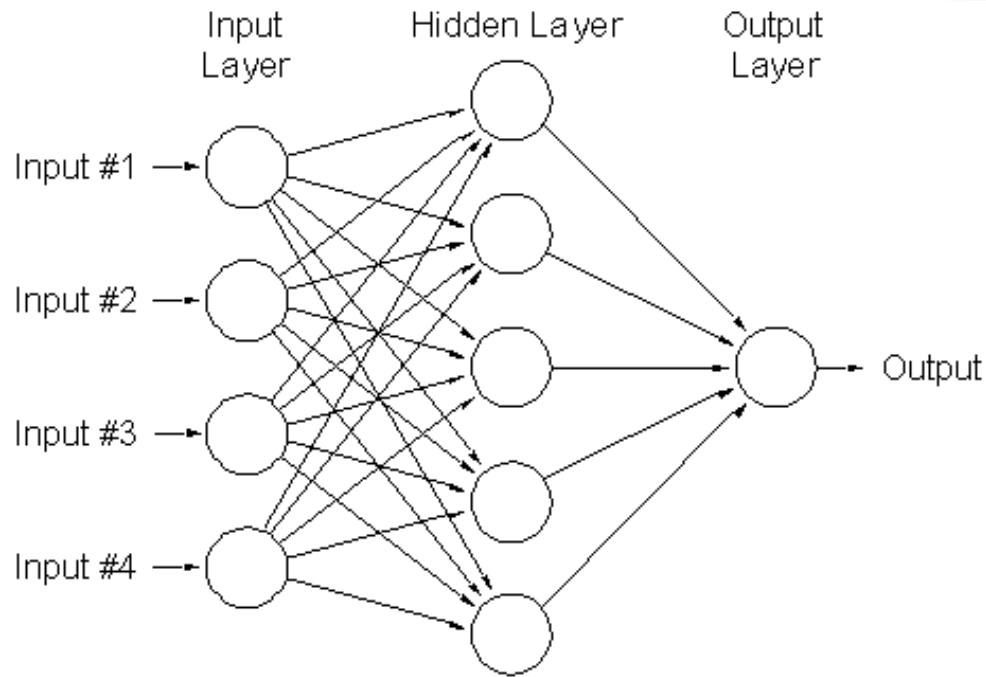
The Perceptron

Weights

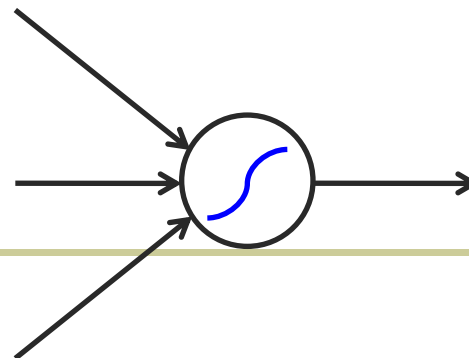




Two-layer neural network



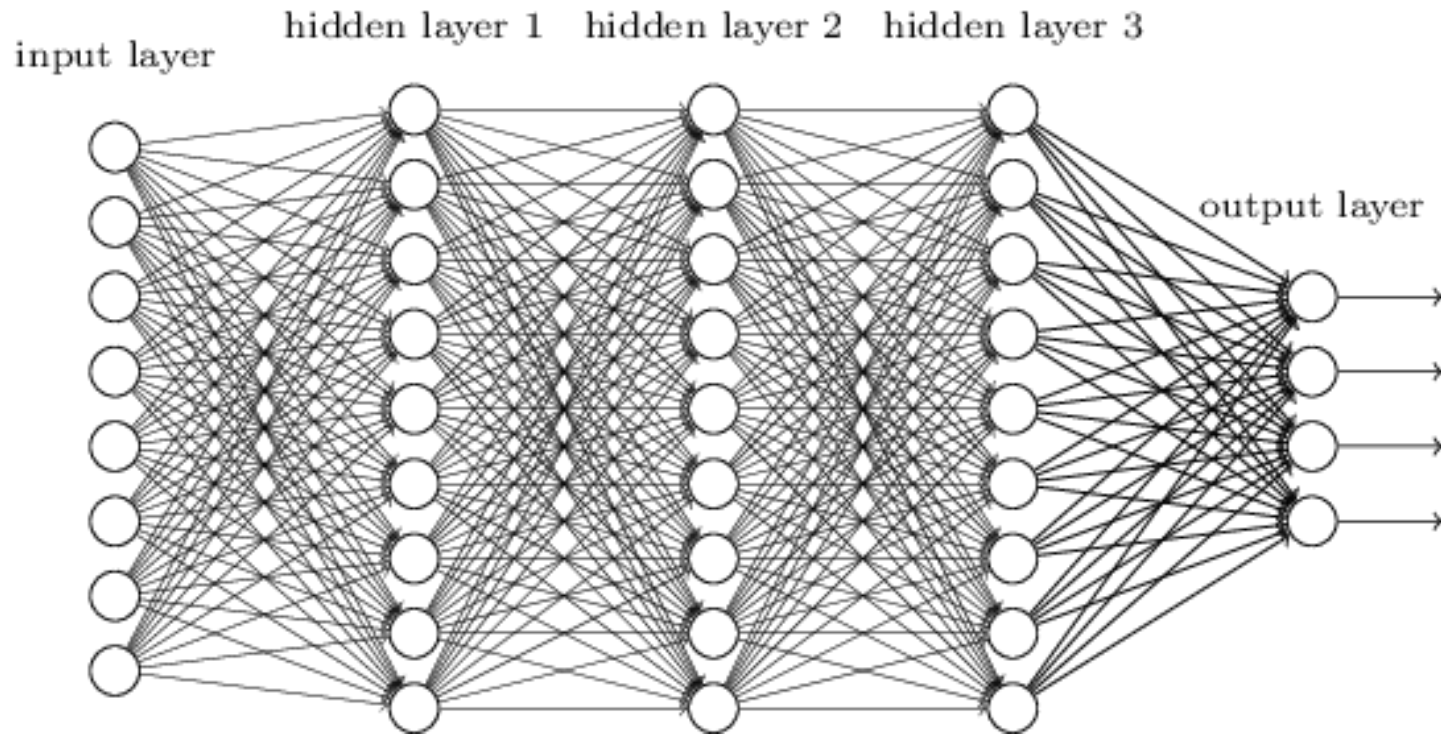
- Can learn nonlinear functions provided each perceptron has a differentiable nonlinearity



Sigmoid: $g(t) = \frac{1}{1 + e^{-t}}$



Multi-layer neural network





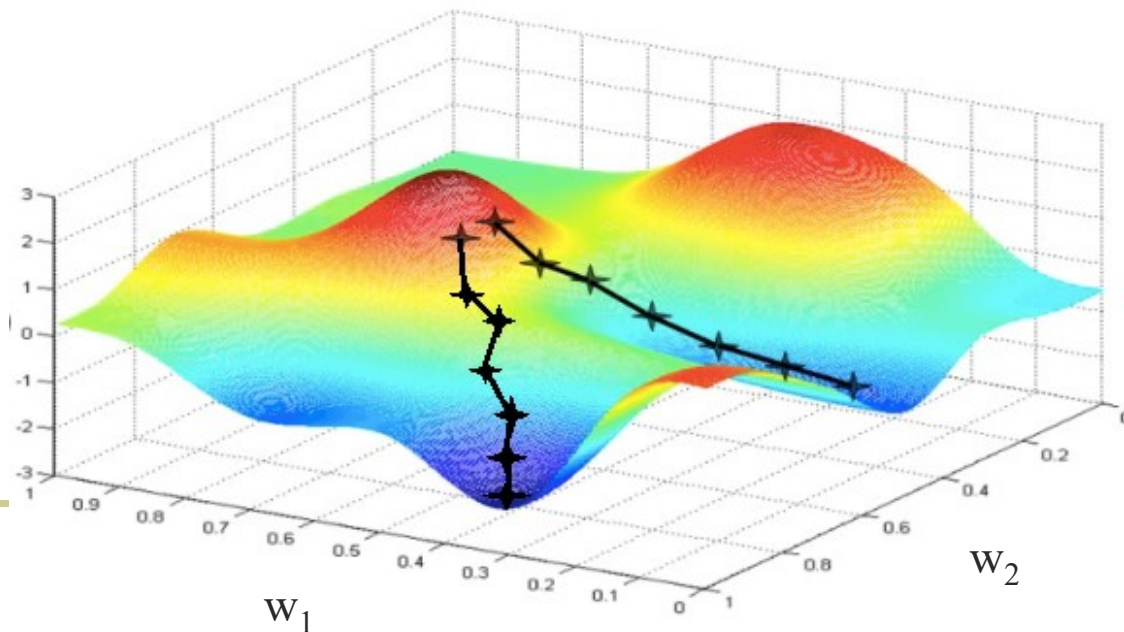
Training of multi-layer networks



- Find network weights to minimize the *training error* between true and estimated labels of training examples, e.g.:

$$E(\mathbf{w}) = \sum_{i=1}^N (y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2$$

- Update weights by **gradient descent**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$





Training of multi-layer networks



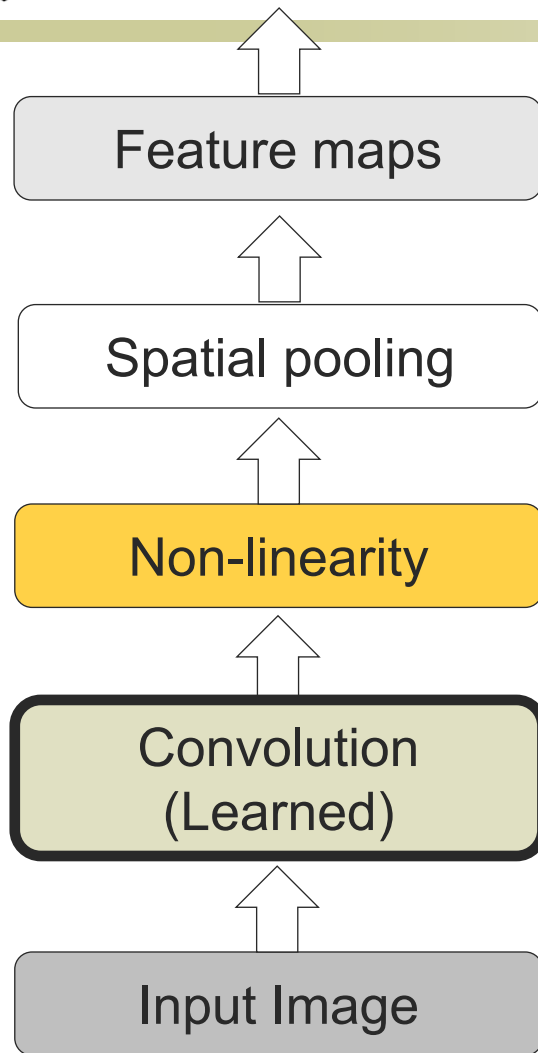
- Find network weights to minimize the *training error* between true and estimated labels of training examples, e.g.:

$$E(\mathbf{w}) = \sum_{i=1}^N (y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2$$

- Update weights by **gradient descent**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$
- Back-propagation**: gradients are computed in the direction from output to input layers and combined using chain rule
- Stochastic gradient descent**: compute the weight update w.r.t. a small batch of examples at a time, cycle through training examples in random order in multiple epochs



Key operations in a CNN



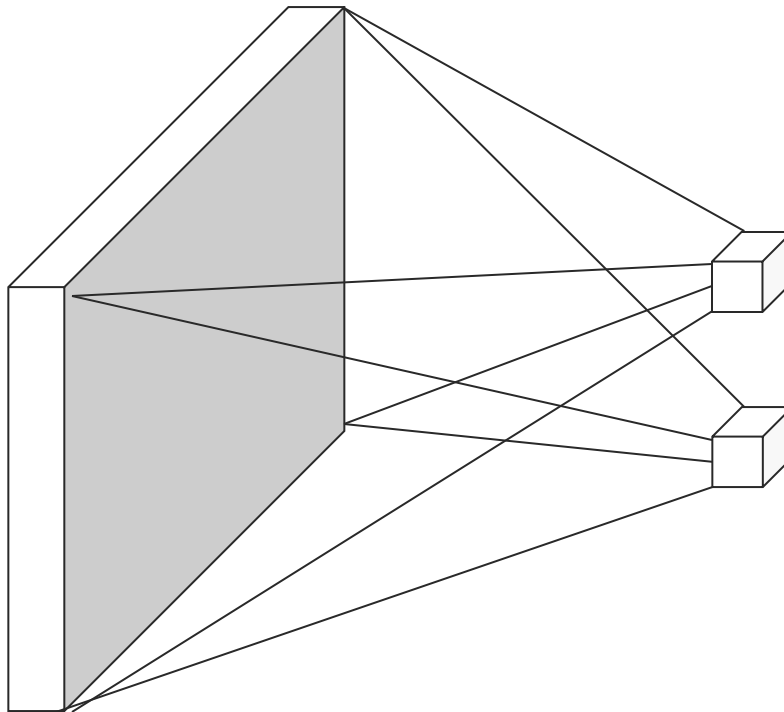
Input



Feature Map

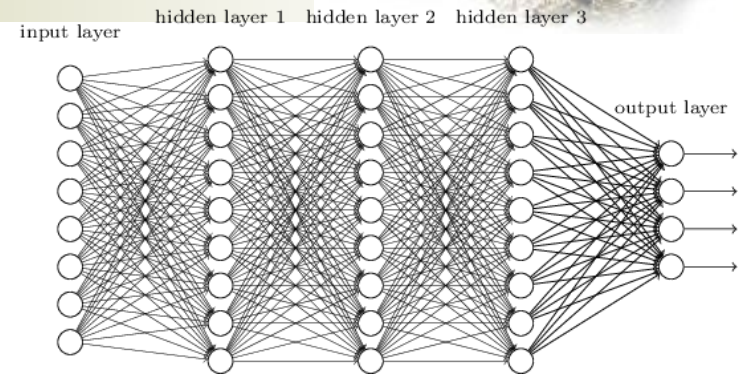


From fully connected to convolutional networks



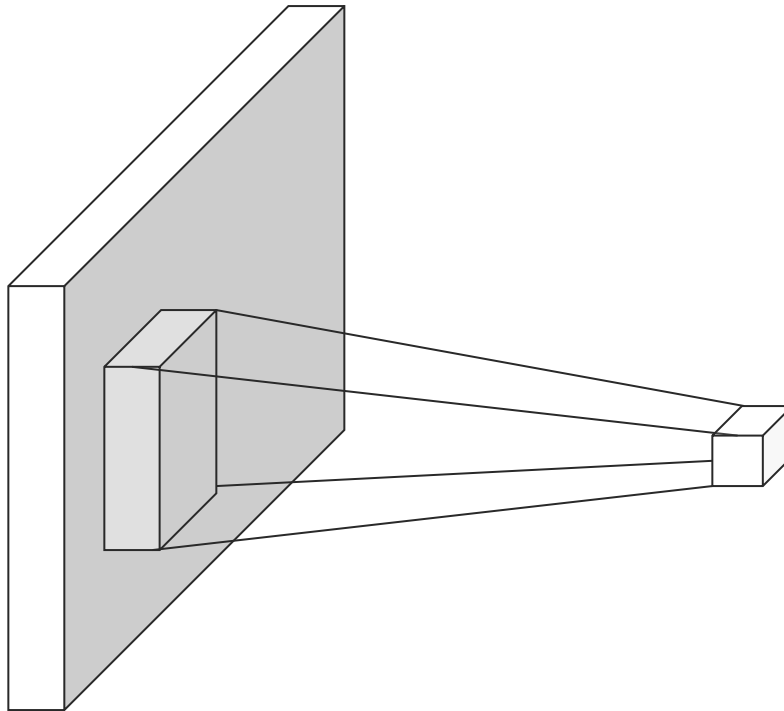
image

Fully connected layer





From fully connected to convolutional networks

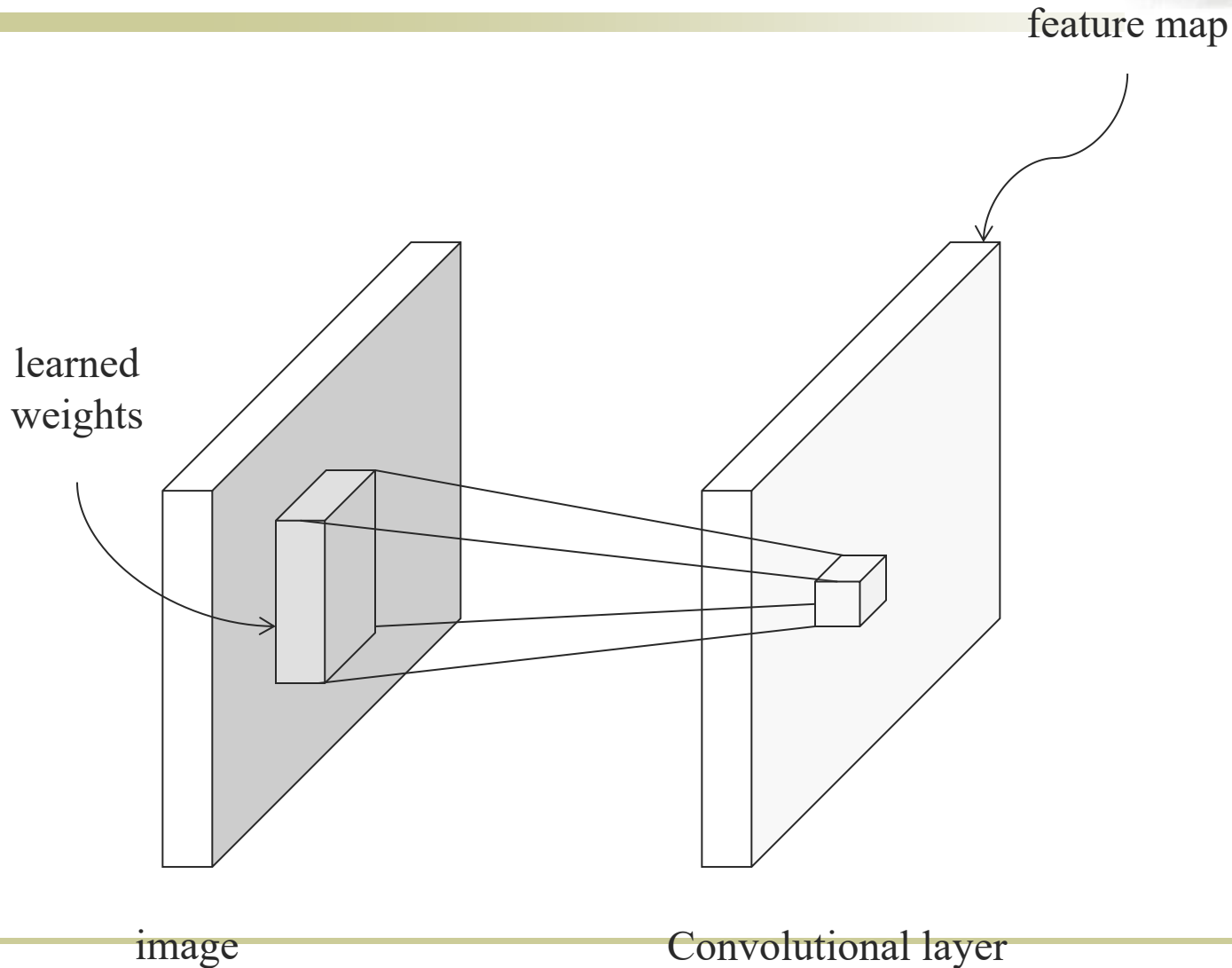


image

Convolutional layer

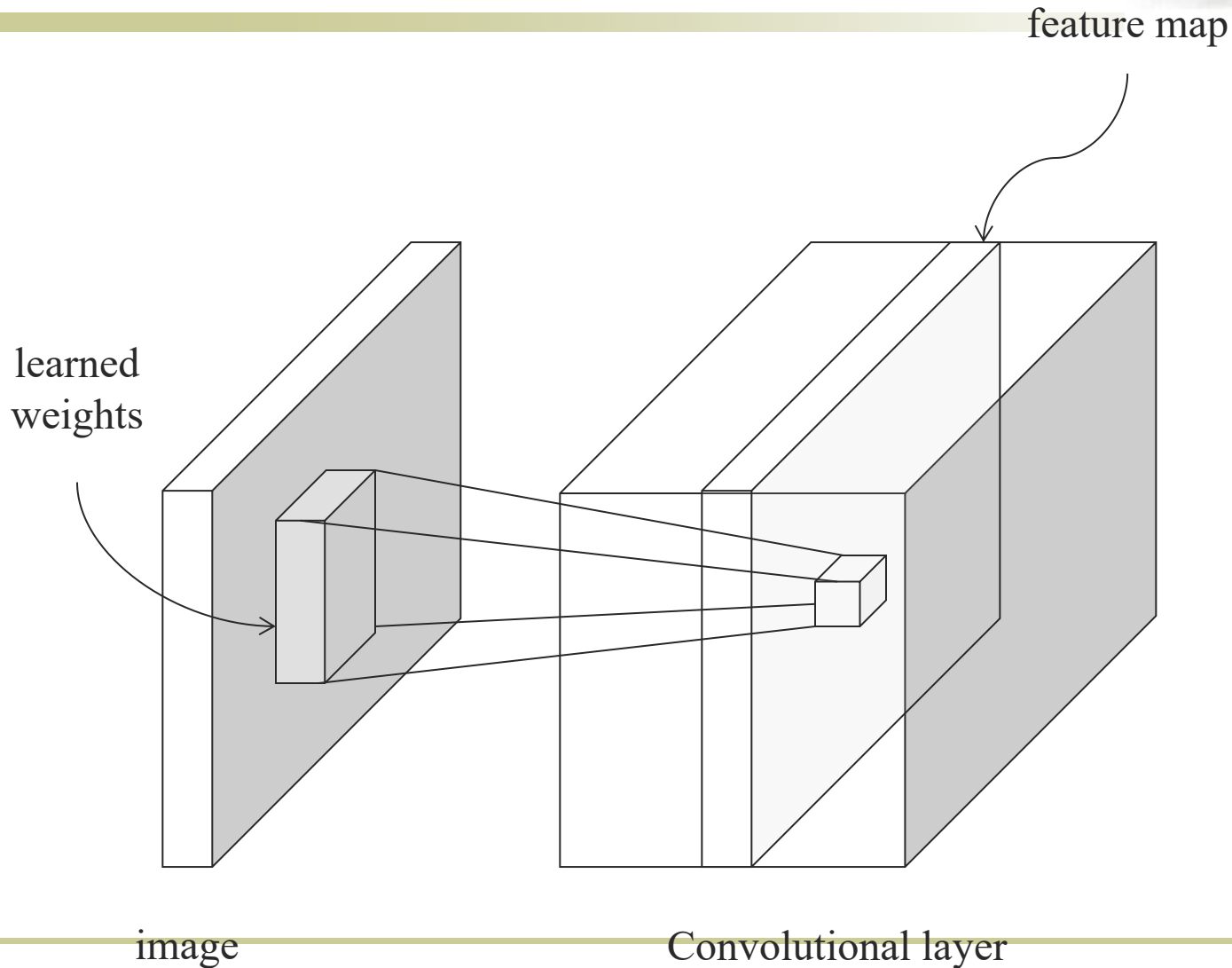


From fully connected to convolutional networks





From fully connected to convolutional networks





Convolution as feature extraction



...



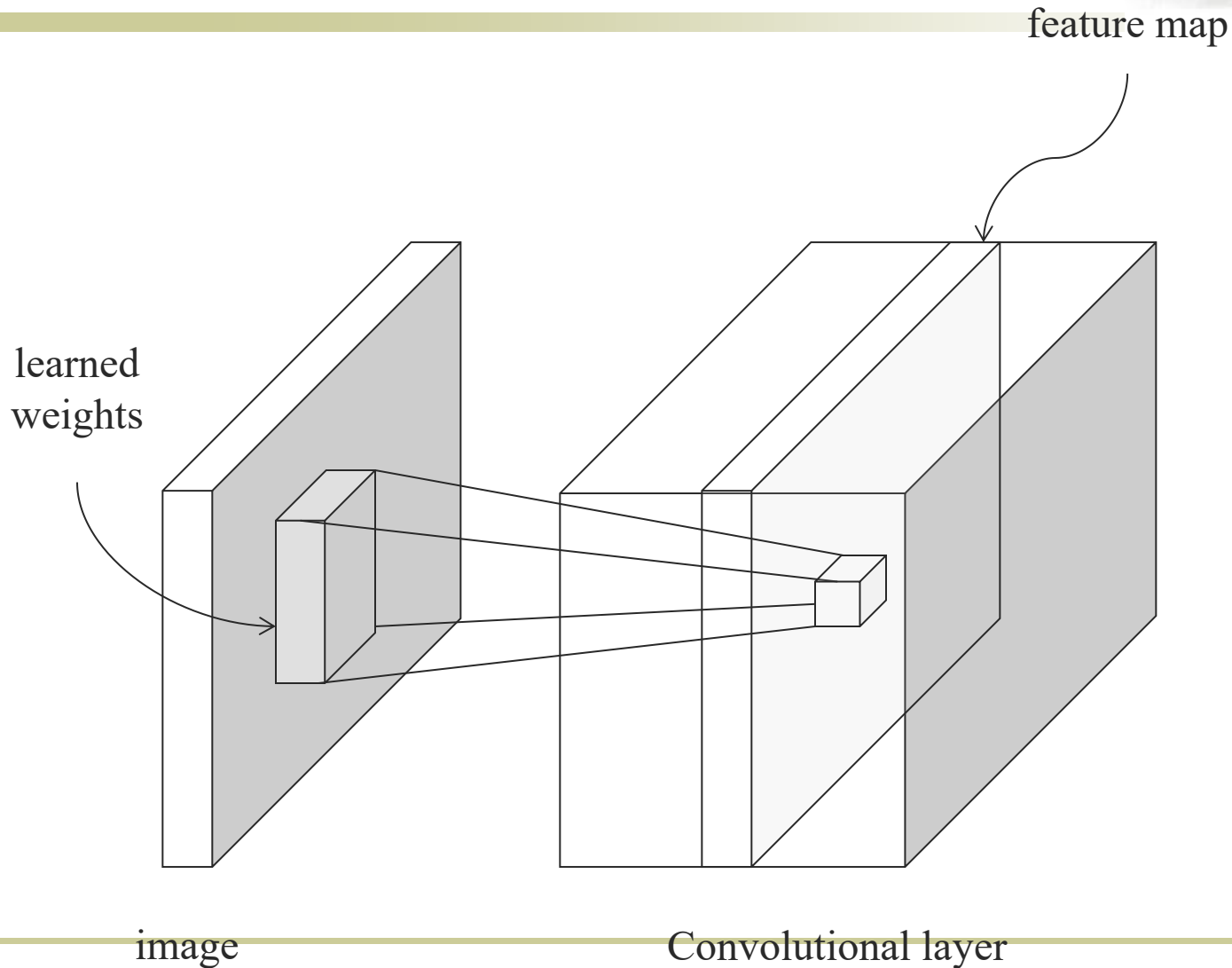
Input

Feature Map

78

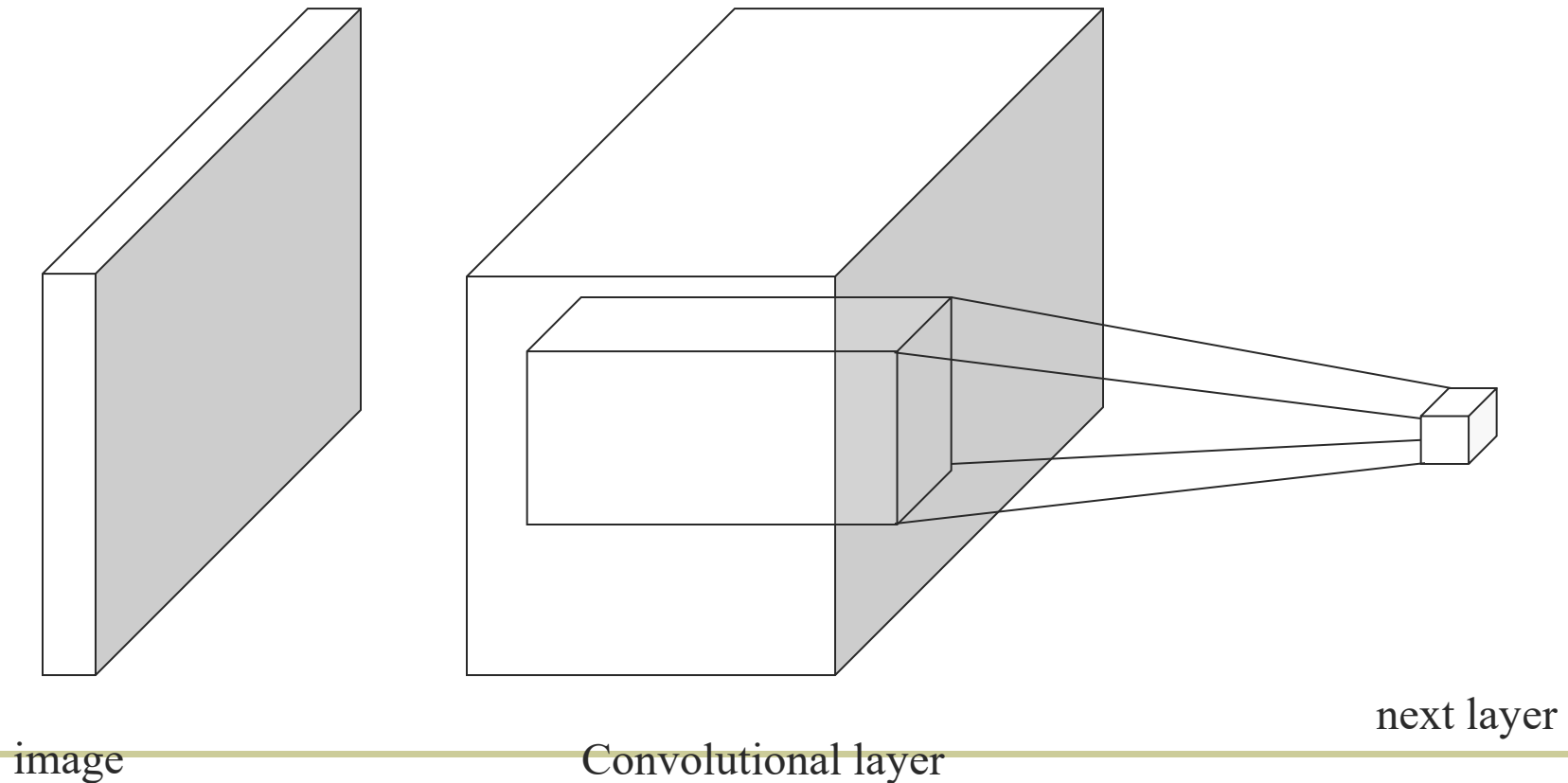


From fully connected to convolutional networks



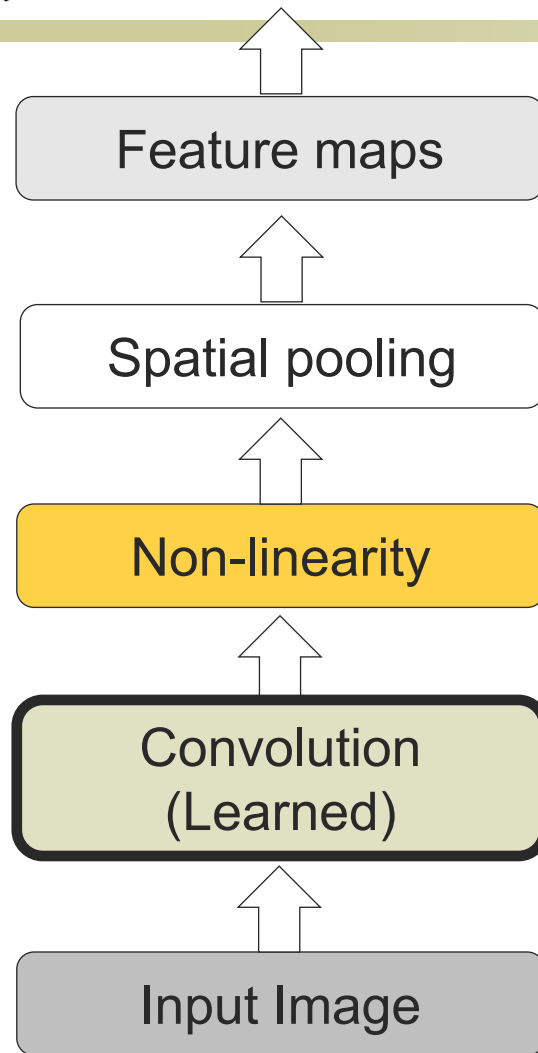


From fully connected to convolutional networks





Key operations in a CNN



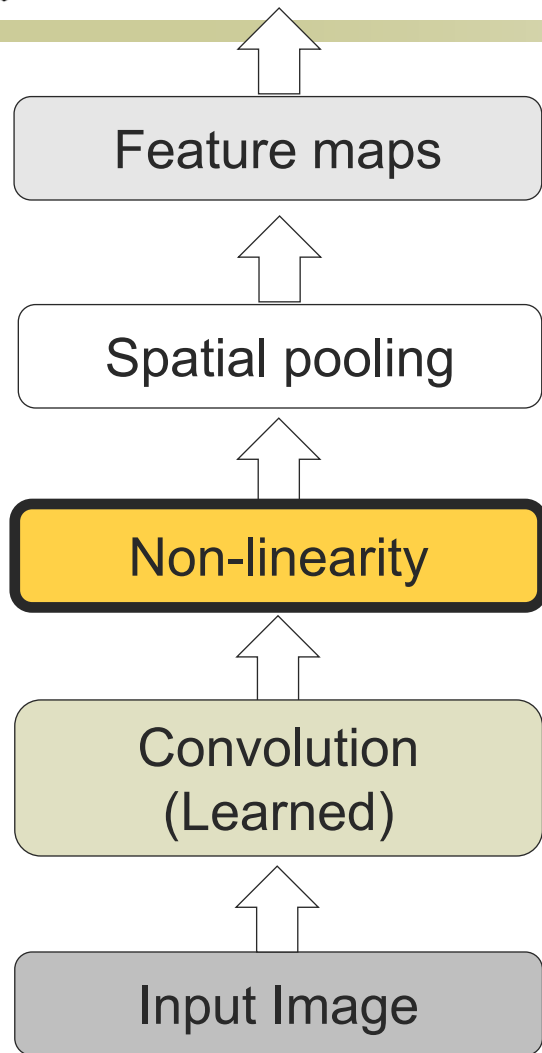
Input



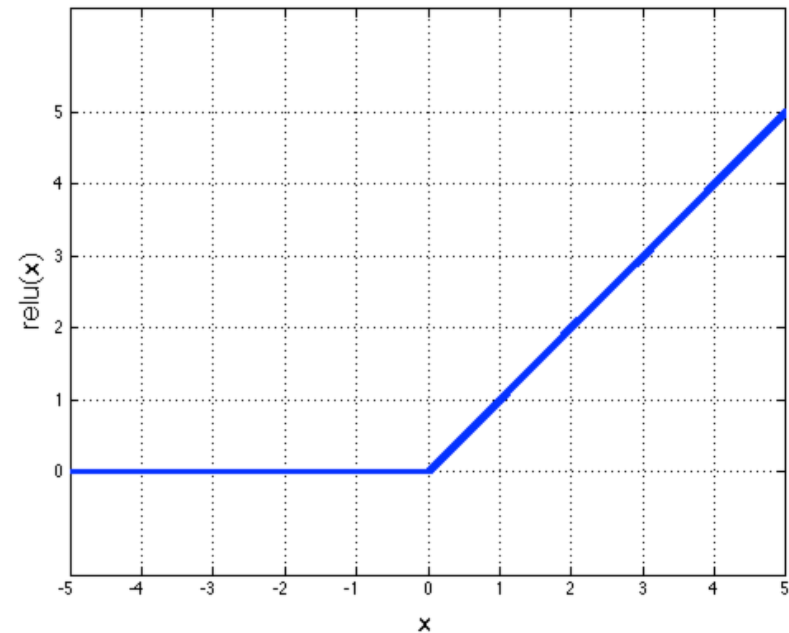
Feature Map



Key operations

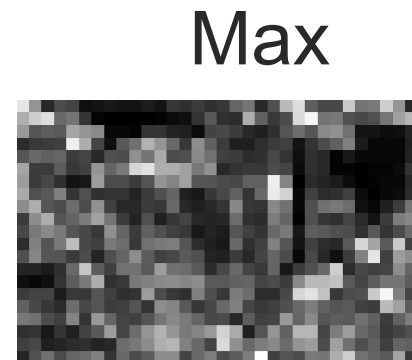
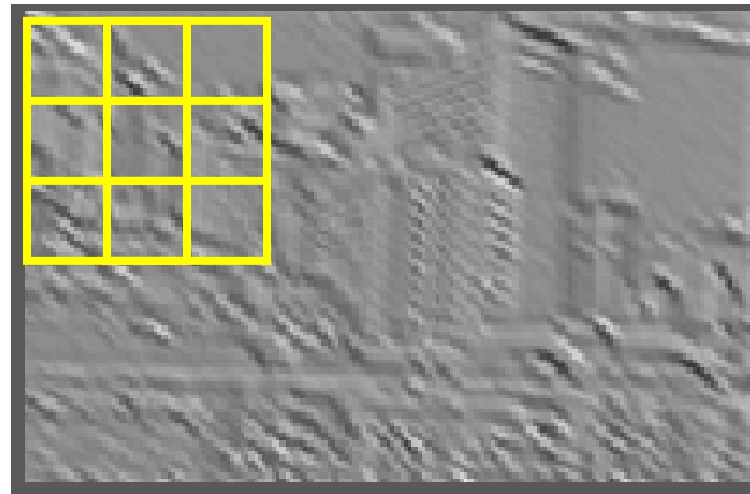
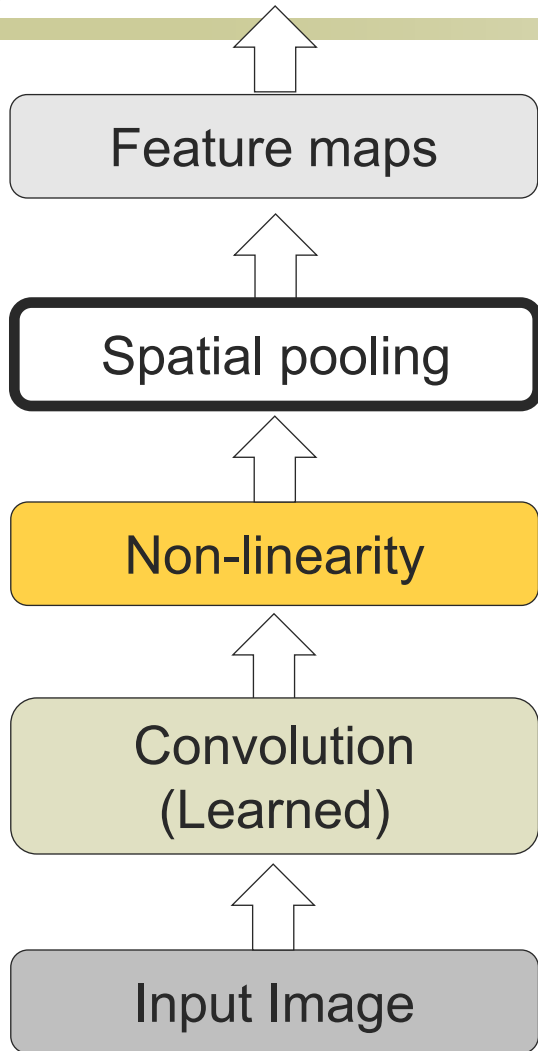


Rectified Linear Unit (ReLU)



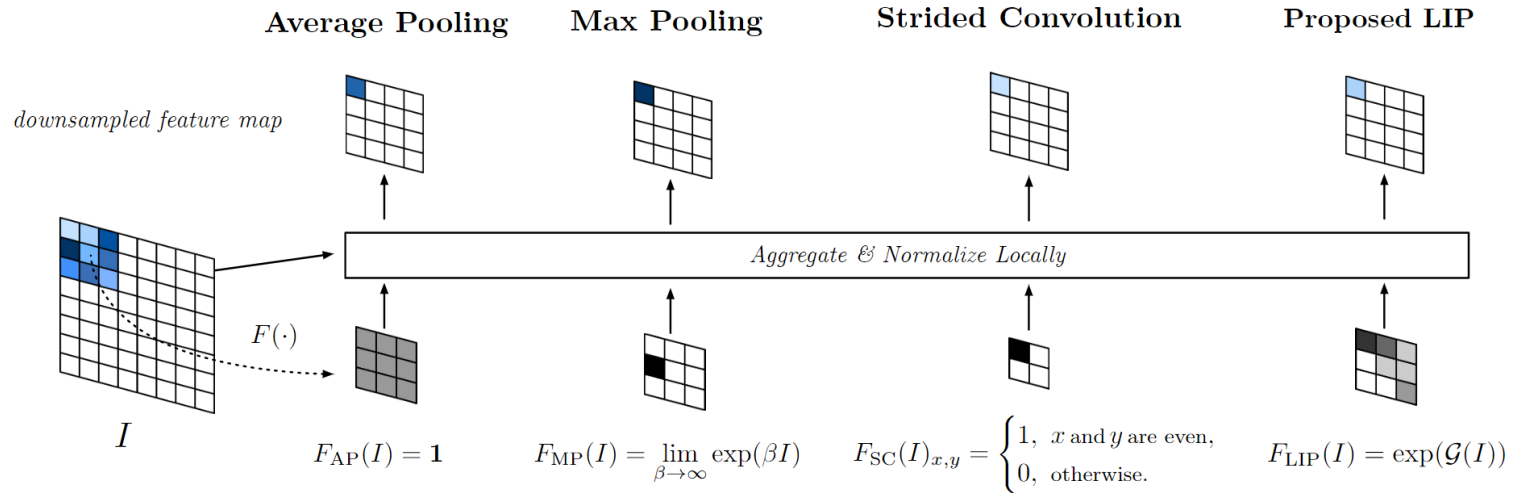


Key operations





Local Importance Pooling (ICCV19 & IJCV23)



$$O_{x',y'} = \frac{\sum_{(\Delta x, \Delta y) \in \Omega} F(I)_{x+\Delta x, y+\Delta y} \mathcal{K}(I)_{x+\Delta x, y+\Delta y}}{\sum_{(\Delta x, \Delta y) \in \Omega} F(I)_{x+\Delta x, y+\Delta y}}$$



Comparison to Pyramids with SIFT

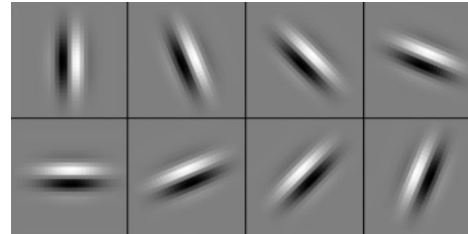


Lowe [IJCV 2004]

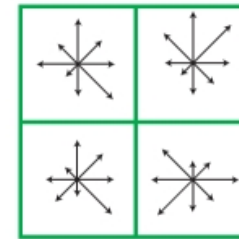
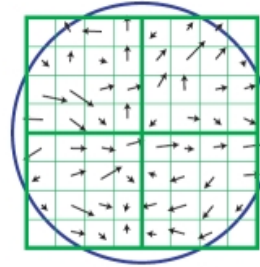
Image
Pixels



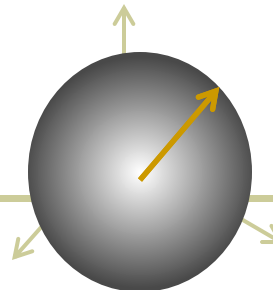
Apply
oriented filters



Spatial pool
(Sum)



Normalize to
unit length



Feature
Vector



Comparison to Pyramids with SIFT

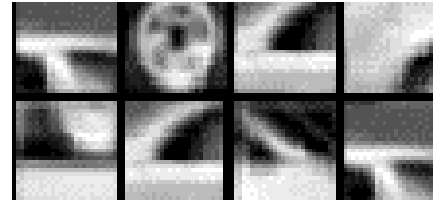


Lazebnik,
Schmid,
Ponce
[CVPR 2006]

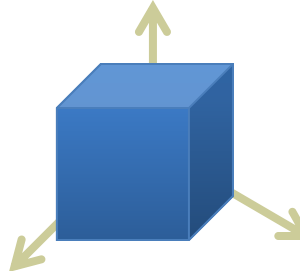
SIFT
Features



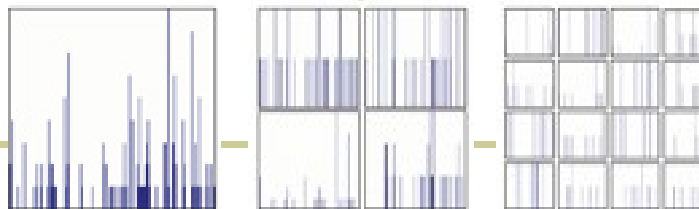
Filter with
Visual Words



Max



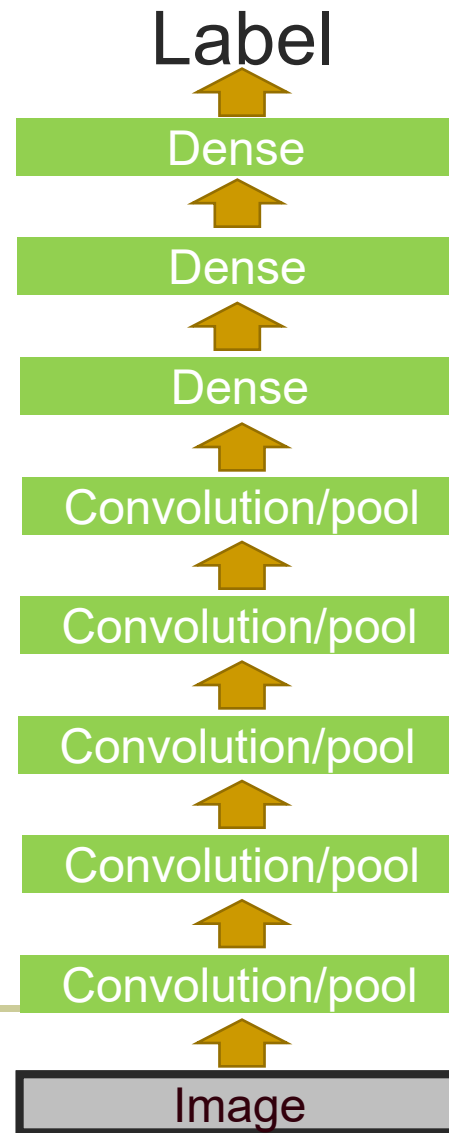
Multi-scale
spatial pool
(Sum)



Classifier

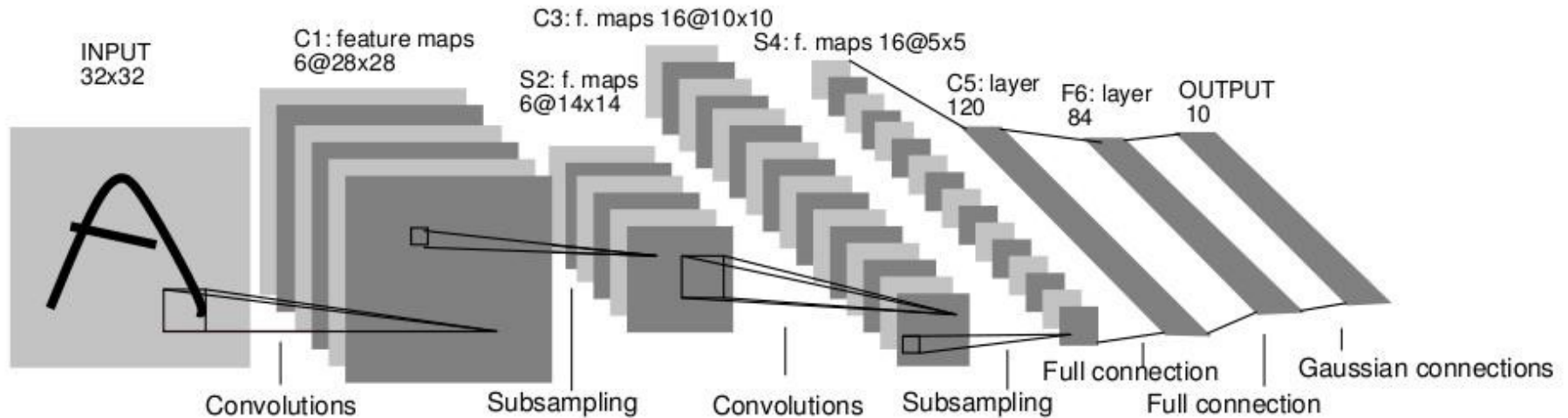


Key idea: learn features and classifier that work well together (“end-to-end training”)





LeNet-5



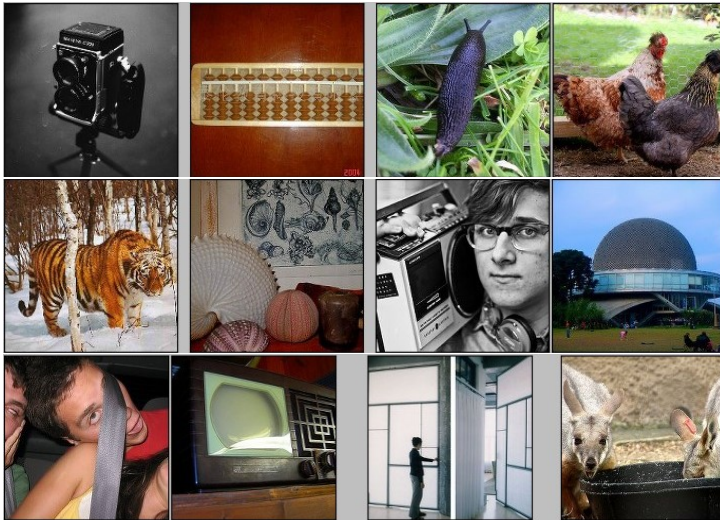
- Average pooling
- Sigmoid or tanh nonlinearity
- Fully connected layers at the end
- Trained on MNIST digit dataset with 60K training examples



Fast forward to the arrival of big visual data...



IMAGENET

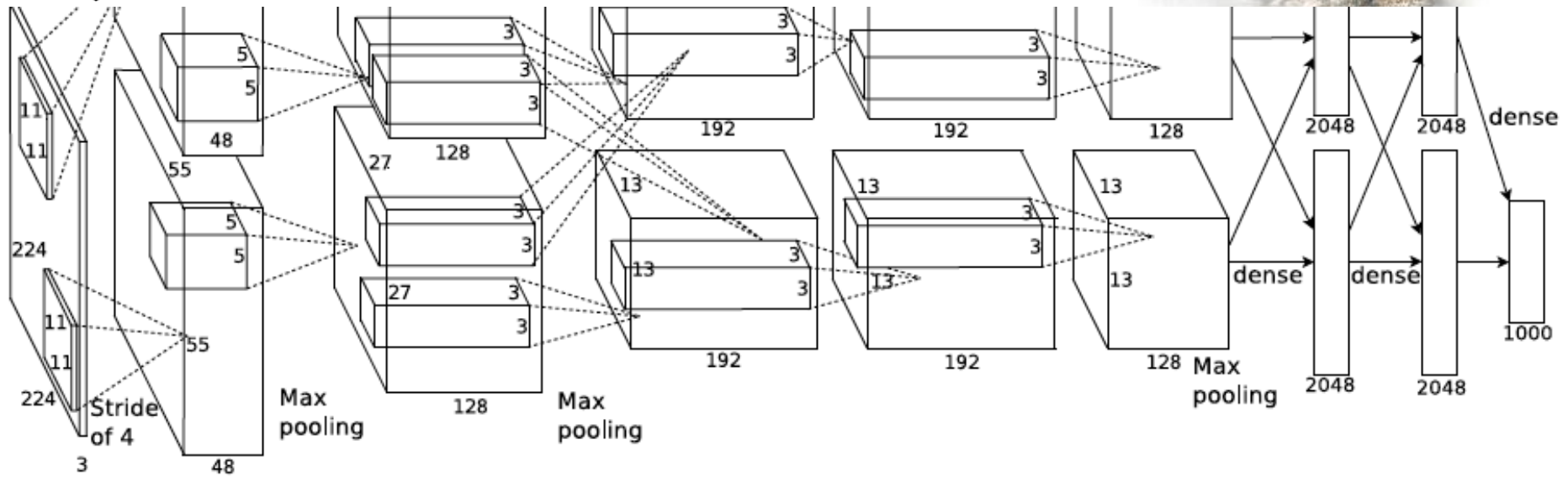


- ~14 million labeled images, 20k classes
 - Images gathered from Internet
 - Human labels via Amazon MTurk
 - ImageNet Large-Scale Visual Recognition Challenge (ILSVRC): 1.2 million training images, 1000 classes

www.image-net.org/challenges/LSVRC/



AlexNet: ILSVRC 2012 winner



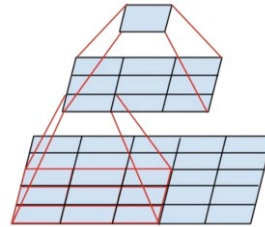
- Similar framework to LeNet but:
 - Max pooling, **ReLU nonlinearity**
- **More data and bigger model** (7 hidden layers, 650K units, 60M params)
 - GPU implementation (**50x speedup** over CPU)
 - Trained on two GPUs for a week
 - Dropout regularization



VGGNet

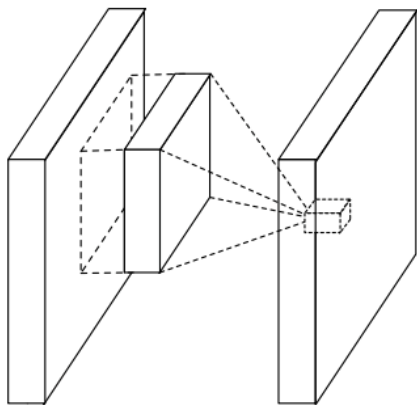


- Sequence of deeper networks trained progressively
- Large receptive fields replaced by successive layers of 3x3 convolutions (with ReLU in between)

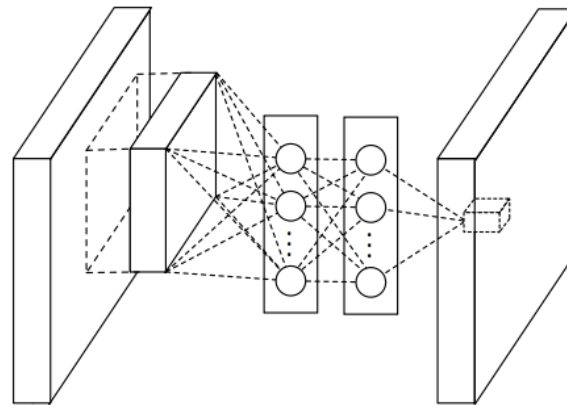


- One 7x7 conv layer with C feature maps needs $49C^2$ weights, three 3x3 conv layers need only $27C^2$ weights

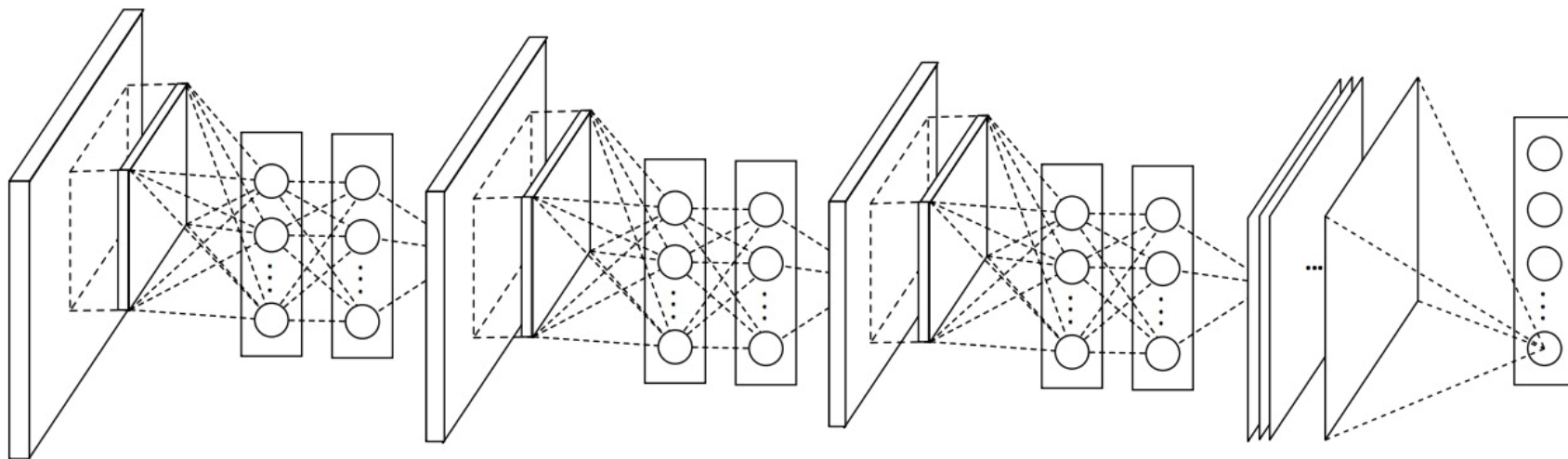
Network in network



(a) Linear convolution layer

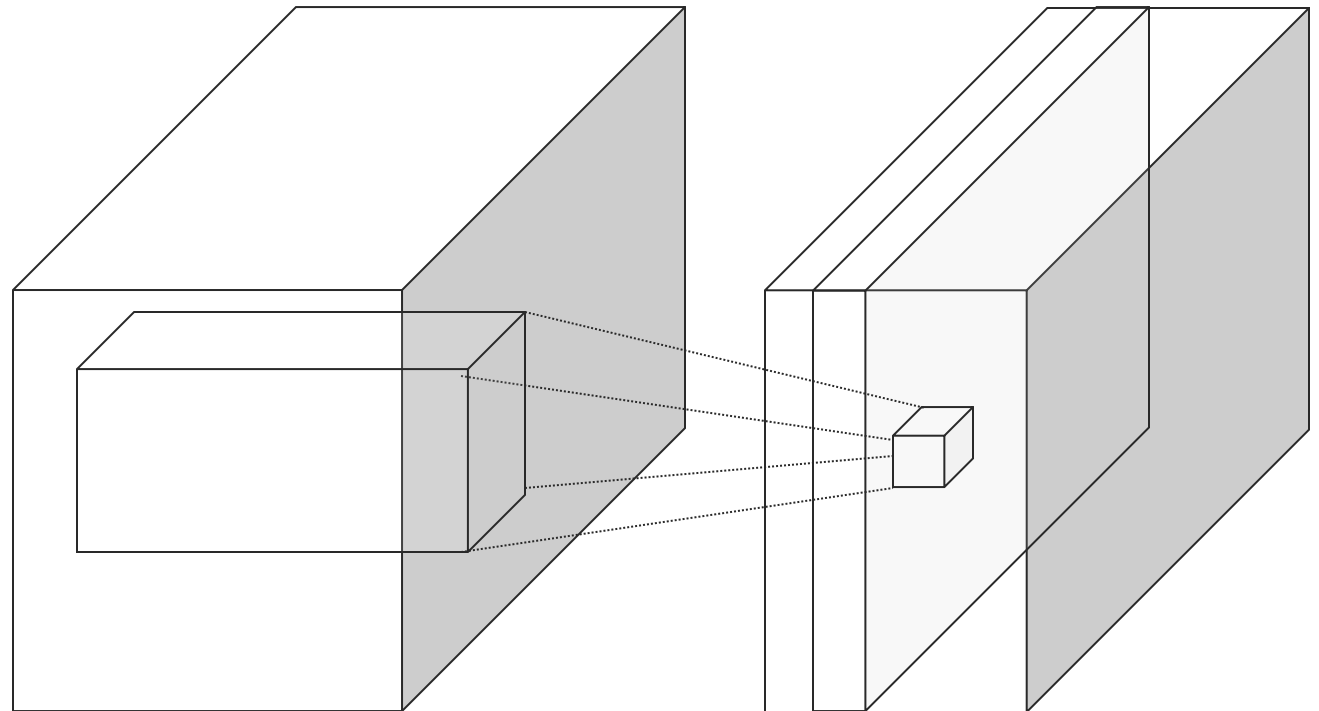


(b) Mlpconv layer





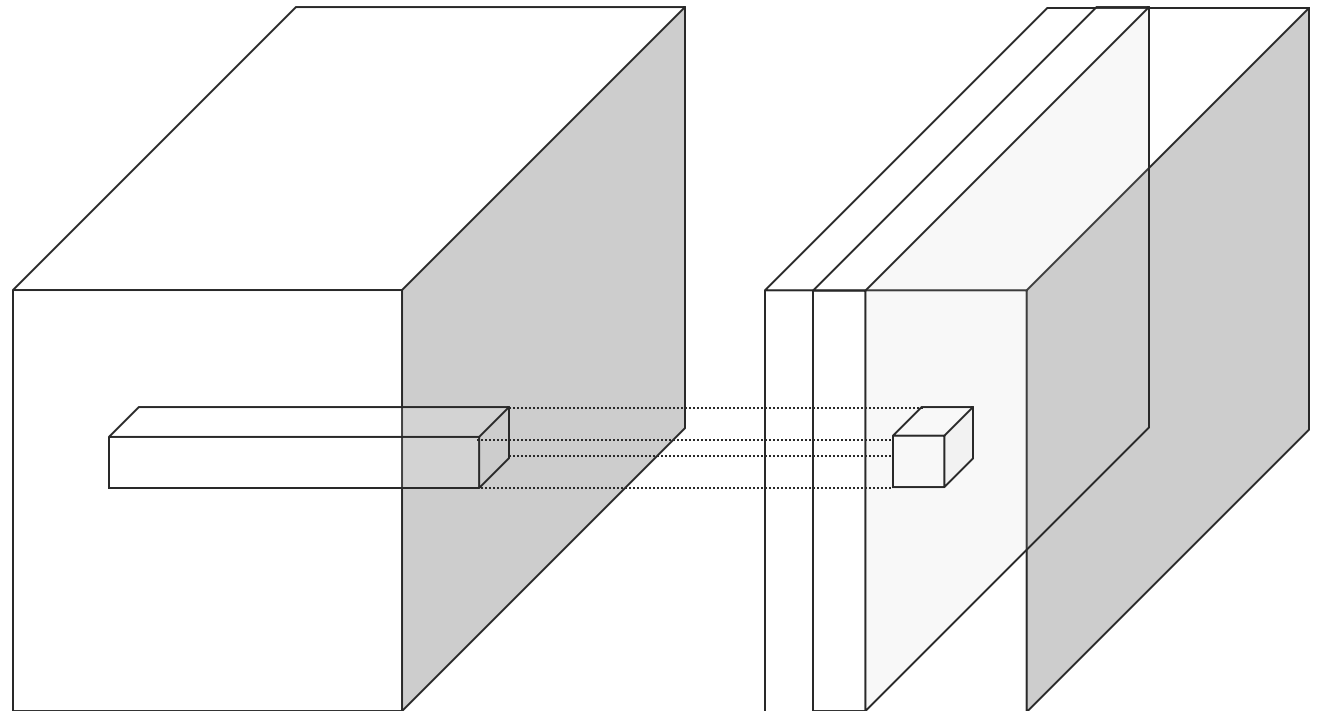
3x3 convolutions



conv layer



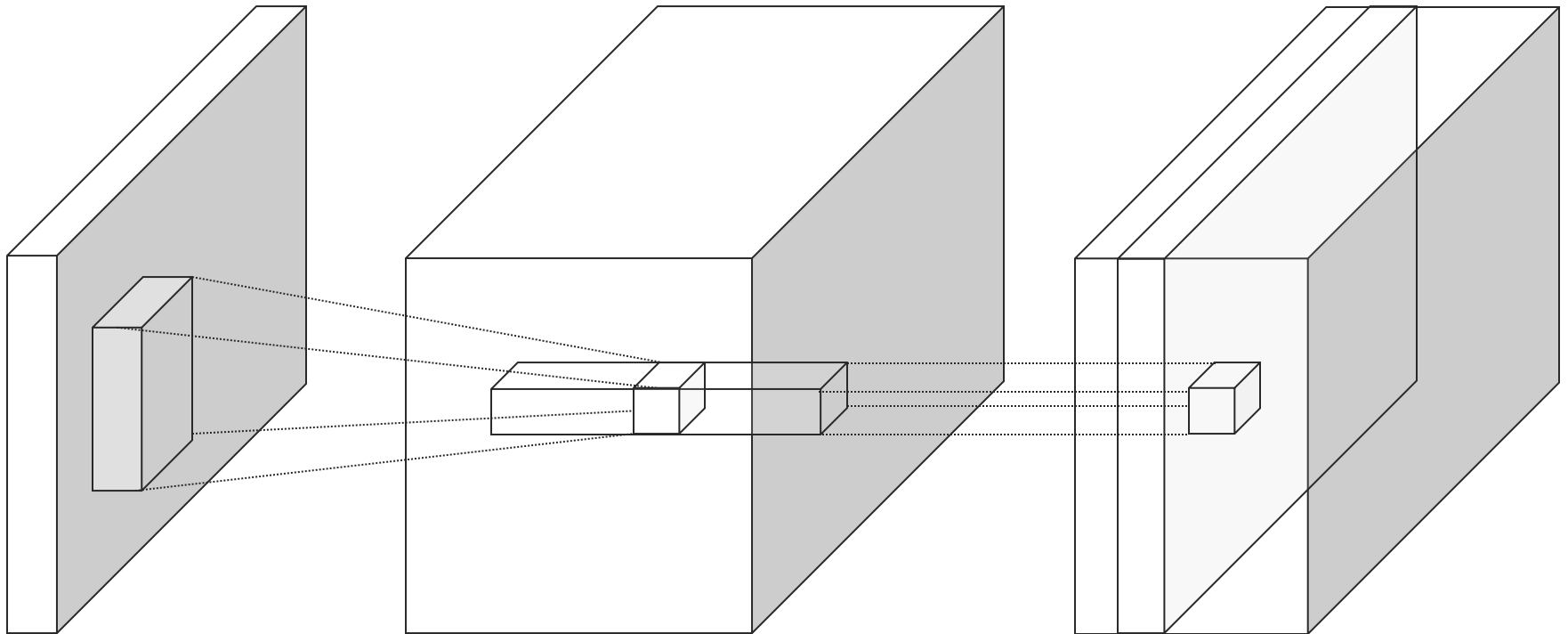
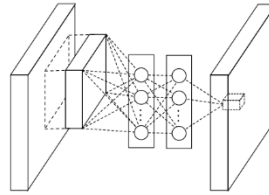
1x1 convolutions



1x1 conv layer



1x1 convolutions



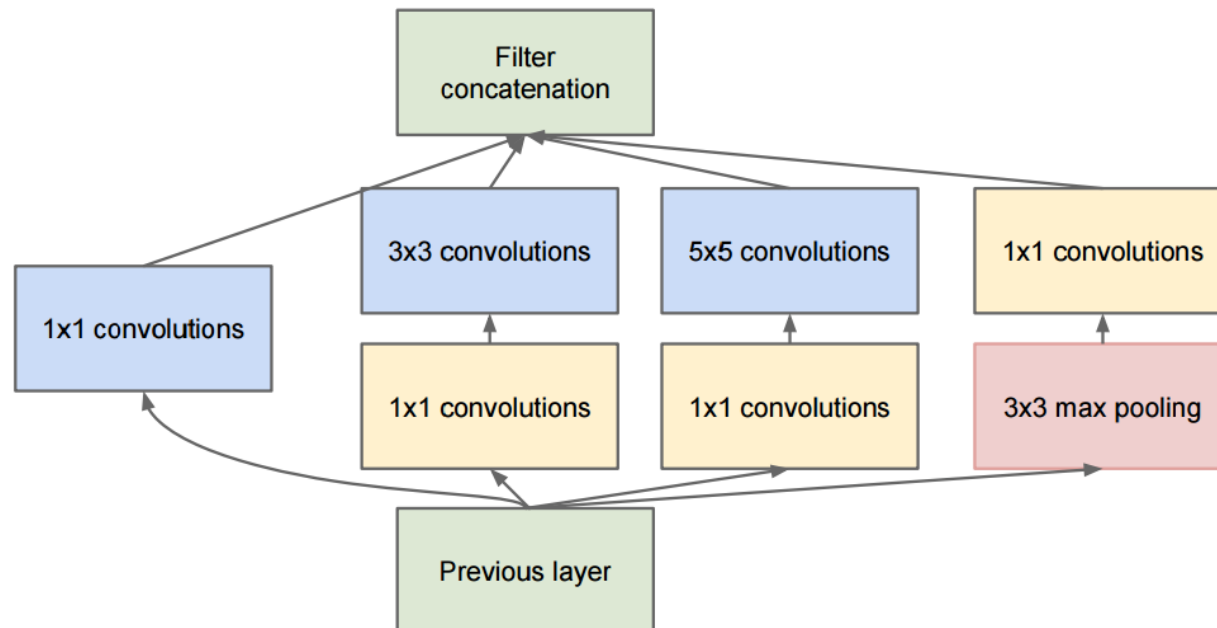
1x1 conv layer



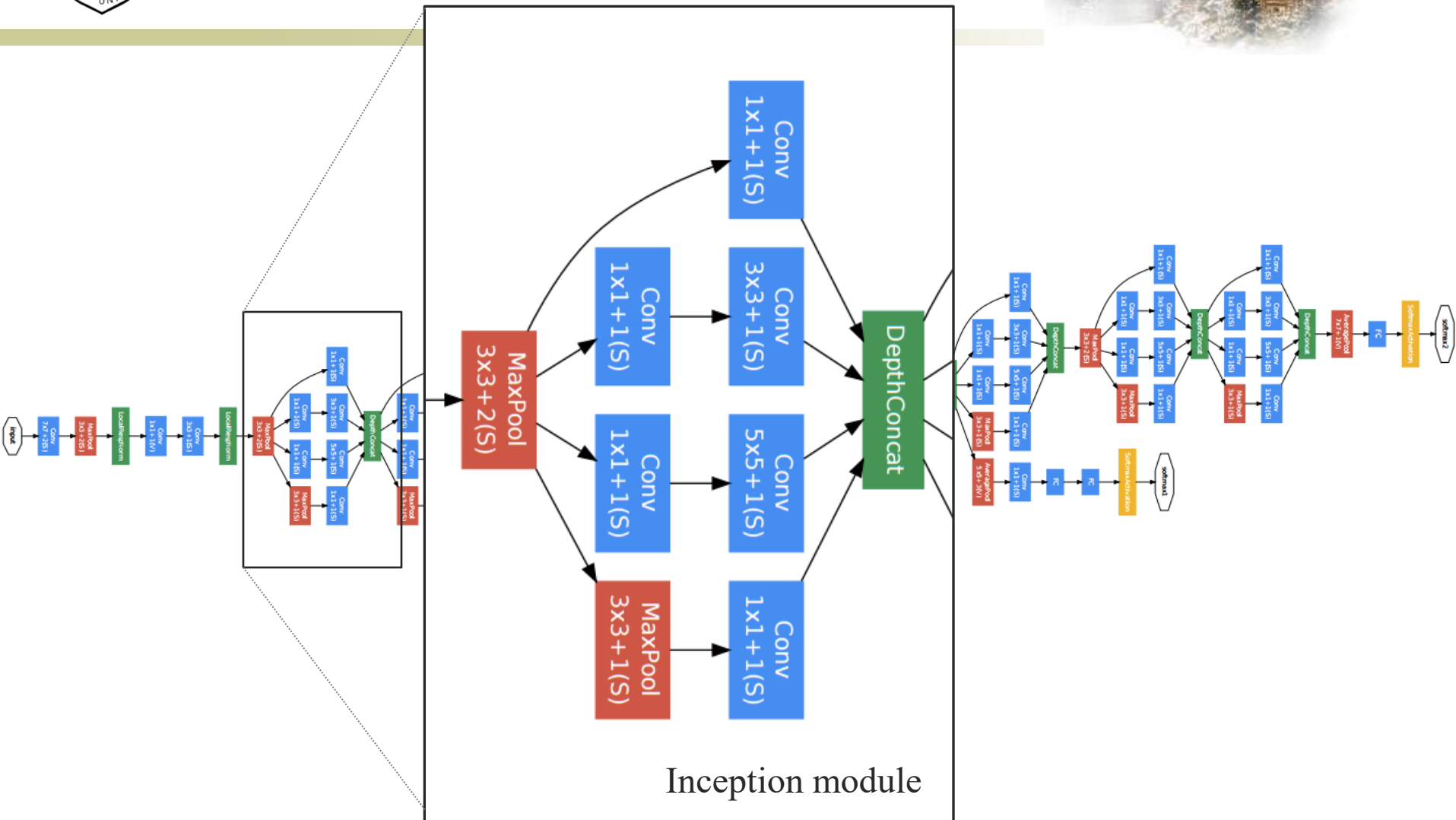
GoogLeNet: Inception module



- Parallel paths with different receptive field sizes and operations to capture sparse patterns of correlations
- 1x1 convolutions for dimensionality reduction before expensive convolutions

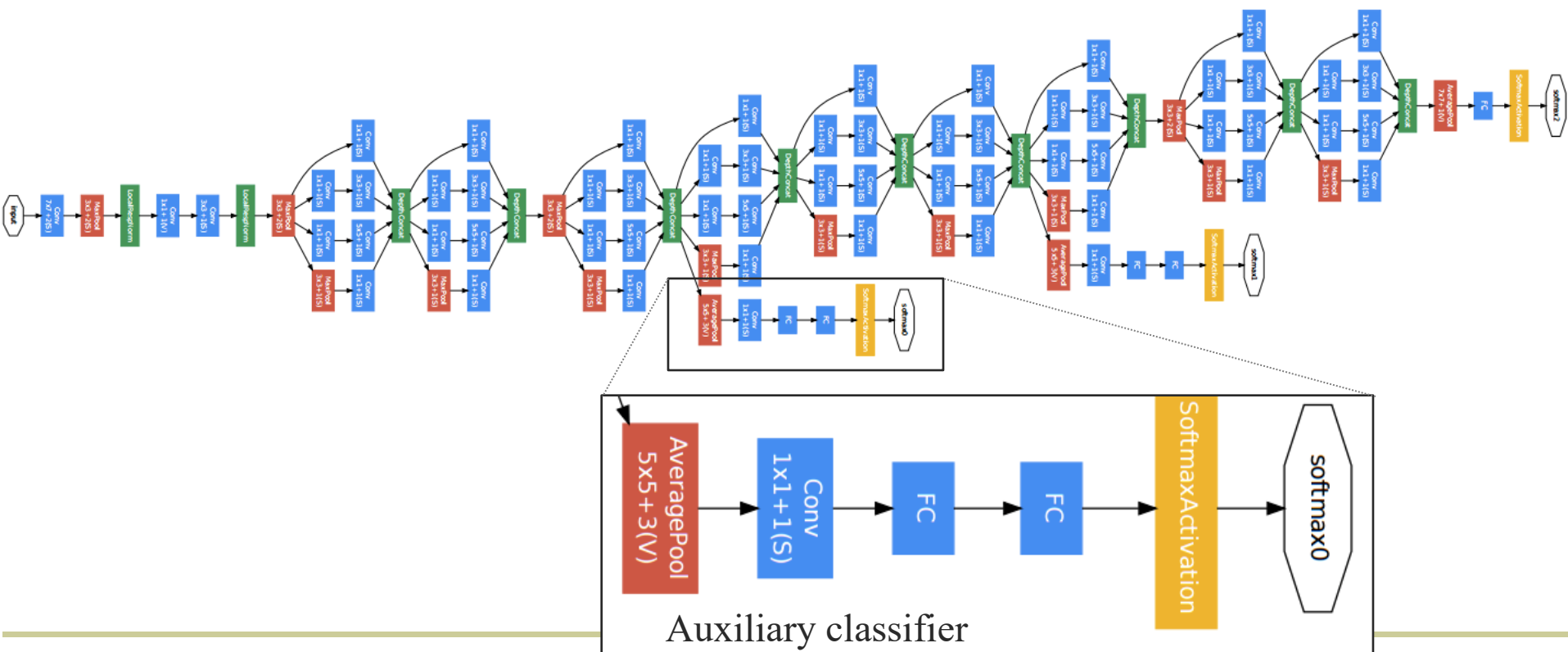


GoogLeNet





GoogLeNet

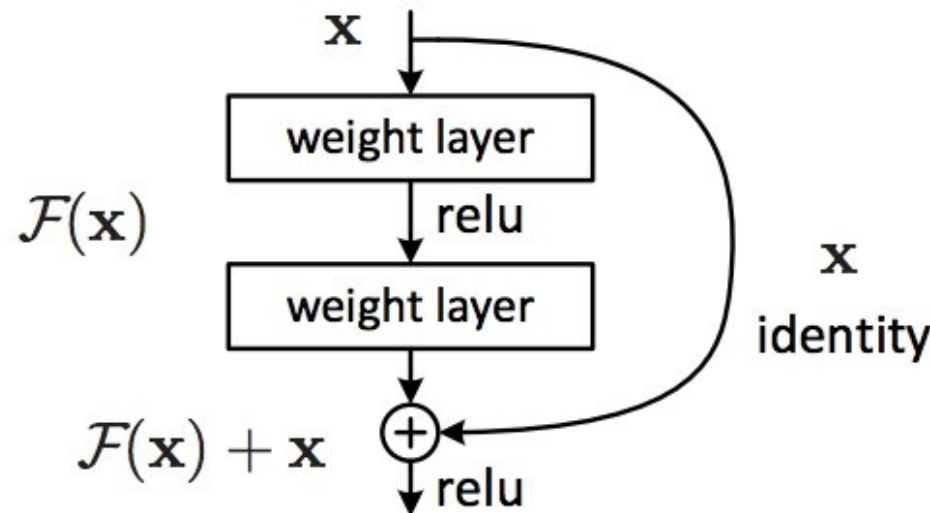




ResNet: the residual module



- Introduce *skip* or *shortcut* connections (existing before in various forms in literature)
- Make it easy for network layers to represent the identity mapping
- For some reason, need to skip at least two layers

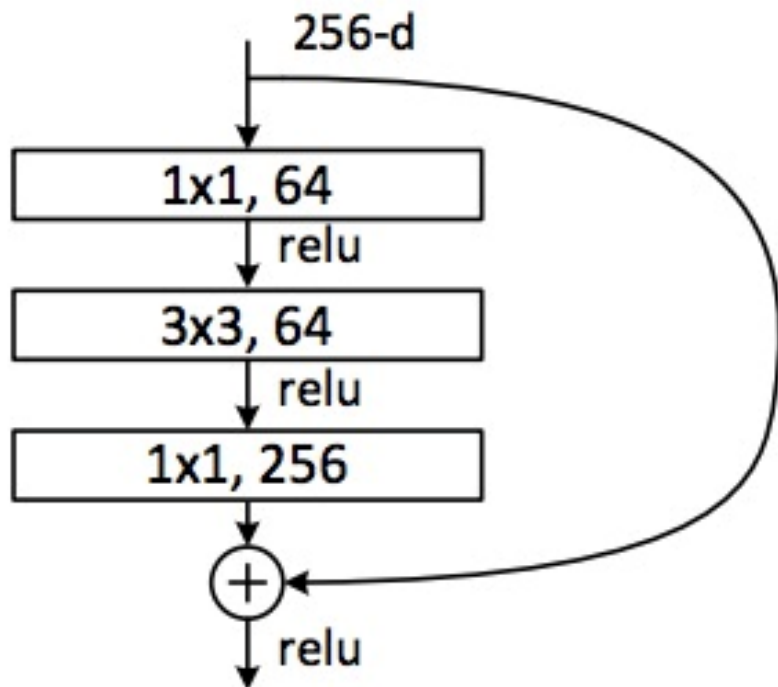




ResNet



Deeper residual module (bottleneck)



- Directly performing 3x3 convolutions with 256 feature maps at input and output:
 $256 \times 256 \times 3 \times 3 \sim 600K$ operations
- Using 1x1 convolutions to reduce 256 to 64 feature maps, followed by 3x3 convolutions, followed by 1x1 convolutions to expand back to 256 maps:
 $256 \times 64 \times 1 \times 1 \sim 16K$
 $64 \times 64 \times 3 \times 3 \sim 36K$
 $64 \times 256 \times 1 \times 1 \sim 16K$
Total: $\sim 70K$



ResNet: going real deep



Revolution of Depth

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)

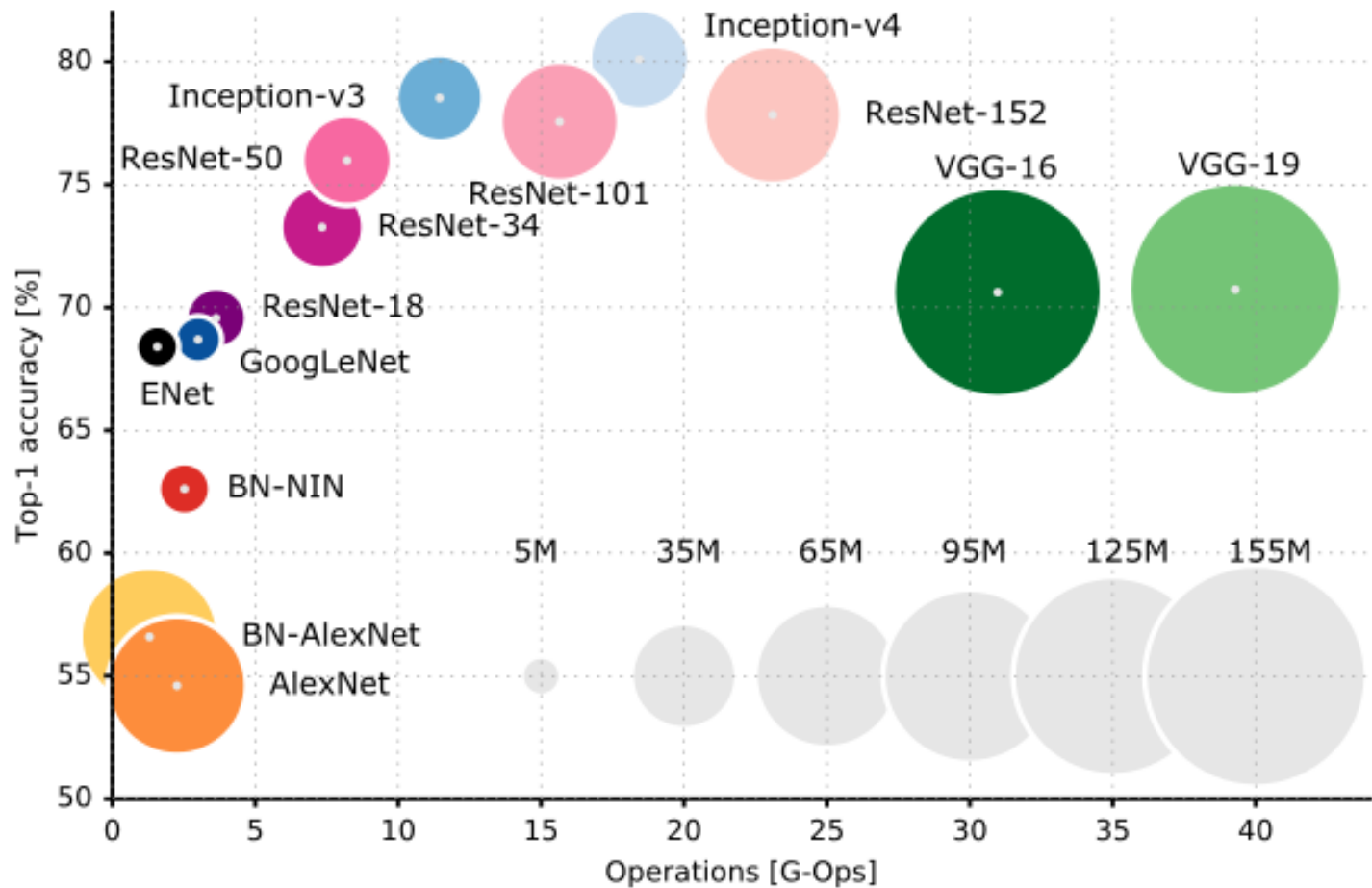


ResNet, **152 layers**
(ILSVRC 2015)





Bigger not better: innovations typically reduce parameters, despite deeper nets





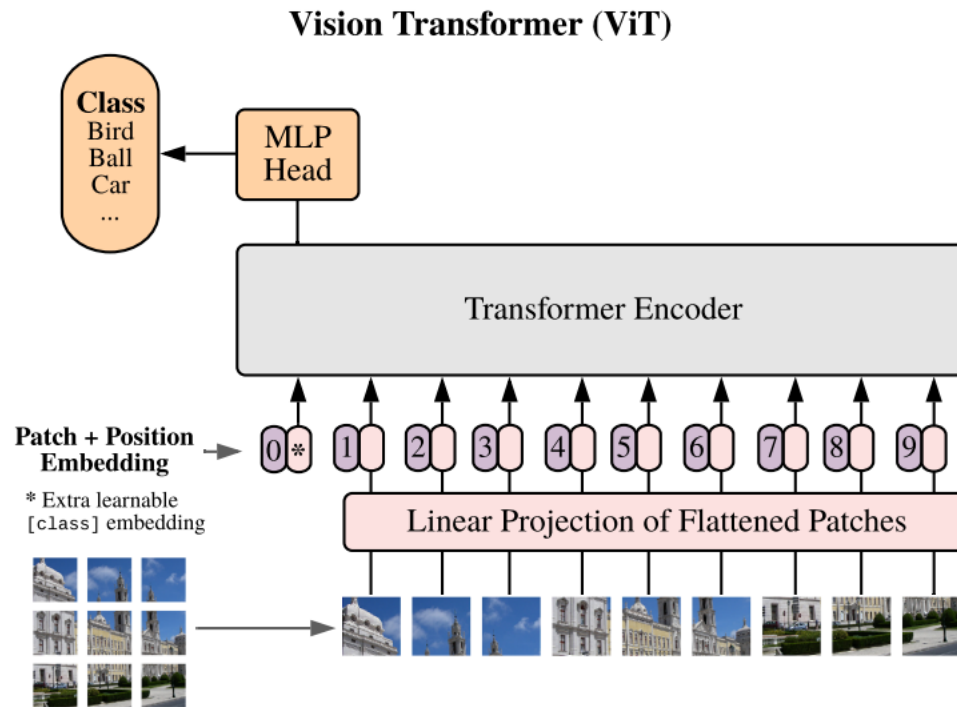
Key ideas of CNN Architectures



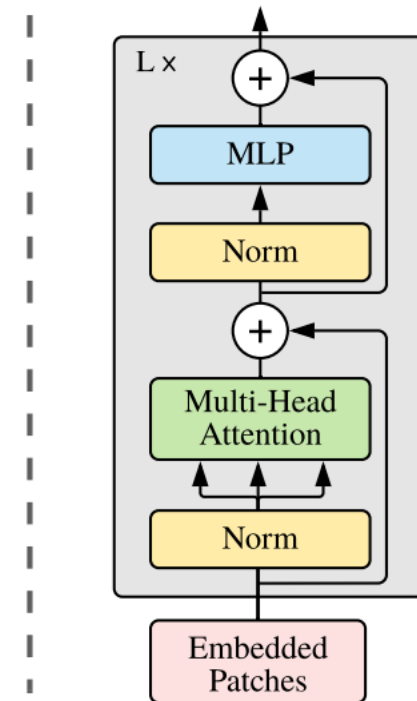
- Convolutional layers
 - Same local functions evaluated everywhere → much fewer parameters
- Pooling
 - Larger receptive field and translational invariance
- ReLU: maintain a gradient signal over large portion of domain
- Limit parameters
 - Sequence of 3x3 filters instead of large filters (also encodes that local pixels are more relevant)
 - 1x1 convs to reduce feature dimensions
- Skip network
 - Prevents having to maintain early layers (just add residual)
 - Acts as ensemble



Vision Transformer



Transformer Encoder

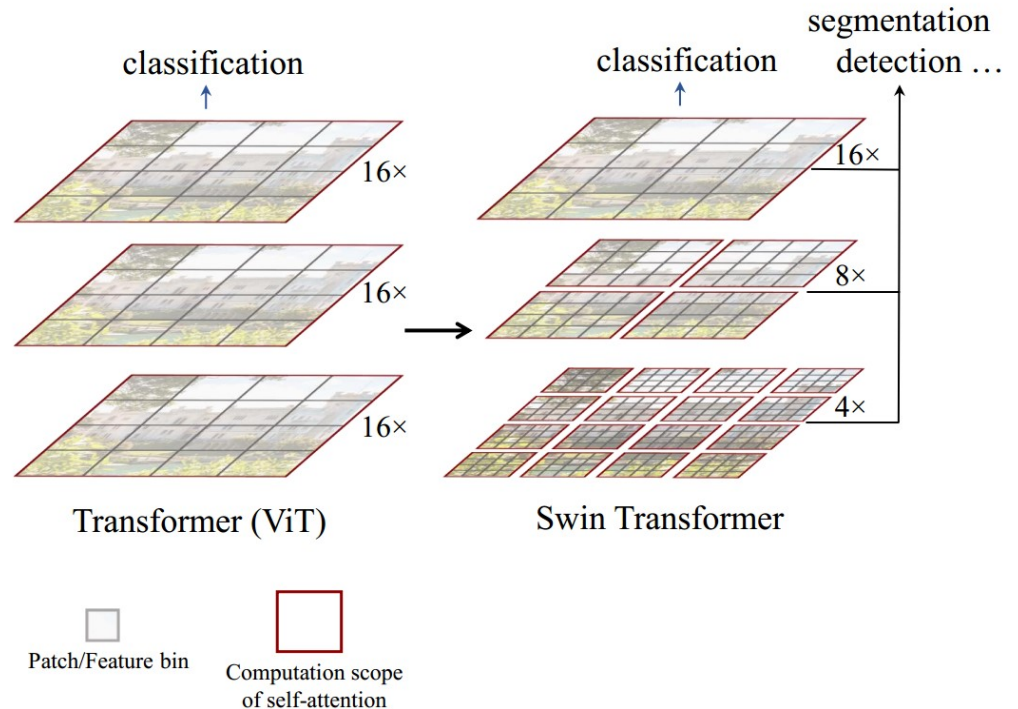




Swin Transformer



- Transformer
 - Strong modeling power
- + good priors for visual modeling
 - Hierarchy
 - Locality
 - Translational invariance





ViT-22B (2023)



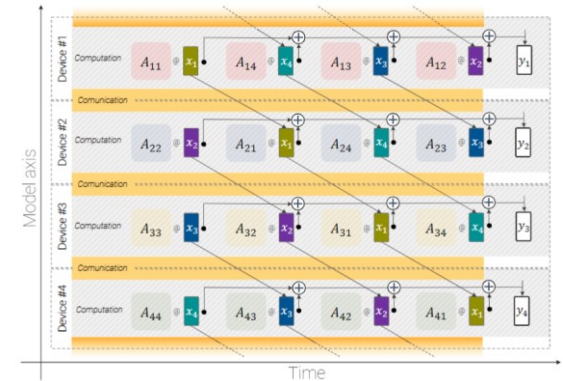
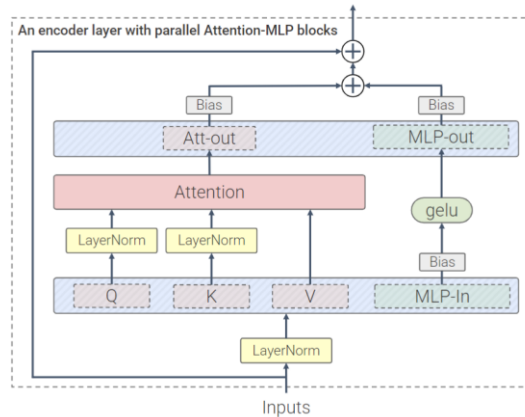
Scaling Vision Transformers to 22 Billion Parameters

Mostafa Dehghani* Josip Djolonga* Basil Mustafa* Piotr Padlewski* Jonathan Heek*
 Justin Gilmer Andreas Steiner Mathilde Caron Robert Geirhos Ibrahim Alabdulmohsin
 Rodolphe Jenatton Lucas Beyer Michael Tschannen Anurag Arnab Xiao Wang
 Carlos Riquelme Matthias Minderer Joan Puigcerver Utku Evci Manoj Kumar
 Sjoerd van Steenkiste Gamaleldin F. Elsayed Aravindh Mahendran Fisher Yu
 Avital Oliver Fantine Huot Jasmijn Bastings Mark Patrick Collier Alexey A. Gritsenko
 Vighnesh Birodkar Cristina Vasconcelos Yi Tay Thomas Mensink Alexander Kolesnikov
 Filip Pavetić Dustin Tran Thomas Kipf Mario Lučić Xiaohua Zhai Daniel Keysers
 Jeremiah Harmsen Neil Houlsby*
 Google Research

Name	Width	Depth	MLP	Heads	Mio. Param	GFLOPs	
						224 ²	384 ²
s/28	256	6	1024	8	5.4	0.7	2.0
s/16	256	6	1024	8	5.0	2.2	7.8
S/32	384	12	1536	6	22	2.3	6.9
Ti/16	192	12	768	3	5.5	2.5	9.5
B/32	768	12	3072	12	87	8.7	26.0
S/16	384	12	1536	6	22	9.2	31.2
B/28	768	12	3072	12	87	11.3	30.5
B/16	768	12	3072	12	86	35.1	111.3
L/16	1024	24	4096	16	303	122.9	382.8
g/14	1408	40	6144	16	1011	533.1	1596.4
G/14	1664	48	8192	16	1843	965.3	2859.9

Name	Width	Depth	MLP	Heads	Params [M]
ViT-G	1664	48	8192	16	1843
ViT-e	1792	56	15360	16	3926
ViT-22B	6144	48	24576	48	21743

Model	IN	ReaL	INv2	ObjectNet	IN-R	IN-A
<i>224px linear probe (frozen)</i>						
B/32	80.18	86.00	69.56	46.03	75.03	31.2
B/16	84.20	88.79	75.07	56.01	82.50	52.67
ALIGN (360px)	85.5	-	-	-	-	-
L/16	86.66	90.05	78.57	63.84	89.92	67.96
g/14	88.51	90.50	81.10	68.84	92.33	77.51
G/14	88.98	90.60	81.32	69.55	91.74	78.79
e/14	89.26	90.74	82.51	71.54	94.33	81.56
22B	89.51	90.94	83.15	74.30	94.27	83.80
<i>High-res fine-tuning</i>						
L/16	88.5	90.4	80.4	-	-	-
FixNoisy-L2	88.5	90.9	80.8	-	-	-
ALIGN-L2	88.64	-	-	-	-	-
MaxViT-XL	89.53	-	-	-	-	-
G/14	90.45	90.81	83.33	70.53	-	-
e/14	90.9	91.1	84.3	72.0	-	-





Today' class



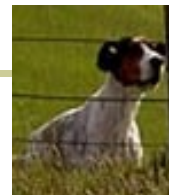
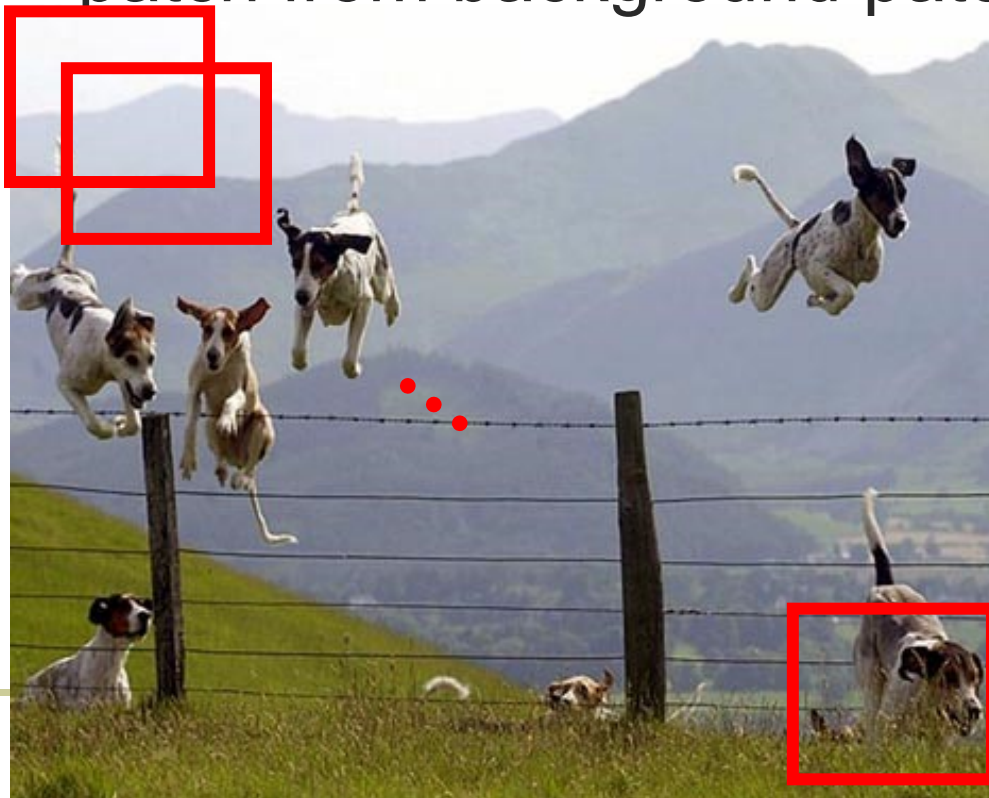
- Overview of image categorization
- Spatial pyramids bag-of-words categorizer
- Deep convolutional neural networks (CNNs)
- **Object detection**



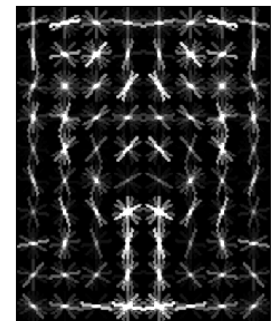
Object Category Detection



- Focus on object search: “Where is it?”
- Build templates that quickly differentiate object patch from background patch



Dog Model



**Object or
Non-Object?**



General Process of Object Recognition



Specify Object Model



Generate
Hypotheses



Score Hypotheses



Resolve Detections

What are the object parameters?



Specifying an object model

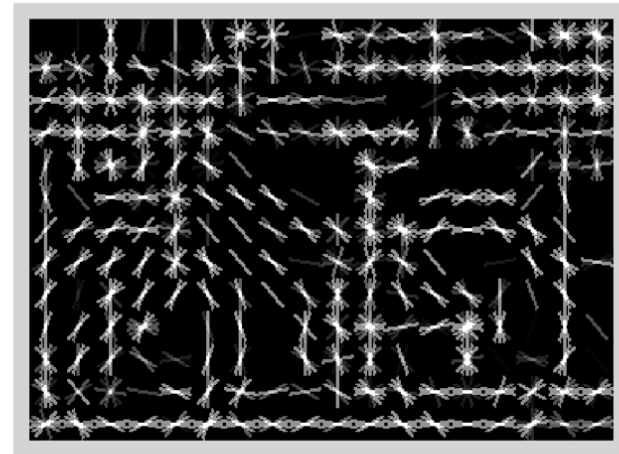


1. Statistical Template in Bounding Box

- Object is some (x,y,w,h) in image
- Features defined wrt bounding box coordinates



Image



Template Visualization

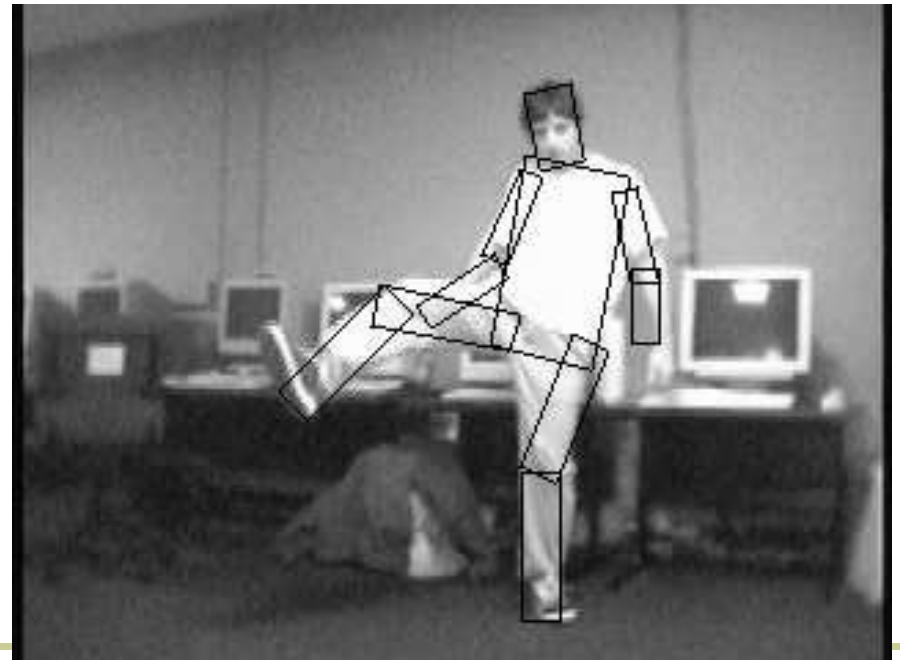
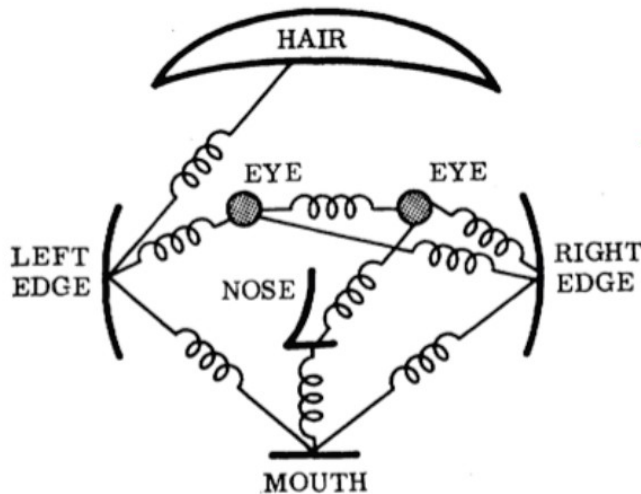


Specifying an object model



2. Articulated parts model

- Object is configuration of parts
- Each part is detectable



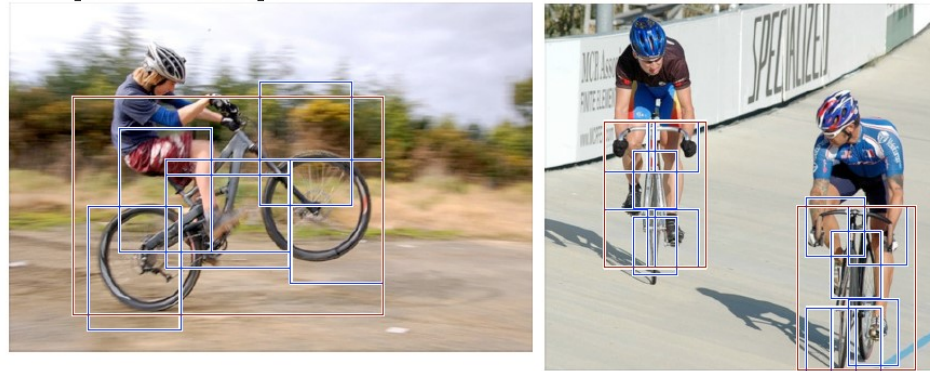


Specifying an object model

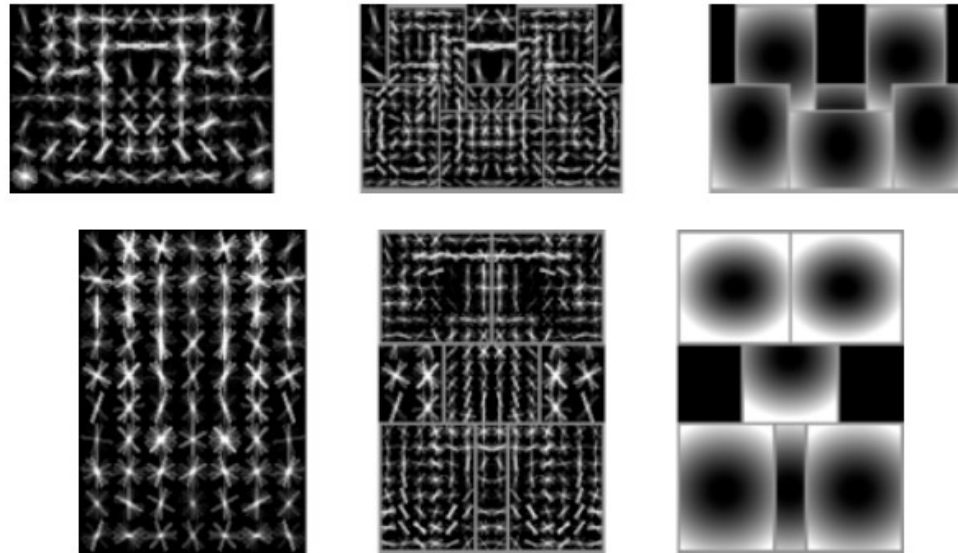


3. Hybrid template/parts model

Detections



Template Visualization



root filters
coarse resolution

part filters
finer resolution

deformation
models



General Process of Object Recognition



Specify Object Model



Generate
Hypotheses

Propose an alignment of the
model to the image



Score Hypotheses



Resolve Detections



Generating hypotheses



1. Sliding window

- Test patch at each location and scale





Generating hypotheses



1. Sliding window

- Test patch at each location and scale





Generating hypotheses

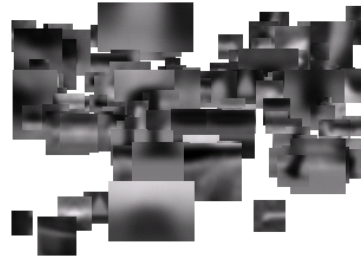


2. Voting from patches/keypoints

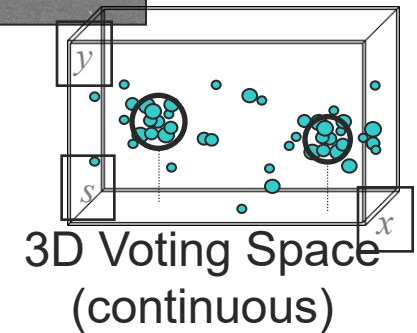
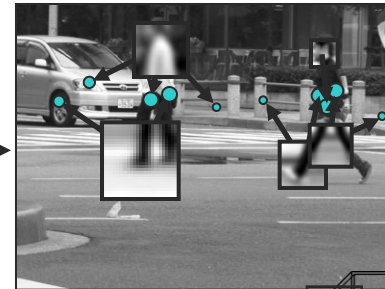
Interest Points



Matched Codebook Entries



Probabilistic Voting

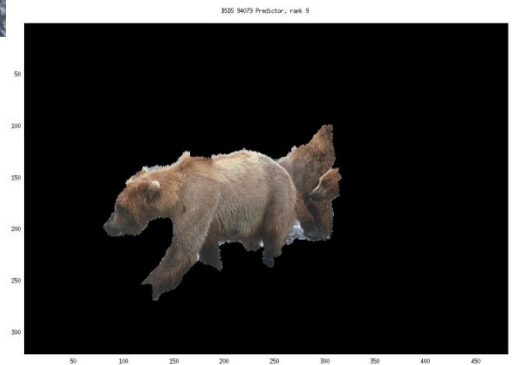
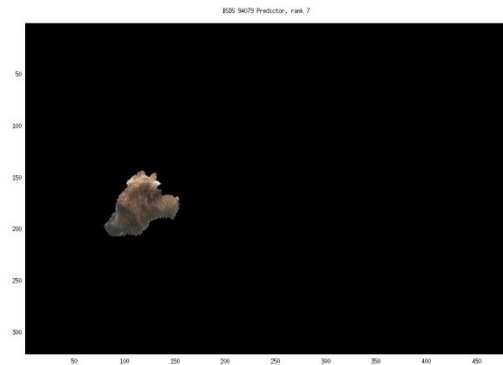
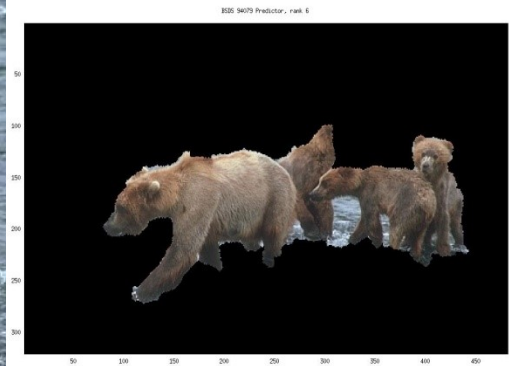
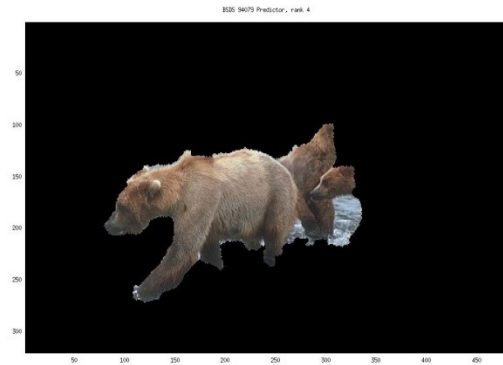




Generating hypotheses



3. Region-based proposal





Sliding window vs. region proposals



Sliding window

- Comprehensive search over position, scale (sometimes aspect, though expensive)
- Typically 100K candidates
- Simple
- Speed boost through convolution often possible
- Repeatable
- Even with many candidates, may not be a good fit to object

Region proposals

- Search over regions guided by image contours/patterns with varying aspect/size
- Typically 2-10K candidates
- Random (not repeatable)
- Requires a preprocess (currently 1-5s)
- Often requires resizing patch to fit fixed size
- More likely to provide candidates with very good object fit



General Process of Object Recognition



Specify Object Model



Generate
Hypotheses



Score Hypotheses



Resolve Detections

Currently CNN features and
classifiers



General Process of Object Recognition



Specify Object Model



Generate
Hypotheses



Score Hypotheses



Resolve Detections

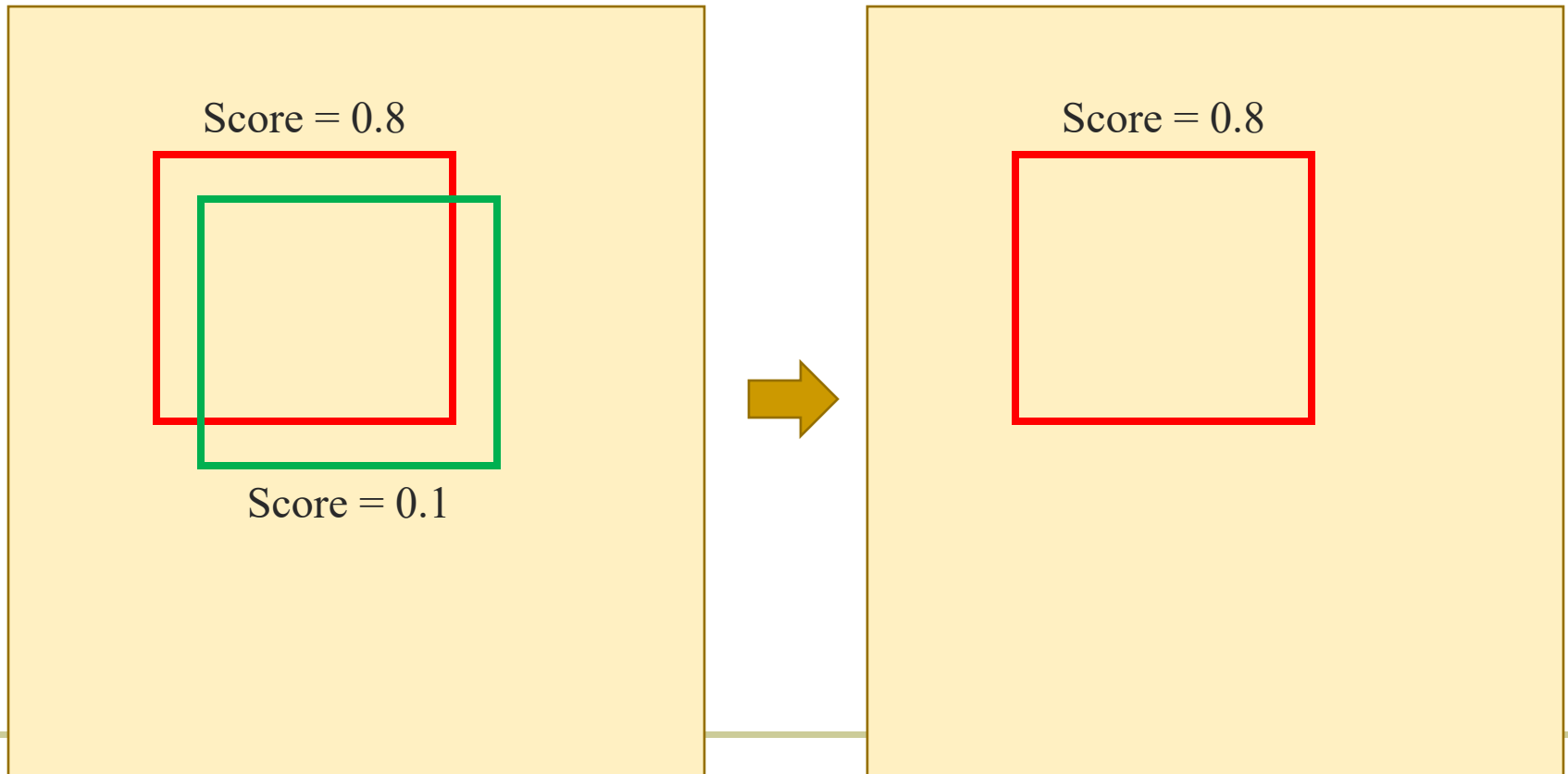
Optionally, rescore each proposed
object based on whole set



Resolving detection scores



1. Non-max suppression

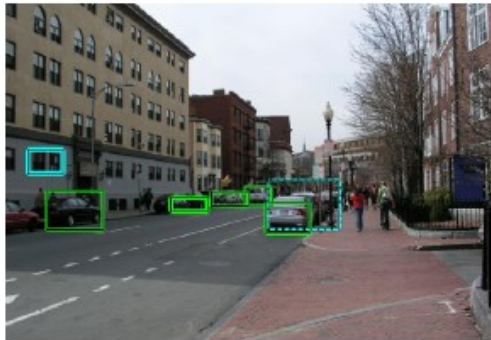




Resolving detection scores



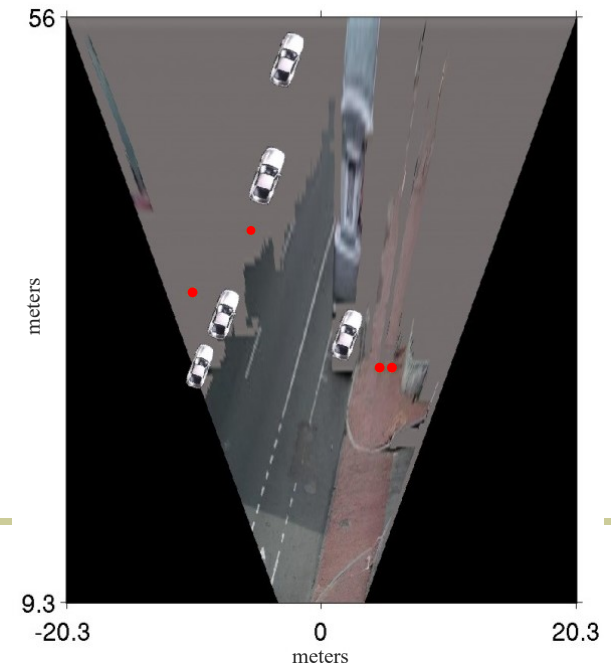
2. Context/reasoning



(g) Car Detections: Local



(h) Ped Detections: Local





R-CNN (Girshick et al. CVPR 2014)

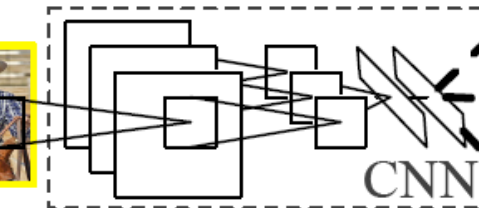


1. Input image



2. Extract region proposals (~2k)

warped region



3. Compute CNN features

aeroplane? no.

⋮

person? yes.

⋮

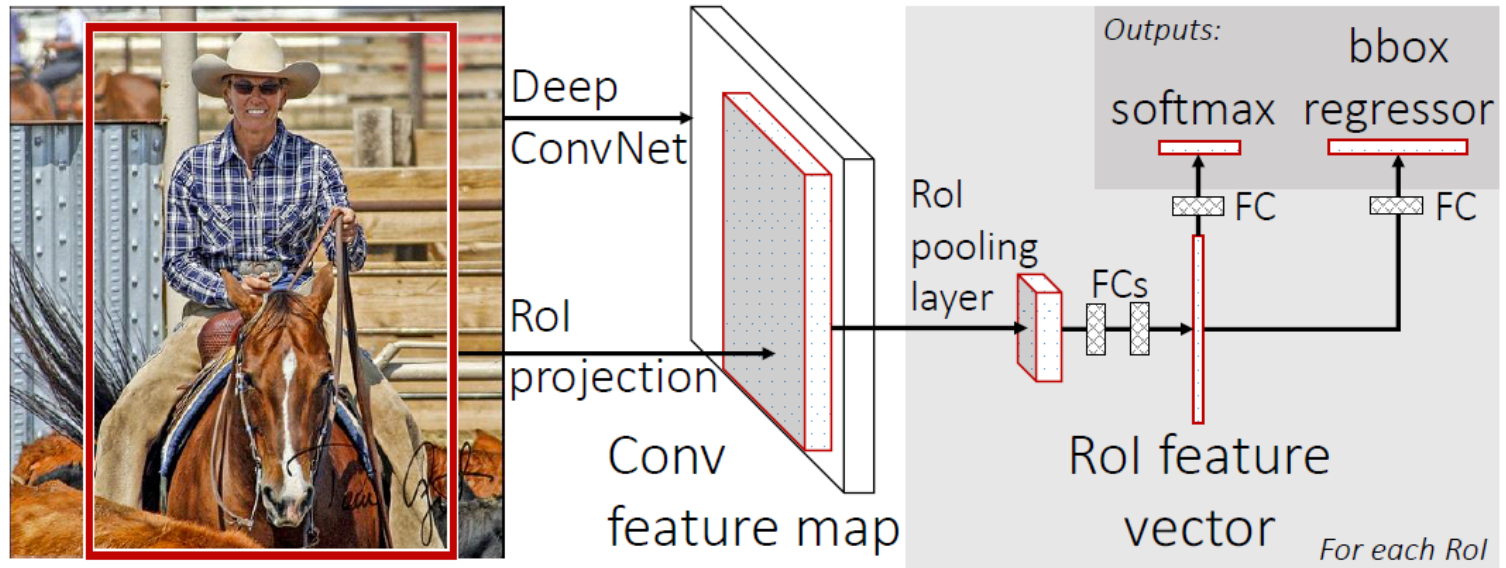
tvmonitor? no.

4. Classify regions

- Replace sliding windows with “selective search” region proposals (Uijilings et al. IJCV 2013)
- Extract rectangles around regions and resize to 227x227
- Extract features with fine-tuned CNN (that was initialized with network trained on ImageNet before training)
- Classify last layer of network features with SVM



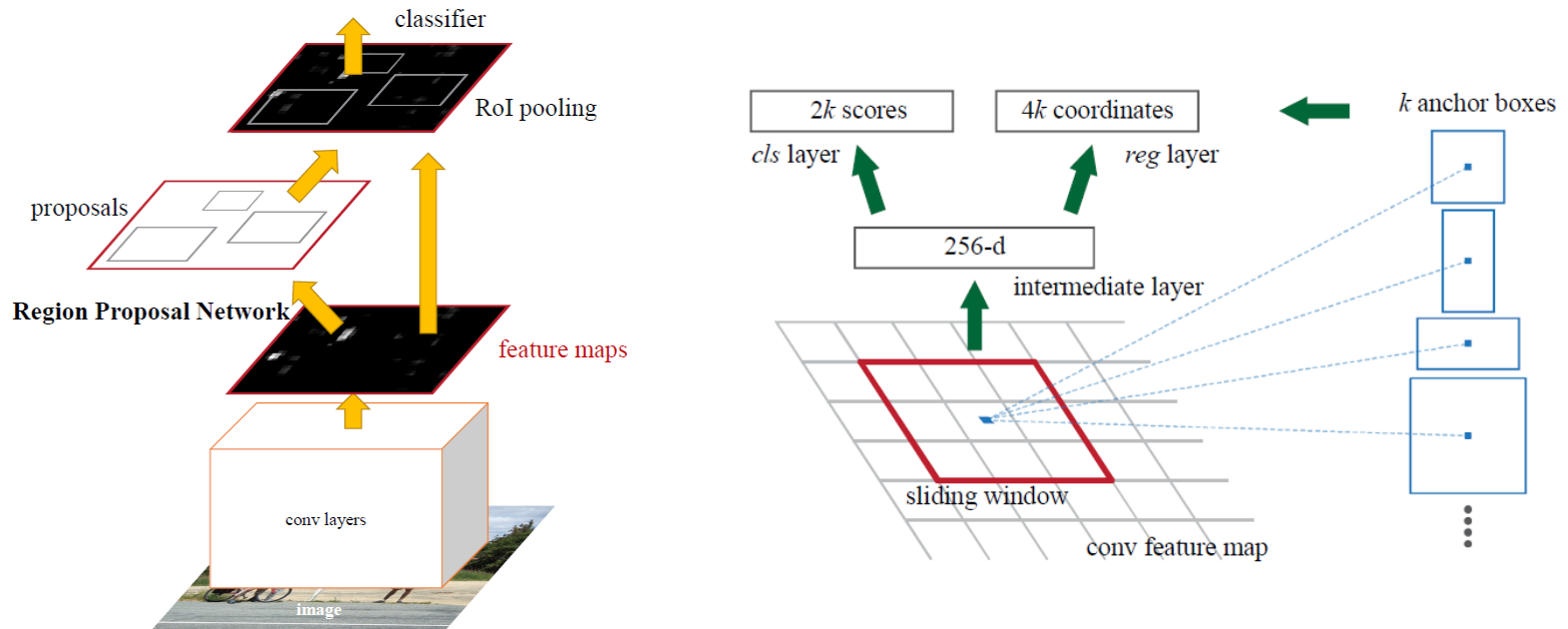
Fast R-CNN – Girshick 2015



- Compute CNN features for image once
- Pool into 7x7 spatial bins for each region proposal, output class scores and regressed bboxes
- 100x speed up of R-CNN (0.02 – 0.1 FPS → 0.5-20 FPS) with similar accuracy



Faster R-CNN – Ren et al. 2016



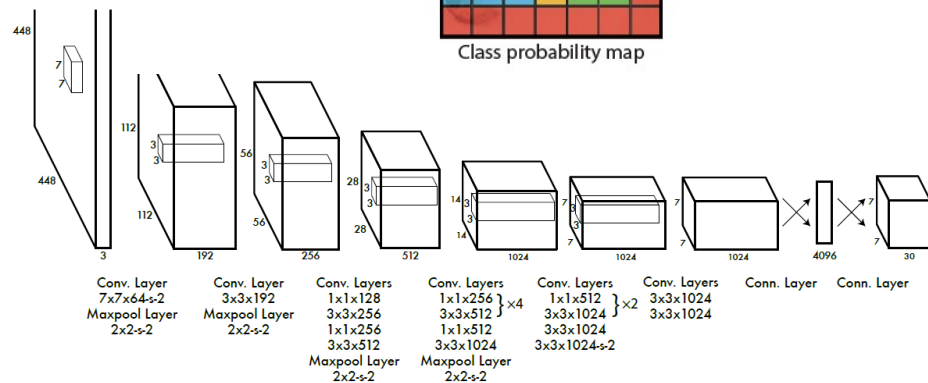
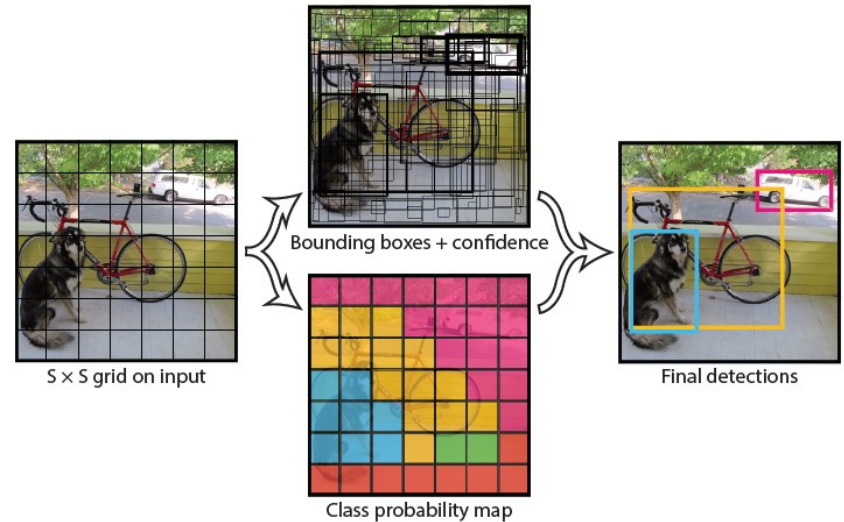
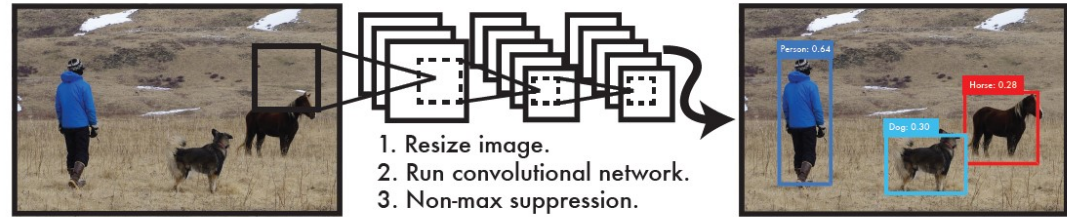
- Convolutional features used for generating proposals and scoring
 - Generate proposals with “objectness” scores and refined bboxes for each of k “anchors”
 - Score proposals in same way as Fast R-CNN
- Similar accuracy to Fast R-CNN with 10x speedup



YOLO – Redmon et al. 2016



1. CNN produces 4096 features for 7x7 grid on image (fully conv.)
 2. Each cell produces a score for each object and 2 bboxes w/ conf
 3. Non-max suppression
- 7x speedup over Faster RCNN (45-155 FPS vs. 7-18 FPS)
 - Some loss of accuracy due to lower recall, poor localization





Yolo v2 – Redmon et al. 2017



- Batch normalization
- Pre-train on higher resolution ImageNet
- Use and improve anchor box idea from Faster RCNN
- Train at multiple resolutions
- Very good accuracy, very fast

	YOLO								YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓				
new network?					✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓
location prediction?						✓	✓	✓	✓
passthrough?							✓	✓	✓
multi-scale?								✓	✓
hi-res detector?									✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8	78.6

Detection Frameworks	Train	mAP	FPS
Fast R-CNN [5]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[15]	2007+2012	73.2	7
Faster R-CNN ResNet[6]	2007+2012	76.4	5
YOLO [14]	2007+2012	63.4	45
SSD300 [11]	2007+2012	74.3	46
SSD500 [11]	2007+2012	76.8	19
YOLOv2 288 × 288	2007+2012	69.0	91
YOLOv2 352 × 352	2007+2012	73.7	81
YOLOv2 416 × 416	2007+2012	76.8	67
YOLOv2 480 × 480	2007+2012	77.8	59
YOLOv2 544 × 544	2007+2012	78.6	40

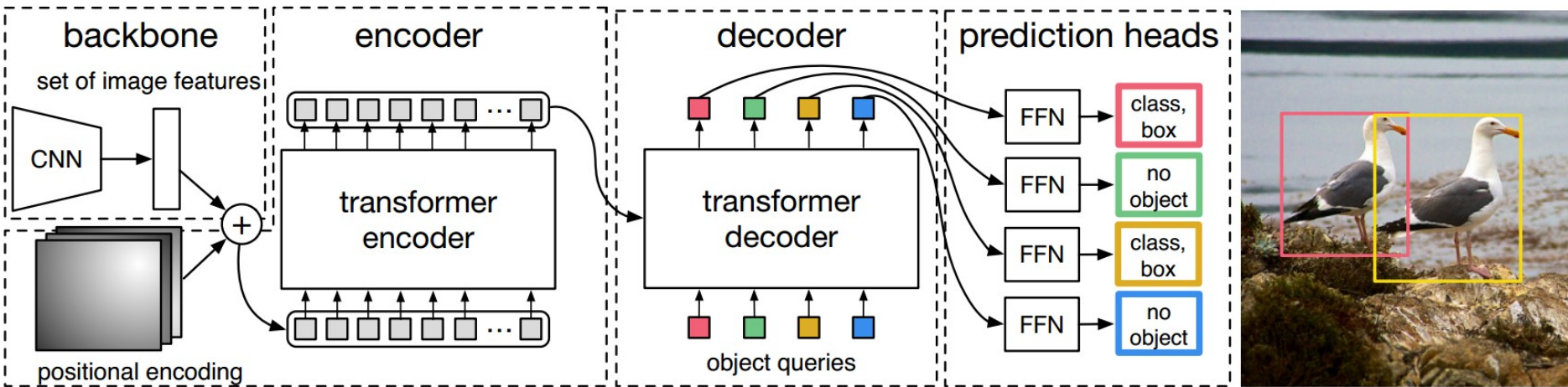
<https://youtu.be/VOC3huqHrss>



Query Based Detection (DETR)



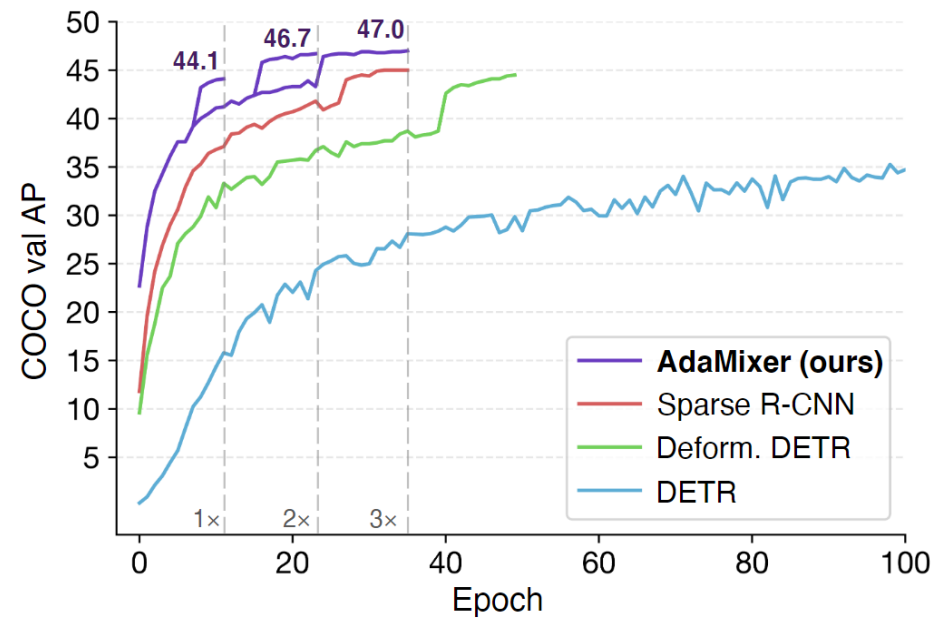
- End-to-End object detection without using priors



$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}),$$



AdaMixer (CVPR 2022)





AdaMixer (CVPR 2022)

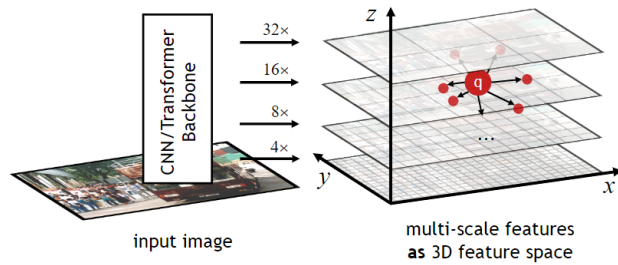


Figure 2. **3D feature sampling process.** A query first obtains sampling points in the 3D feature space and then perform 3D interpolation on these sampling points.

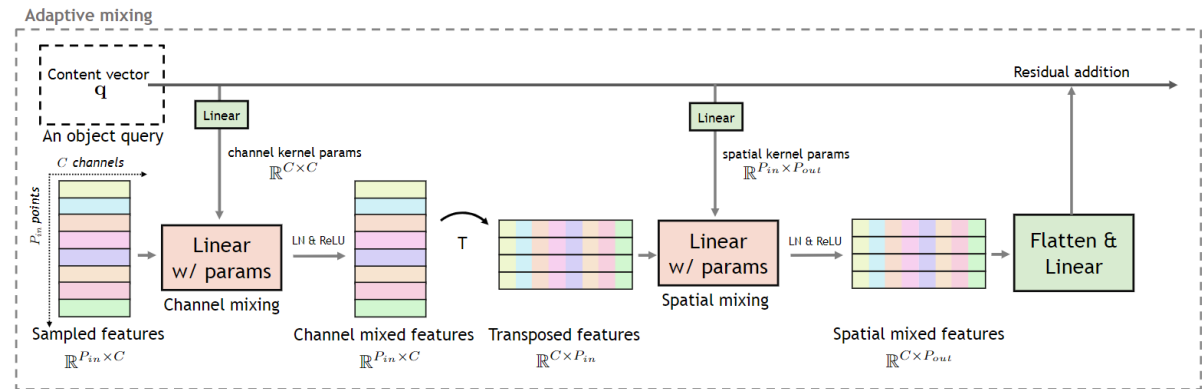


Figure 3. **Adaptive mixing procedure** between an object query and sampled features. The object query first generates adaptive mixing weights and then apply these weights to mix sampled features in the channel and spatial dimension. Note that for clarity, we demonstrate adaptive mixing for one sampling group.



AdaMixer (CVPR 2022)

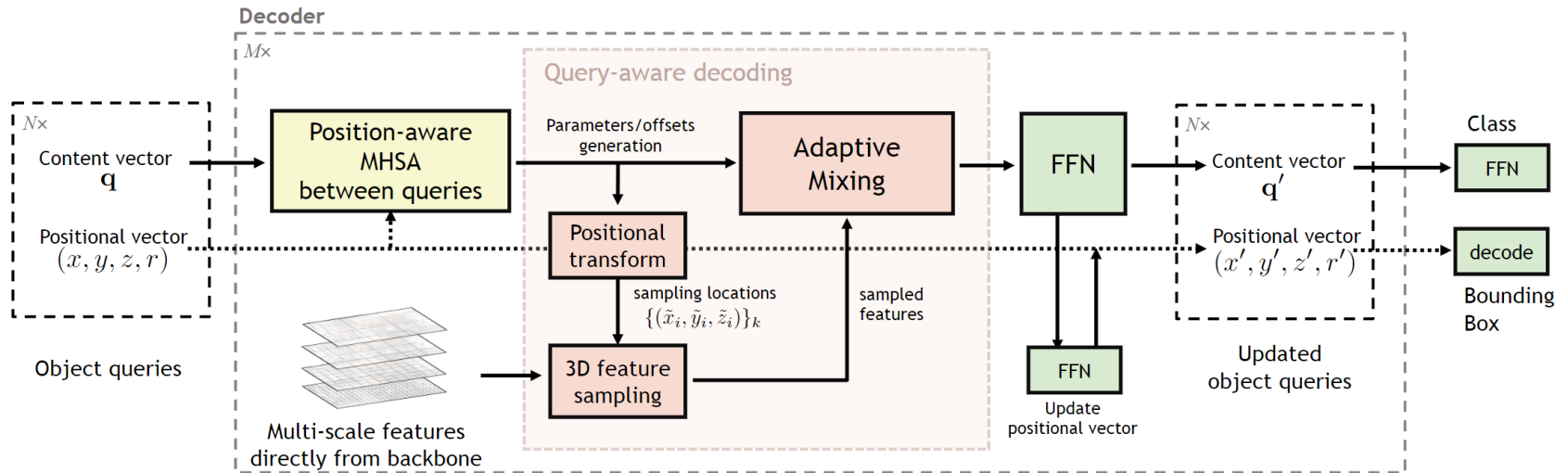


Figure 4. **Our decoder structure** of the AdaMixer. There are two operator streams on a query: one on its content vector $\mathbf{q}$ (the solid horizontal line) and one on its positional vector (x, y, z, r) (the dashed horizontal line). Each operator on the content vector in the decoder is followed by a residual addition and LayerNorm.

Pixel2Seq

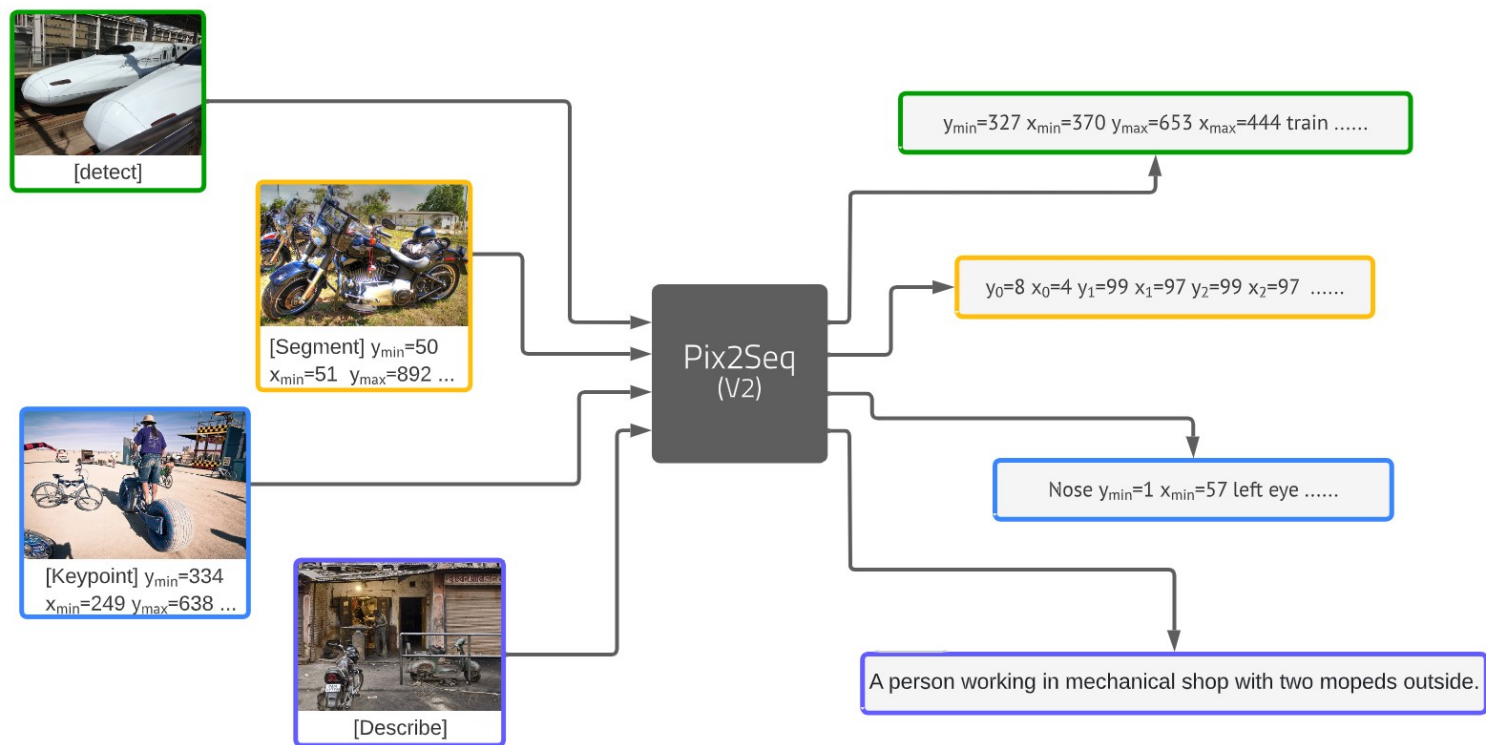


Figure 1: An illustration of the proposed framework. An image and a sequence of task prompt is given, the model produce a sequence of discrete tokens corresponding to the desired output.



Influential Works in Detection



■ Sung-Poggio (1994, 1998) : ~2412 citations

- Basic idea of statistical template detection (I think), bootstrapping to get “face-like” negative examples, multiple whole-face prototypes (in 1994)

■ Rowley-Baluja-Kanade (1996-1998) : ~4953

- “Parts” at fixed position, non-maxima suppression, simple cascade, rotation, pretty good accuracy, fast

■ Schneiderman-Kanade (1998-2000,2004) : ~2600

- Careful feature/classifier engineering, excellent results, cascade

■ Viola-Jones (2001, 2004) : ~27,000

- Haar-like features, Adaboost as feature selection, hyper-cascade, very fast, easy to implement

■ Dalal-Triggs (2005) : ~18000

- Careful feature engineering, excellent results, HOG feature, online code

■ Felzenszwalb-Huttenlocher (2000): ~2100

- Efficient way to solve part-based detectors

■ Felzenszwalb-McAllester-Ramanan (2008,2010): ~10788

- Excellent template/parts-based blend

■ Girshick-Donahue-Darrell-Malik (2014) : ~19788

- Region proposals + fine-tuned CNN features (marks significant advance in accuracy over hog-based methods)

■ Redmon, Divvala, Girshick, Farhadi (2016): ~16386

- Refine and simplify RCNN++ approach to predict directly from last conv layer